

Master's Thesis

Network Security Group, Department of Computer Science, ETH Zurich

Intent-Driven Network Control in SCION

**From Single-Path Selection to Joint
Application-Multipath Co-Optimization**

by Octave Charrin

Spring 2026

ETH student ID: 24-937-385
E-mail address: ocharrin@ethz.ch

Supervisors: Prof. Dr. Adrian Perrig
Liwen Xu

Date of submission: August 25, 2026

Abstract

The Internet forwards packets without regard for why they were sent, so the network optimizes a target that rarely matches what an application actually values. Path-aware architectures such as SCION change this by letting endpoints choose among multiple end-to-end paths, placing the path decision in the hands of the one party that knows the application’s intent. This thesis studies how to make that choice well: how to select the best path – or the best combination of paths – given both the application’s intent and the current state of the network. We build two learned controllers to answer it, at two granularities: a single-path selector for general flows, and a joint controller for real-time video that decides how much traffic each path carries *and* what the application produces. Both take a single idea from recent congestion-control work – that application intent should be a first-class input which reshapes a network mechanism rather than a constant compiled into it – and carry it one layer deeper than it has been taken before.

The strongest result is that the two decisions cannot be made well apart. A two-timescale pair of reinforcement-learning agents co-optimizing a video call’s encoding bitrate and its per-path split over Multipath QUIC beats all seven comparison points under all three application intents, including a WebRTC-style deployed stack, by 0.065 to 0.143 QoE – three to seven times the spread between training runs of the same method – and removing either agent forfeits most of the achievable quality. What makes this possible is architectural rather than incidental: a permutation-equivariant head that scores each path with shared weights, so one policy serves any number of paths. Substituting a conventional fixed-dimension head costs 0.231 QoE and drops the system from first place in nine of nine evaluation cells to none, which makes it the largest measured effect in this thesis and inverts the usual trade, since the dynamic-input design is also less than half the size of the fixed one it replaces. The same architecture carries the single-path study, where it raises goodput by 41% over a fixed-action agent and brings a learned selector within 0.25% of a per-context oracle while probing one path per decision instead of thirty.

Intent conditioning is the thread that connects the two, and its honest form is a design lemma with a measured boundary. We show analytically that *where* the conditioning enters decides whether it has any effect at all: a signal reaching only the terms shared by every candidate cancels under the comparison that selects a path, exactly and on the entire action when that comparison is a softmax, however much the policy is trained on it. Routing it into the per-candidate comparison instead lets one policy serve a range of intents without retraining, including intents it never trained on, and in the multipath setting lets one policy

replace three separately trained specialists at a third of the training cost with no penalty detectable at the resolution our experiments achieve. But the guarantee is narrower than the behavior. The cancellation is exact only for scorers additively separable in the conditioning, and we measure three ways an ordinary deployed architecture violates that premise, so that a trained nonlinear encoder routes intent past a cancellation the theory says should stop it. The principle survives as a statement about which routings carry a guarantee, not which happen to work. Both settings also bound how much such conditioning can be worth: intents must genuinely conflict, and in the single-path environment loss is near zero on 99% of selections, leaving goodput against latency as the one axis on which alignment can be demonstrated.

We report where the results do not reach, and two of those findings are more useful than what they replaced. What governs the payoff of joint control is not whether the network is moving but how unevenly capacity is spread across the paths and how strictly the application prices delay – a stationary but asymmetric network defeats application-only adaptation as thoroughly as a churning one. And on a packet-level backend the ranking transfers for three of six independently trained policies and not the other three, with analytical-backend score uncorrelated with which, so a cheap surrogate can be an adequate training environment while being useless for selecting among the policies it produces. Four results in this thesis were reported from a single training run and did not survive being run three times; we report that pattern as a result, because on this problem the spread between training runs exceeds the confidence interval over evaluation episodes by an order of magnitude. Together the two parts argue that on a path-aware Internet, application intent should steer every controllable decision between the application and the wire, and they chart what remains to make such control deployable.

[TODO: one scope sentence to add once you are happy with the wording above: the multipath study assumes the path multiplicity a path-aware architecture provides rather than running on a SCION data plane – there is no beaconing, path store, or segment composition in Chapter 5, and the six-path topology is hand-specified (Table A.4). Given the title, a SCION-group reader will ask, and it is better answered here in one clause than discovered in §5.2.]

Acknowledgements

[TODO: Personal acknowledgements to be written before submission.] I would like to thank my supervisor Liwen Xu for his guidance throughout this thesis, and Prof. Dr. Adrian Perrig for the opportunity to carry out this work in the Network Security Group at ETH Zurich. I am grateful to the members of the group for their feedback, and to the authors of the SCION DQN path-selection and Mortise systems, whose work this thesis builds upon. **[TODO: add thanks to family, friends, and anyone else who supported the work.]**

Contents

Abstract	iii
Acknowledgements	v
1 Introduction	1
1.1 Motivation and Problem Statement	1
1.1.1 The Research Challenge	1
1.1.2 Key Insights and Approach	2
1.1.3 Guiding Questions	3
1.2 Contributions	3
1.3 Organization	5
2 Background & Technical Foundations	7
2.1 The SCION Path-Aware Architecture	7
2.2 Quality of Experience and Transport Mechanisms	8
2.2.1 Quality of experience for real-time video	8
2.2.2 Adaptive bitrate and learned control	9
2.2.3 Multipath QUIC	9
2.3 Deep Reinforcement Learning Machinery	9
2.3.1 Markov decision processes	9
2.3.2 Deep Q-networks and refinements	9
2.3.3 Soft actor-critic	10
2.3.4 Variable and structured action spaces	10
2.4 Goal-Conditioned Reinforcement Learning	10
2.4.1 Conditioning a policy on its objective	10
2.4.2 Where the conditioning signal enters	11
3 Problem Statement & Desired Properties	13
3.1 Problem Formulation	13
3.2 Desired Properties	14
3.3 Assumptions & Scope	15
4 Motivating Study: Intent-Conditioned Single-Path Selection	17
4.1 Selection Framework: Environment, Decision, and Scoring	17
4.1.1 Starting Point and Its Limitations	17
4.1.2 Improved SCION Simulation Environment	18
4.1.3 Decision Problem: State, Action, Reward	21

4.1.4	Handling a Variable Path Set: The Scoring Architecture . . .	22
4.2	Where Intent Conditioning Must Enter	23
4.2.1	Where conditioning cancels	23
4.2.2	Conditioning the per-path comparison	23
4.2.3	Training across intents	24
4.2.4	Relation to Mortise	25
4.3	Implementation	25
4.3.1	Agent Family	25
4.3.2	Evaluation Pipeline	26
4.3.3	Baselines and Probing Accounting	27
4.4	Experimental Evaluation and Intent Alignment	28
4.4.1	Experimental Setup	28
4.4.2	Where Must Intent Enter? The Conditioning Ablation	30
4.4.3	Intent Alignment	33
4.4.4	Zero-Shot Generalization to Unseen Intents	36
4.4.5	Variable Path Sets	38
4.4.6	Probing Overhead	39
4.5	The Single-Path Performance Ceiling	41
5	Joint Application–Network Optimization over Multipath	45
5.1	System Design: Co-Optimizing Rate and Traffic Split	46
5.1.1	Motivation and Strawmen	46
5.1.2	Two-Timescale Hierarchical Agents	46
5.1.3	Observations, Actions, and Rewards	47
5.1.4	Coupling the Two Agents	48
5.1.5	Continuous Scoring over a Variable Path Set	49
5.1.6	Where Intent Must Enter a Continuous Allocation	49
5.2	Implementation: Multipath Real-Time Video Testbed	52
5.2.1	Data-Plane Abstraction and Two Backends	52
5.2.2	Real-Time Video and Multipath Model	53
5.2.3	Agents and QoE Surrogate	53
5.2.4	Known Gaps	54
5.3	Experimental Evaluation: Regimes of Value	54
5.3.1	Setup, Baselines, and Statistical Protocol	55
5.3.2	Does the Learned Pair Win, and Under Which Intent?	57
5.3.3	When Does Splitting Help? The Governing Quantity	59
5.3.4	Do the Intents Produce Different Policies?	60
5.3.5	One Conditioned Policy, or Three Specialists?	61
5.3.6	Does It Survive a Packet-Level Transport Stack?	65
5.3.7	Ablations	68
5.3.8	What the Retractions Have in Common	71
5.3.9	Summary	71

6	Discussion and System Synthesis	75
6.1	Synthesis of Findings	75
6.1.1	Interpretation	75
6.1.2	Implications	77
6.1.3	Limitations	77
6.2	System Analysis	78
6.2.1	Overhead	78
6.2.2	Scalability	79
6.2.3	Generalization	80
6.3	Deployment Considerations and Trust Assumptions	81
6.3.1	Deployment Gaps	81
6.3.2	Trust Assumptions	81
7	Related Work	83
7.1	Learned Routing and Traffic Engineering	83
7.2	Learned Congestion Control and Multipath Scheduling	83
7.2.1	Mortise: intent-driven tuning of a transport mechanism	84
7.3	Adaptive Bitrate and QoE-Driven Control	84
7.4	Goal-Conditioned and Modulated Policies	85
7.5	Intent-Based Networking	85
8	Conclusion	87
	Bibliography	91
A	Reproducibility Details	95
A.1	Single-Path Selection: Reward Profiles	95
A.2	Single-Path Selection: Agent Hyperparameters	95
A.3	Multipath System: QoE Weights and SAC Hyperparameters	96
A.4	Supplementary Results	98
A.5	Code and Artifact Availability	99

List of Figures

4.1	Reward matrix R (intent told, objective scored) for the conditioned policy. Cell color is the reward relative to its column mean, so a red diagonal means that conditioning on an intent is the best way to score well under that same objective. The diagonal wins every column; the margin is large on the two objectives that conflict with the high-bandwidth default and small on the two that do not. Table 4.5 gives the underlying numbers.	35
4.2	Distributions of the chosen path’s metrics as the conditioned policy is told each intent, over a fixed set of decision contexts. The Low-Latency intent lowers the chosen latency and the Throughput intent raises the chosen goodput. Trust is scored with the canonical weights of Eq. (4.1) and is therefore maximized by the Low-Latency intent, not the Low-Loss one; the fourth panel reports loss exposure directly, since a boxplot of loss rate is a flat line at zero.	37
4.3	Behavior of the conditioned policies along the unseen intent segment of Eq. (4.5). Only the two dashed verticals were trained on; the shaded band is interpolation and the flanks are extrapolation. Chosen latency falls monotonically and smoothly, which is the signature of a learned mapping rather than of memorized endpoints. Value-Concat interpolates smoothly too, but traverses less than half as much behavior, as Proposition 4.1 predicts.	38
4.4	Mean regret against the best path in the presented candidate set, by set size (log scale, 10752 decisions per point, Balanced intent). One model per method, no retraining and no padding, across a 15-fold range in N . Every scoring agent stays two orders of magnitude below the flat DQN, which must address a fixed 30-way action head and degrades as N moves into the middle of its range.	40
4.5	Achieved goodput against probing cost per selection under the Balanced intent (top-left is better). The learned selector reaches the goodput of exhaustive widest-path selection at roughly 40× less probing, because it inspects the one path it selects rather than all 30. It is also the cheapest method on the plot: every load-blind heuristic must measure all 30 candidates to apply its rule, and so probes about 2.6× more than the learned selector while choosing worse.	41

- 4.6 Achieved goodput (left) and intent-weighted reward (right) against realized congestion, 1792 decisions per bin per method. The learned selector tracks the widest-path ceiling at every load, but that ceiling itself falls by about 25% from light to heavy congestion: past a point, no single path can carry the demand. The right panel is the bridging argument of this thesis in one picture – the selector’s *reward* stays flat while its goodput descends, because the reward measures selection quality and is blind to the ceiling by construction. The policy is doing its job; the action space is the limit. 43
- 5.1 The mechanism, within a single 30 s episode of the dynamic six-path scenario under interactive. Per-path goodput (a), smoothed RTT (b), and loss (c) are the state the transport agent observes; (d) is the split it produces. A path taken down by churn appears in (c) as loss saturating at 1.0 — path 0 at $\approx 3\text{--}5.5\text{ s}$ and again at $\approx 13\text{--}15.5\text{ s}$, path 4 at $\approx 7\text{--}11\text{ s}$ — and in every case the corresponding trace in (d) falls to zero over the same window and recovers when the path does. The agent also shades allocation away from the 40 ms latency trap (path 2, the upper curve in (b)) without abandoning it. Source: `rl-mpquic/runs/ab-prof2-evinteractive-trinteractive/interactive/seed-1000-t1`. . . . 73
- 5.2 Inference cost per decision, by method, on a 16-core CPU host with no GPU and with warm-up calls discarded. (a) Mean cost of the bitrate decision and of the split decision, on a log scale. (b) Distribution of the per-frame split decision, the design’s binding real-time constraint. The learned split costs 0.507 ms against a 33 ms frame budget at 30 fps. Source: `rl-mpquic/runs/ab-prof2-evinteractive-trinteractive/interactive/seed-1000-t1`. . . . 74
- A.1 Training curves for the single-path agents of §4.3.1: episode reward (25-episode moving average, left) and training loss (right) over the 533 or 666 episodes each agent was trained for. The four path-scoring agents converge to within 0.01 of one another at roughly +0.81, while the flat DQN plateaus near +0.05 with a *rising* loss despite the identical budget, data, and optimizer – the evidence that its deficit is architectural rather than a matter of training time. The simple scoring agent’s loss also diverges, which is why the enhanced variant is used throughout. 98

List of Tables

4.1	Link capacities in the evaluated topology, by origin. BRITE contributes a uniform 10 Gbit/s fabric; every capacity difference the selector can exploit is introduced by the SCION converter.	20
4.2	Intent-conditioning ablation. Mean intent-weighted reward under each of the four intents; <i>Adapt.</i> is the mean behavioral divergence of Eq. (4.4) (0 = intent-invariant path choice, 1 = a different path per intent), <i>Ent.</i> the mean choice entropy in bits, and <i>Params</i> the <i>Q</i> -network’s parameter count in thousands. The oracle is the per-context reward arg max under the intent being scored, so each column reads as a normalized optimality gap. Each reward cell aggregates 10752 decisions over 336 held-out hours from the single reference run that the rest of the chapter is also anchored on; Table 4.3 gives every method’s spread over five independent training seeds.	31
4.3	Seed variance over five training runs per method, evaluated on the same 10752 held-out contexts. Cells are mean [95% confidence interval]. Both of the chapter’s structural comparisons reproduce with disjoint intervals: flat against per-path scoring on Throughput, and value-stream against per-path conditioning on Low-Latency and on adaptivity.	34
4.4	The same ablation on 32 pairs across 32 distinct source ASes, none visited during training. Absolute rewards fall on Low-Latency, but so does the oracle’s, so the drop reflects these pairs’ available headroom rather than a generalization failure. The flat agent is the exception: its Throughput reward goes negative.	34
4.5	The reward matrix behind Fig. 4.1: mean reward when the policy is told the row intent and scored under the column objective, over 10752 contexts. Column maxima in bold; the diagonal attains all four. Note the very different column ranges, which the figure’s column-relative coloring deliberately hides.	36
4.6	Mean intent-weighted reward by method and intent, with measurement cost, over the same 10752 contexts. Column maxima in bold. No heuristic is strong under more than one objective: each collapses under the other’s. The learned selector is the only method that is at or near the top of every column, and it is the cheapest method in both probes and milliseconds.	42

- 5.1 The three application intents evaluated in §5.3, each a bundle of QoE weights (Eq. (5.1)), normalizers, and a frame deadline, keyed to ITU-T G.1010 use-case classes. Source: `rl-mpquic/configs/profiles/*.yaml`. The delay weight b falls and the deadline widens as the role relaxes; loss stays dominant for the two content-critical roles. These play the part the reward profiles of Table A.1 play in Chapter 4. 49
- 5.2 Behavioral divergence of the traffic split as the intent is swept with the decision context frozen, over a 21-point sweep on six paths. Divergence is the mean pairwise total-variation distance between the resulting splits, in $[0, 1]$ — the continuous analogue of Eq. (4.4), which counts distinct discrete choices. A scorer obeying Proposition 5.1 must read exactly zero under global-only conditioning. Source: `rl-mpquic/tests/test_intent_conditioning.py`. 51
- 5.3 Mean QoE (Eq. (5.1)) on the dynamic six-path scenario, by policy and application intent, over three training seeds $\times$ three evaluation seeds $\times$ five episodes. SD_{train} in brackets, omitted for the deterministic baselines that carry no learned component. Each column is evaluated under its own reward and deadline, so columns are not comparable to one another. Source: `rl-mpquic/runs/ab-prof2-ev*-tr*/ab_summary.json`. 58
- 5.4 Intent-transfer matrix: mean QoE of each intent-trained policy (rows) evaluated under each intent’s own reward and deadline (columns), on the dynamic scenario, over three training seeds $\times$ three evaluation seeds. SD_{train} in brackets. Read *down* a column, which fixes the evaluation intent. The diagonal is set in bold and the column winner underlined; they coincide only for interactive, and no column’s margin clears its own training-seed spread. Source: `rl-mpquic/runs/ab-prof2-ev*-tr*/ab_summary.json`. 61
- 5.5 A single intent-conditioned policy against the best specialist per intent. Bar = column maxima of Table 5.4. Both conditioned arms are single policies evaluated at all three intents; specialists are three policies. Six training seeds per conditioned arm, SD_{train} in brackets; σ is the pooled training-seed spread of the two arms being differenced. Source: `rl-mpquic/runs/ab-intent6-{pp,gl}*/ab_summary.json`. 62
- 5.6 Zero-shot transfer to the packet-level NS-3 backend: six mock-trained checkpoints evaluated on the NS-3 body without retraining, dynamic scenario, TCP subflows, 6×3 cells. SD_{train} is the spread across training seeds; it is the column that decides the reading. Note the inversion of single, mid-table on the mock and last here. Source: `rl-mpquic/runs/ab-ns3-6seed/ab_summary.json`. 66
- 5.7 Per-training-seed mean QoE on both backends, over three evaluation seeds each. The NS-3 column is bimodal with the path-only ablation (0.380) sitting in the gap, and that gap is what decides each seed’s verdict. The mock does not resolve the difference at all. Source: `rl-mpquic/runs/ab-ns3-6seed/, runs/ab-mock-6seed/`. 66

A.1	The six reward profiles (application intents) used in the single-path selection study (Chapter 4). w_1 weights goodput, w_2 trust, w_3 loss and w_4 delay within the trust term, and w_{probe} the probing penalty. All six are used in training on a stratified schedule; the four marked with a chapter label are also used in the evaluation of §4.4.	95
A.2	Hyperparameters for the single-path selection agents (Chapter 4). .	96
A.3	Soft actor-critic hyperparameters shared by the application and transport agents (§5.2.3). Source: <code>rl-mpquic/configs/dynamic.yaml</code> . .	97
A.4	The dynamic six-path scenario (§5.3.1): per-path bottleneck rate, propagation delay, and mean cross-traffic as a fraction of that rate. No path can carry full-quality video alone. Paths 5 and 6 form a shared-bottleneck group that degrades together. Source: <code>rl-mpquic/configs/dynamic.yaml</code>	97

1 Introduction

1.1 Motivation and Problem Statement

The Internet was designed to deliver packets, not experiences. A router forwards a datagram toward its destination without any notion of *why* the datagram was sent: whether it carries a frame of an interactive video call that will be worthless if it arrives late, a byte of a background software update that only cares about aggregate throughput, or a control message whose loss would be catastrophic. For decades this deliberate ignorance – the end-to-end principle – was a feature. Yet as applications have become more diverse and more demanding, the mismatch between what the network optimizes and what the application actually values has grown into one of the central inefficiencies of modern networking.

Path-aware Internet architectures such as SCION [13] change the terms of this problem. In SCION, endpoints discover *multiple* end-to-end paths to a destination and choose among them, rather than deferring blindly to whatever route BGP happens to install. This turns path selection from a network-internal decision into an application-facing one: for the first time, the entity that knows the application’s intent – the endpoint – also holds the lever that determines how traffic is carried. The fundamental question this thesis addresses is:

How should an endpoint choose the best path – or the best combination of paths – given both the intention of the application and the current state of the network?

1.1.1 The Research Challenge

Answering this question is hard for three compounding reasons:

1. **“Best” is not a fixed target:** A path optimal for a bulk transfer (high goodput, tolerant of latency) is a poor choice for a video call (which prizes low latency and jitter). Hand-tuned objectives fail across diverse workloads, and training a separate reinforcement learning (RL) policy for every conceivable objective does not scale.
2. **The action space is dynamic and unbounded:** The set of candidate paths differs across source–destination pairs and fluctuates over time as links appear, fail, congest, or recover. Standard policies assuming a fixed, enumerated action set cannot cope.

3. **State is partially and expensively observable:** Fresh per-path measurements incur active probing overhead. A practical policy must decide well while measuring little.

1.1.2 Key Insights and Approach

This thesis addresses these challenges through a unified principle inspired by recent transport-layer auto-tuning. The Mortise system [16] shows that network mechanisms can be steered by treating application *intent* as an explicit, dynamic input rather than a constant compiled into an algorithm. We transplant this principle across two sequential operational control layers:

1. Motivating Foundation: Intent-Aware Single-Path Selection. We first investigate how application intent can modulate discrete path choice for general network flows. Building on the DQN path selector of Bourak et al. [3], we introduce a permutation-equivariant *path-scoring* architecture that supports variable candidate path sets, and which turns out to carry most of the quantitative gain in this chapter: it raises goodput by 41% over a fixed-action agent, at less than half the parameter count. We then condition the agent on an explicit vector of application intent, and establish a *design lemma* governing where that conditioning may enter: *in comparison-based selection, a signal that reaches only the terms shared by every candidate cancels under the argmax and cannot change the chosen path, no matter how the policy is trained.* The algebra is one line; what makes it worth stating is that a plausible and widely-used routing — conditioning the value stream of a dueling head — sits exactly on the wrong side of it, and that the consequence is measurable, with such a policy re-ranking roughly three times less over five training seeds with disjoint intervals. Routing the intent into the *per-candidate* comparison instead lets a single policy serve diverse application intents without retraining, and interpolate to intents it never trained on. This study also exposes two limits: the environment offers only one axis on which intents genuinely conflict, and when congestion surges, even the best single path reaches an architectural performance ceiling.

2. Joint Application–Network Optimization over Multipath. Driven by the limitations of single-path selection, we expand our framework to co-optimize both the application and the network simultaneously. Leveraging SCION’s native path multiplicity and Multipath-QUIC [5] transport, we address interactive real-time video streaming by controlling both *what* the application produces (the video encoder’s target bitrate) and *how* the network carries it (the continuous per-path traffic split), via a two-timescale hierarchical reinforcement-learning system: a slow application agent for bitrate adaptation and a fast transport agent for path scheduling. This is the thesis’s best-evidenced claim. The pair beats all seven comparison points under all three application intents, including a WebRTC-style deployed stack, at margins three to seven times the spread between training runs

of the same method, and *neither decision can be made well alone* — removing either agent forfeits most of the achievable quality. The permutation-equivariant head is again what makes it possible rather than merely convenient: substituting a fixed-dimension one costs 0.231 QoE, the largest measured effect in this thesis, and drops the system from first in nine of nine evaluation cells to none.

Two further findings correct intuitions we began with. *What governs how much the joint controller earns is not whether the network is stationary*: it is how unevenly capacity is spread across the paths – whether an even split fits inside every path’s residual capacity – and how strictly the application prices delay. Path churn and congestion bursts matter because they destroy that evenness transiently, not because motion is the relevant variable; a stationary but asymmetric network defeats application-only adaptation just as thoroughly. And the conditioning principle carries into continuous allocation, where one policy taking the intent as an input replaces three separately trained specialists at a third of the training cost, with no penalty detectable at the resolution our experiments achieve — but it also finds that principle’s boundary. The cancellation result is exact only for scorers additively separable in the conditioning; we measure three ways an ordinary deployed architecture violates that premise, and a trained nonlinear encoder consequently routes intent past a cancellation the theory says should block it. The theory says where conditioning must enter to be *guaranteed* to act; the experiment shows a deep encoder arriving there without a guarantee.

1.1.3 Guiding Questions

Throughout this investigation, four central puzzles guide the reader:

- *Can a single RL policy realize a range of application intents without retraining, and where must the intent enter the network for it to change the policy’s choice at all?*
- *Where are the exact operational boundaries where single-path selection falls short?*
- *When is it worth splitting traffic across multiple paths, and does joint application–network co-optimization strictly beat optimizing either layer in isolation?*
- *How do these learned controllers perform regarding probing overhead, real-time inference latency, and deployability?*

1.2 Contributions

This thesis makes the following explicit, verifiable contributions:

1. **An intent-conditioned single-path selection framework for SCION:** We extend the DQN path selector of Bourak et al. [3] with a permutation-equivariant path-scoring network and per-path intent conditioning, enabling

a single policy to realize a range of application intents without per-objective retraining, and to serve intents it never trained on *zero-shot*: behavior interpolates monotonically between trained objectives (Spearman $\rho = -1.000$), including on source–destination pairs never seen in training (Chapter 4).

2. **An analysis of where intent conditioning must enter:** We show analytically that a conditioning signal reaching only the terms shared by every candidate cannot change a comparison-based selection, and confirm empirically, over five training seeds with disjoint confidence intervals, that such a policy re-ranks roughly three times less than one whose conditioning reaches the per-candidate comparison (Chapter 4).
3. **Verifiable probing reduction at near-oracle quality:** We demonstrate that our learned single-path selector falls short of a per-context reward oracle by at most 0.0018 reward on any evaluated intent — while inspecting one path per decision rather than all thirty — and outperforms every heuristic baseline on three of four intents, cutting probing cost by roughly 40× against exhaustive bandwidth measurement and probing less than every heuristic in absolute terms. We also bound what that proximity means: the oracle’s own re-ranking rate is 0.161, so most decision contexts admit no better answer under a different intent, and near-oracle performance partly reflects a task with little headroom (Chapter 4).
4. **A hierarchical system for joint application–network co-optimization:** We design and implement a two-timescale hierarchical SAC framework for real-time video over Multipath QUIC, co-optimizing target bitrates and continuous path splits across NS-3 packet simulation and analytical testbeds. It beats all seven comparison points under all three application intents — including a WebRTC-style deployed stack, which it leads by 0.065, 0.118, and 0.143 QoE against training-seed spreads of 0.018–0.023 — ranking first in every evaluation cell under two intents and eight of nine under the third, and delivering higher VMAF at lower delivered latency in each case (Chapter 5).
5. **The permutation-equivariant path head is load-bearing, not a convenience:** Scoring each candidate with shared weights is usually presented as the way to accommodate a variable action set. We show it is what makes the system work at all. Substituting a conventional fixed-dimension head, with the two configurations otherwise field-identical, costs 0.231 ± 0.029 QoE, positive in all nine evaluation cells against training-seed spreads of 0.013 and 0.037 — the largest measured effect in this thesis, and enough that with the flat head the preceding contribution is simply false, the learned pair ranking first in zero of nine cells rather than nine. The same architecture raises single-path goodput by 41% over a fixed-action agent, at less than half the parameter count, so on both problems the dynamic-input design is cheaper to store as well as better to run (Chapters 4 and 5).

6. **Intent conditioning extended to continuous allocation, with its premise measured:** We prove the cancellation result’s softmax analogue — for an additively separable scorer, conditioning entering only path-independent terms leaves the traffic split numerically identical, exactly rather than up to re-ranking — and show that a single conditioned policy replaces three per-intent specialists at a third of the training cost, with no penalty detectable at the resolution these experiments achieve, adapting continuously across the space between them. We state that as the null it is: the training-seed spread is 0.041–0.059 against effects of order 0.02–0.05, and the bar itself is noise-limited, since the specialists proved not to be measurably specialized. We then measure three mechanisms by which the deployed architecture violates the proposition’s premise, so that a nonlinear encoder routes conditioning past the cancellation in practice: the result identifies which routings carry a *guarantee*, not which happen to work (Chapter 5).
7. **Characterization of when joint multipath control pays off:** We show the governing quantity is not network volatility but the relationship between the load an even split places on each path and that path’s residual capacity, together with how strictly the application prices delay. A stationary but asymmetric topology defeats application-only adaptation as thoroughly as a churning one, while on a well-balanced topology the measured margin falls below the spread between training runs of the same method — so the regime a scheduler is benchmarked in decides whether it can be measured at all (Chapter 5 and §6.2).
8. **A measurement discipline for learned network control, arrived at by retraction:** Four results in this thesis were reported from a single training run and did not survive being run three times, because the spread over training seeds exceeds the confidence interval over evaluation seeds by an order of magnitude on this problem. Sampling training seeds is necessary and, we show, not sufficient: across six independently trained policies the analytical backend’s ranking is uncorrelated with the packet-level backend’s ($\rho = -0.086$, the best surrogate checkpoint being the worst on the packet-level body), so a cheap surrogate can be an adequate training environment while carrying no information for selecting among the policies it produces (Chapter 5).

1.3 Organization

The remainder of this thesis is organized as follows:

Chapter 2 reviews technical foundations, including SCION, real-time video transport, deep RL machinery, and goal-conditioned policies. Chapter 3 formalizes the decision problems, system models, desired properties, and trust assumptions.

Chapter 4 presents our study on intent-conditioned single-path selection, detailing its scoring architecture, the conditioning result and its proof, and the performance limits it exposes. Chapter 5 expands into joint application–network optimization over multipath, detailing the two-timescale SAC architecture and evaluation across dynamic network regimes.

[TODO: say here, where the reader forms expectations about what each chapter contains, that the two studies do not share a substrate. Chapter 4 runs on a generated SCION topology with beaconing, segments, and a path store; Chapter 5 runs on a hand-specified six-path configuration (Table A.4) with no SCION control plane at all, using only the path multiplicity SCION provides. This is a defensible scoping decision — the multipath question is about transport and application control, not about path discovery — but leaving it implicit invites the reader to assume a continuity of environment that does not exist, and to be surprised in §5.2. One sentence here, and the corresponding one in Chapter 5’s opening, settle it.]

Chapter 6 unifies the findings, analyzing overhead, scalability, generalization, and deployment considerations (Sections 6.1 and 6.2). Chapter 7 places our work within the broader research landscape, and Chapter 8 concludes.

2 Background & Technical Foundations

This chapter reviews the systems, control mechanisms, and machine learning techniques upon which this thesis builds. Section 2.1 introduces the SCION path-aware Internet architecture and the path-selection opportunities it creates. Section 2.2 covers quality-of-experience metrics for real-time video, adaptive bitrate control, and multipath transport mechanisms. Section 2.3 recalls the core reinforcement learning (RL) machinery: Markov decision processes, deep Q-networks with standard refinements, soft actor-critic, and permutation-equivariant scoring architectures for dynamic action spaces. Finally, Section 2.4 details goal-conditioned policy representations and the goal-conditioning machinery that lets one policy serve many objectives.

2.1 The SCION Path-Aware Architecture

SCION [13] is a next-generation, path-aware Internet architecture designed for high availability and transparency of the forwarding path. Its defining departure from today’s Internet is that *end hosts, not intermediate routers, choose the path a packet takes*. The network is organized into *isolation domains* (ISDs), each governed by a set of core autonomous systems (ASes). Path segments are discovered by a beaconing process: core ASes flood *path-segment construction beacons* (PCBs) that accumulate hop information as they propagate, giving rise to three segment types – *up* segments from a leaf AS to its ISD core, *core* segments between core ASes across ISDs, and *down* segments from a core to a destination leaf. An end host combines an up, an (optional) core, and a down segment into a complete forwarding path and embeds it in the packet header.

Two properties of SCION matter for this thesis. First, **path multiplicity**: a source– destination pair is typically connected by *several* distinct paths that differ in latency, available bandwidth, loss, and the ASes they traverse. This is precisely the raw material an intent-conditioned path-selection agent needs, and the same multiplicity makes multipath transport natural. Second, **path transparency and trust**: because the full AS-level path is visible to the endpoint, an agent can reason about properties beyond raw performance – for instance a *trust* score that reflects how much confidence to place in a path’s advertised or measured quality. We exploit this in our single-path selection studies, where the reward balances measured goodput against a trust term. **[TODO: the “trust” term used in Chapter 4 is defined purely as $1 - (w_3 \text{ loss} + w_4 \text{ delay})$ (Eq. (4.1)) – a reliability score derived from measured loss and delay, with no AS reputation, certificate, or path-authentication**

content. Either rename it (“reliability”, “path quality”) throughout the thesis, or state explicitly here and at Eq. (4.1) that it is a performance proxy and not trust in the SCION control-plane sense. A reader from this group will otherwise read the stronger meaning into it.] The flip side of endpoint path choice is a coordination problem: whereas a router runs one routing process, every SCION host must decide for itself, ideally without flooding the network with probes. Minimizing that probing overhead is an explicit objective throughout our work.

[TODO: this chapter is missing three pieces of background that Chapter 4 uses without introducing. (i) Topology generation: the study’s environment is built with BRITE and a Barabási–Albert AS graph model, which is never explained. (ii) The heuristics used as baselines: equal-cost multi-path (ECMP) and what a “default” SCION path policy is (shortest AS-path, tie-broken by latency) both need a sentence here, since §4.3.3 compares against them. (iii) Prior SCION path selection: how endpoints choose paths today – the path policy language, the path service, and existing selection heuristics – so that the learned selector has a stated status quo to improve on.]

2.2 Quality of Experience and Transport Mechanisms

2.2.1 Quality of experience for real-time video

To evaluate end-to-end intent control, we target interactive video, whose value to a user is captured by quality-of-experience (QoE) rather than any single network metric. Perceptual quality at a given encoding is commonly summarized by VMAF [10], a learned metric that predicts subjective video quality; interactivity adds extreme sensitivity to end-to-end latency, jitter, and frame loss, since a late frame in a live call is effectively a lost frame. Our multipath reward composes a VMAF-based quality term with penalties for latency, jitter, and loss, making the QoE objective explicit and tunable.

[TODO: two pieces of background this section owes Chapter 5 and does not supply. (i) Google Congestion Control and the WebRTC stack: GCC is the bitrate estimator behind both the webrtc baseline — the strongest hand-written policy in Table 5.3 and the one every headline margin is quoted against — and the path-only ablation, yet the reader first meets it in an evaluation table with no introduction. A paragraph on how GCC estimates available bandwidth from delay gradients and loss is what lets a reader judge whether beating it by 11–21% is impressive. (ii) The ITU-T ladder: Table 5.1 keys its three intents to G.1010 use-case classes, anchors its delay weights on G.114, its jitter normalizer on Y.1541 class-0 IPDV, and fits (b, d) against a G.1070 opinion model. Four standards carry the reward’s provenance and none is introduced here. This background is also what makes the reward-calibration gap at Table 5.1 land as a methodological contribution rather than a hyperparameter note.]

2.2.2 Adaptive bitrate and learned control

The application-side lever in joint control is adaptive bitrate (ABR) control: choosing the video encoder’s target rate to match available capacity. Learned ABR controllers such as Pensieve [11] showed that RL can outperform hand-crafted heuristics for buffered video streaming. Our setting differs in two critical ways: it is strictly real-time (lacking a large playback buffer to hide network variance), and the bitrate decision is tightly coupled to a simultaneous *network-side* decision regarding how to allocate traffic across multiple paths.

2.2.3 Multipath QUIC

QUIC [8] is a modern, encrypted, multiplexed transport running over UDP. Multipath QUIC [5] extends it to send a single connection’s data across several network paths at once, in the spirit of Multipath TCP [6] but with user-space deployability. Over a path-aware substrate like SCION, multiple paths are readily available from path discovery, making MPQUIC a natural transport for exploiting SCION’s path multiplicity. The open question MPQUIC raises – how to *schedule* bytes across heterogeneous paths – is one of the primary control dimensions learned in this thesis.

2.3 Deep Reinforcement Learning Machinery

2.3.1 Markov decision processes

We model path and rate control as sequential decision-making in a Markov decision process (MDP) $(\mathcal{S}, \mathcal{A}, P, r, \gamma)$, where at each step the agent observes a state $s \in \mathcal{S}$, takes an action $a \in \mathcal{A}$, receives a scalar reward $r(s, a)$, and transitions to s' with probability $P(s' | s, a)$. The agent seeks a policy π that maximizes the expected discounted return $\mathbb{E}[\sum_t \gamma^t r_t]$ with discount $\gamma \in [0, 1)$. The reward function is where application intent enters: by changing what the agent is rewarded for, we change what “best” means.

2.3.2 Deep Q-networks and refinements

The action-value function $Q^\pi(s, a) = \mathbb{E}[\sum_t \gamma^t r_t | s_0 = s, a_0 = a]$ gives the expected return of taking a in s and following π thereafter. A deep Q-network (DQN) [12] approximates the optimal Q^* with a neural network trained to minimize the temporal-difference error against a slowly updated *target* network, sampling transitions from a replay buffer to decorrelate them. We utilize three standard refinements. *Double DQN* [18] decouples action selection from action evaluation in the bootstrap target, reducing the overestimation bias of vanilla DQN. The *dueling* architecture [19] factors $Q(s, a) = V(s) + (A(s, a) - \frac{1}{|\mathcal{A}|} \sum_{a'} A(s, a'))$ into a

state-value stream V and an advantage stream A , which stabilizes learning when many paths have comparably good value. *Prioritized experience replay* (PER) [15] samples high-error transitions more often, correcting the resulting bias with importance-sampling weights.

2.3.3 Soft actor-critic

For continuous control domains – such as determining target bitrates and continuous traffic split ratios – value-based DQN is ill-suited. We adopt soft actor-critic (SAC) [7], an off-policy, maximum-entropy actor-critic method. SAC augments the reward with a policy-entropy term, $\mathbb{E}[\sum_t \gamma^t (r_t + \alpha \mathcal{H}(\pi(\cdot | s_t)))]$, which encourages exploration and robustness; it trains twin Q -critics to curb overestimation and automatically tunes the temperature α against a target entropy. SAC’s sample efficiency and stability make it a strong fit for expensive, noisy rollouts in network simulators.

2.3.4 Variable and structured action spaces

A central difficulty in network path control is that the number of candidate paths is dynamic: it varies across source–destination pairs and changes over time. A classical neural network with a fixed output head cannot represent this unbounded decision space. The remedy, used throughout this thesis, is a *scoring* architecture that is *permutation-equivariant* over the path set: a shared network maps each path’s feature vector (together with a global context) to a scalar score, and the scores are compared – by $\arg \max$ for discrete single-path selection, or by a softmax to produce a continuous traffic split. Invalid or absent paths are masked out. Aggregating a set of per-element embeddings in a way that is invariant to their ordering is achieved via the Deep Sets construction [22], which we employ in our continuous critic design. This structural approach decouples the policy from the cardinality of the action set, allowing a single network to generalize across topologies with arbitrary numbers of paths.

2.4 Goal-Conditioned Reinforcement Learning

2.4.1 Conditioning a policy on its objective

A policy that must serve diverse application preferences can be made a function of the objective itself. In goal-conditioned RL, the policy $\pi(a | s, g)$ and value function are conditioned on a goal vector g , so that a single neural network represents a whole family of behaviors. Universal value function approximators (UVFAs) [14] formalize this by learning $V(s, g)$ jointly over states and goals, while hindsight experience replay [2] improves sample efficiency by relabeling trajectories with goals achieved *a posteriori*. In this thesis, the “goal” represents application intent,

encoded as a vector of reward weights: the agent is explicitly told how much it should prioritize throughput versus latency, jitter, loss, or trust, and is expected to adapt its decisions dynamically.

*[TODO: this subsection reaches for UVEAs and hindsight relabeling and misses the literature that is actually about what this thesis does. Conditioning a policy on a vector of reward weights and expecting it to span the induced family of optima is **multi-objective reinforcement learning**, not general goal-conditioning: the goal here is a preference over a scalarization, not a target state, and HER — which relabels with achieved goals — does not even apply to it. The MORL literature on weight-conditioned value functions covers exactly this construction and is not cited once, here or in §7.4. This is the most conspicuous citation hole in the thesis and the first thing a reviewer from the learning side will find. Add a paragraph placing the thesis’s intent vector as a preference weighting over a linear scalarization, note what MORL already establishes (one network can represent the whole Pareto front over a weight simplex), and be explicit about the delta this thesis claims on top of it, which is real and narrow: MORL policies are not comparison-based, so nothing in that literature addresses where in a scoring architecture the weights must enter — which is the entire content of Proposition 4.1. Stating that boundary strengthens the contribution; omitting the literature makes it look unaware of it.]*

2.4.2 Where the conditioning signal enters

Where the goal vector enters the network is as consequential as whether it is supplied at all, and the literature on goal-conditioned RL says little about it because its usual policies are not comparison-based. Our policies are. When the action is an arg max over per-alternative scores, a conditioning signal that enters only terms shared by every alternative shifts all scores together and cancels under the comparison, so the policy can consume the goal without ever acting on it. The signal must instead reach the per-alternative terms, where it can interact with each candidate’s own features and change their relative order. Section 4.2.1 makes this precise as Proposition 4.1, and §4.4.2 measures its consequences.

3 Problem Statement & Desired Properties

This chapter formalizes the network decision problems addressed in this thesis. We define the system model, state the operational decision spaces across single-path and multipath regimes, enumerate the essential properties a satisfactory solution must exhibit, and outline our core operating assumptions and trust boundaries.

3.1 Problem Formulation

Consider an endpoint host communicating over a path-aware architecture such as SCION. At any decision epoch t , the host discovers a dynamic set of N_t candidate end-to-end paths $\mathcal{P}_t = \{p_1, \dots, p_{N_t}\}$. Each path p_i is characterized by partially observable and dynamic performance properties – latency, loss rate, available bandwidth, hop count, and path trust. Simultaneously, the running application expresses its *intent* $g \in \mathcal{G}$, defined as a declarative preference vector prioritizing specific performance metrics (e.g., maximizing throughput, minimizing delay, or optimizing video quality).

Because network utilization and link availability fluctuate over time, the host must dynamically determine how best to utilize the path set $\mathcal{P}_t$ to serve intent g . We formulate this decision across two progressive operational settings:

1. **Intent-Aware Single-Path Selection (Discrete Decision):** For general application flows constrained to a single path, the decision is a discrete selection $p^\star \in \mathcal{P}_t$. The objective is to maximize an intent-weighted reward balancing goodput, latency, and trust while minimizing the active probing overhead required to inspect candidate path states (Chapter 4).
2. **Joint Application–Network Multipath Optimization (Continuous Decision):** For high-bandwidth, latency-sensitive applications (e.g., real-time WebRTC video) operating over Multipath QUIC, the decision space expands. The endpoint must jointly choose a continuous traffic split $w = (w_1, \dots, w_{N_t})$ across paths (where $\sum w_i = 1$) and adapt the application’s own sending rate (the video encoder’s target bitrate R_{video}) to maximize real-time Quality of Experience (QoE) (Chapter 5).

In both formulations, the technical crux is twofold: (i) the objective is dynamic and dictated by an intent g that varies across flows, and (ii) the action space

is dynamic and unbounded, as candidate path sets $\mathcal{P}_t$ fluctuate across network topologies and over time.

3.2 Desired Properties

A satisfactory solution must exhibit six key properties, which also define the primary metrics evaluated in subsequent chapters (Chapters 4–6):

For the single-path system, each of the six is measured rather than asserted: §4.4.3 for Intent Alignment, §4.4.4 for Zero-Shot Generalization, and §4.4.5 for Dynamic-Action Handling, with §4.4.2 reporting Structural Generalization on a disjoint set of source–destination pairs. One qualification applies throughout: structural generalization is demonstrated across pairs and candidate-set sizes within a single topology, since only one topology was generated, so generalization across topology *scale* remains a design property rather than an evaluated one.

Intent Alignment. The policy must faithfully reflect the specified preference vector g . Decisions under g_{latency} must prioritize low delay, whereas decisions under $g_{\text{throughput}}$ must prioritize goodput. A policy whose action choices remain invariant to changes in g fails this requirement.

Zero-Shot Generalization Across Objectives. A single trained network must serve the entire spectrum of application intents without requiring per-objective model retraining or fine-tuning.

Dynamic-Action Handling. The policy architecture must natively accept dynamic candidate path counts N_t without structural modification or retraining, remaining invariant to the arbitrary ordering of paths in $\mathcal{P}_t$.

Low Overhead. Decision-making must remain computationally lightweight. In single-path selection, active probing cost must be minimized; in multipath real-time control, per-frame inference latency must fit comfortably within frame processing budgets (≈ 33 ms).

Structural & Topological Generalization. Performance must hold off-distribution across unseen topologies, dynamic path counts, and shifting traffic workloads.

Robustness to Network Volatility. The policy must sustain high performance under non-stationary conditions – path churn, sudden link failures, and transient congestion bursts – where static heuristics fail.

[TODO: this last property states as a requirement the thing §5.3.3 and §6.1.1 explicitly refute as the governing variable. The chapter's finding is that what decides whether a scheduler has anything to earn is the relationship between the load an even split places on each path and that path's residual capacity — headroom heterogeneity — and that churn,

regime shifts, and bursts matter only because they destroy that homogeneity transiently. A stationary but asymmetric topology defeats application-only adaptation as thoroughly as a churning one, and a stationary near-equal topology leaves the scheduler nothing to do. As written, the problem statement sets up an expectation the results correct, and a reader arriving at §5.3.3 will read the correction as a retreat rather than as a sharpening. Restate the property as robustness to headroom heterogeneity, with volatility named as one mechanism that produces it. This also makes Chapter 3 the place the thesis's second-most-interesting finding is foreshadowed rather than contradicted.]

[TODO: the property-to-measurement map above covers the single-path system only — six properties, each pointed at a Chapter 4 section. Add the multipath column, since Chapter 5 measures five of the six and the reader has no index into them: Intent Alignment (Sections 5.3.4 and 5.3.5), Zero-Shot Generalization (§5.3.5, with the caveat that interior intents are in-distribution there), Dynamic-Action Handling (§5.3.7's flat-versus-scoring arm), Structural Generalization (the held-out dynamics arms), and Robustness (§5.3.3). Mark the one that is not measured for the multipath system — Low Overhead, whose per-frame inference cost is the binding real-time constraint of the entire design and is unstated in both chapters (§6.2.1) — so the omission is declared here rather than discovered.]

3.3 Assumptions & Scope

We explicitly state the operating assumptions underlying our design:

1. **Path-Aware Substrate.** The underlying network exposes multiple distinct end-to-end paths to the endpoint and honors source routing decisions, as provided by SCION. We do not assume control over internal intra-AS routing decisions.
2. **Observable Path Feedback.** Per-path metrics (latency, loss, bandwidth) are measurable via active probing (in single-path selection) or derived continuously from transport feedback (e.g., QUIC ACK frames and RTT samples).
3. **Explicit Intent Declaration.** Application intent is declared as an explicit preference vector (e.g., reward profile weights or target QoE parameterizations). We do not attempt to infer latent intent from raw packet inspection.
4. **Episodic Stationarity.** Network state is assumed roughly stationary within a brief decision epoch, though conditions may drift or shift abruptly between epochs.

4 Motivating Study: Intent-Conditioned Single-Path Selection

This chapter investigates the first and most basic control decision a SCION endpoint faces: given a running application’s intent and a changing network, *which* of the candidate paths to a destination should carry a flow? We answer it in five steps. Section 4.1 builds the selection framework – the simulation environment, the decision problem, and a permutation-equivariant scoring architecture that copes with a variable path set. Section 4.2 adds the chapter’s central contribution: we show that *where* an intent signal enters a comparison-based policy decides whether it can change the chosen path at all, and condition the per-path features accordingly. Section 4.3 describes the implementation, the baselines, and the per-context oracle against which everything is measured. Section 4.4 evaluates the conditioned selector on held-out hours, on intents it never trained on, and on candidate sets whose size it never saw. Finally, §4.5 identifies the single-path performance ceiling that motivates the multipath system of Chapter 5.

4.1 Selection Framework: Environment, Decision, and Scoring

We start from the DQN path selector of Bourak et al. [3] and identify three limitations we set out to remove (§4.1.1). We then describe the improved simulation environment (§4.1.2), the state, action, and reward that define the decision problem (§4.1.3), and the permutation-equivariant path-scoring architecture that handles a variable path set (§4.1.4).

4.1.1 Starting Point and Its Limitations

Bourak et al. [3] frame SCION path selection as a reinforcement-learning problem and train a DQN to pick a path from per-path measurements of latency, loss, and bandwidth, rewarding a weighted combination of goodput and trust while charging for probing. They report oracle-level selection quality at a fraction of the probing cost of measuring every path. This validates the RL framing and gives us a reward structure to build on, but three design choices limit it.

Fixed action space. A vanilla DQN has one output per action, so the number of paths must be fixed and known. Real source–destination pairs have different,

time-varying path counts; padding to a maximum wastes capacity and still caps the topology size.

Single baked-in objective. The reward weights are constants chosen before training. The resulting policy embodies *one* trade-off between goodput and trust; a different application intent demands a different, separately trained model.

Environment fidelity. Capturing when path selection actually matters requires paths that genuinely share bottlenecks and conditions that vary over realistic timescales, so that a good choice at one hour is a poor choice at another.

[TODO: reconcile this description of the starting point with the agent that stands in for it, because as written the two do not match and the chapter’s largest quantitative claim rests on the difference. Here Bourak et al. are described as training a DQN “to pick a path from per-path measurements of latency, loss, and bandwidth”. Section 4.3.1 then implements the Flat DQN as “a conventional fixed-action DQN over the 5-dimensional global state” — day, hour, mean utilization, mean trust, and the fraction of loaded paths — which contains no per-path features at all. If that is faithful, say so here and drop the “per-path measurements” phrasing. If it is not, the 41% goodput gap and “the flat architecture is the wrong shape” are measured against a control weaker than the published system, and the honest options are (i) give the flat agent the per-path features the original had, keeping only the fixed action head as the difference under test — which is what would actually isolate the architectural claim — or (ii) relabel it throughout as a fixed-action-space control of our own construction rather than a stand-in for prior work, and confine the comparison to what it licenses. Option (i) is a small change to an agent that trains in 1–3 minutes and would make this the cleanest ablation in the thesis; option (ii) costs nothing but narrows the claim. Either is defensible; the current pairing is not.]

Our design removes all three: a scoring architecture for the first, per-path intent conditioning for the second, and an improved environment for the third. A strawman for the second limitation – appending the reward weights to the state of the existing DQN – is instructive precisely because of *where* it fails, as we show in §4.2.

The third limitation deserves a number rather than an assertion, since it bounds the size of every effect this chapter reports. The quantity that settles it is the spread of achievable goodput across a pair’s candidate paths *within a single decision context* – exactly what a selector has to exploit. We measure it in §4.1.2 and find it wide and time-varying, so path choice is a first-order effect in this environment.

4.1.2 Improved SCION Simulation Environment

Training and evaluating a path selector requires an environment in which the “right” path changes with conditions. We construct one in several stages.

Topology. AS-level topologies are generated with BRITE and converted into SCION structures – ISDs, core and non-core ASes, and inter/intra-ISD links – so that the graph has realistic degree and locality structure rather than being hand-drawn. We describe the conversion concretely, because the capacity heterogeneity the agent exploits comes from the converter rather than from BRITE, and a reader who assumes BRITE-generated bandwidths will misread the results. BRITE runs its AS-level Barabási–Albert model on 90 nodes with the bandwidth distribution pinned to a constant, so *every one of its 382 links has exactly 10 Gbit/s of capacity*. The converter assigns ISDs by k -means on node coordinates and core ASes by degree rank, then adds 146 links of its own with hard-coded capacities (Table 4.1); these additions are the sole source of capacity variation in the graph. Link latency is likewise not BRITE’s delay column, which is on an abstract scale, but $\max(0.5, 0.02d)$ milliseconds in the node coordinate plane, giving a median inter-AS latency of 8.5 ms and a maximum of 23.0 ms.

Control plane. A beaconing simulation propagates path-segment construction beacons top-down within each ISD and across the core mesh between ISDs, yielding the up/core/down segments that compose into end-to-end paths. Peer links are excluded. The result is, for each source–destination pair, a realistic set of candidate paths of varying length and quality – the agent’s action set.

Traffic and link dynamics. We drive the network with 28 days of hourly demand: foreground flows for every routable pair plus randomized background traffic, modulated by diurnal and weekly factors. Crucially, load is aggregated *per link*, so paths that share a bottleneck link contend for its capacity. This is what makes selection non-trivial: the utilization and effective bandwidth of a path depend on what every other flow is doing at that hour, and the best path at 3 a.m. is often congested at 8 p.m.

Link and path model. A link’s utilization is its aggregate offered load over its capacity, capped at 1.5; its latency is a base propagation delay scaled by a queueing multiplier that grows with utilization and saturates at $2\times$; and its loss rate is a piecewise function of utilization alone, zero below 0.8 and capped at 0.2. A path’s latency is the sum over its links, its loss the complement of the product of per-link survival probabilities, its utilization the maximum over its links, and its *available bandwidth* the minimum residual capacity $\max(0, C_e - \text{load}_e)$ across its links.

Two consequences of this model bound how the chapter’s results should be read, and we state them here rather than leave them to be discovered. First, the model is *open-loop*: the agent’s own selection never adds load to the links it traverses, so what the reward calls goodput is the spare capacity at the bottleneck rather than a rate the flow would actually achieve, and there is no per-flow fair share or congestion feedback. Second, utilization is capped at 1.5 rather than 1.0, so links can be oversubscribed; this is what produces the > 1.0 abscissa values in

Table 4.1: Link capacities in the evaluated topology, by origin. BRITE contributes a uniform 10 Gbit/s fabric; every capacity difference the selector can exploit is introduced by the SCION converter.

Origin	Link role	Capacity	Count
BRITE	peer, parent-child, core	10 Gbit/s	382
SCION converter	multi-parent	1 Gbit/s	35
	cross-connect	2 Gbit/s	35
	additional peering	$\mathcal{U}(5, 10)$ Gbit/s	75
	inter-ISD core	50 Gbit/s	1
Total			528

Fig. 4.6. Both are defensible choices for a study of *selection* – which path is best is well defined without modeling the flow’s own back-pressure – but neither would survive into a closed-loop transport study, and Chapter 5 accordingly uses a different data plane.

[TODO: this paragraph is the right qualification in the right place, and it then fails to travel. Every later quotation of an absolute goodput — 8.21 Gbit/s in §4.4.2, the widest-path comparison in §4.4.6, the 9.1 Gbit/s-to-6.9 Gbit/s descent in §4.5, the 41% figure in the chapter summary, and the corresponding numbers in §1.2 and the abstract — reads to an uninstructed reader as throughput a flow would achieve. It is spare capacity at the bottleneck under a model in which this flow places no load. Add a short standing qualifier at first use in each of those places (“residual bottleneck capacity, not an achieved rate”) or define the quantity under a distinct name here — available capacity rather than goodput — and use it consistently. The second is cleaner and removes the problem at the root, at the cost of a global rename. Note the ceiling figure of §4.5 is the one place where the distinction bites hardest, since the whole bridging argument to Chapter 5 is about capacity a single path cannot deliver.]

Does path choice matter here? The environment is only interesting if the candidate paths of a pair differ materially at a given hour, so we measure that spread directly over all 10752 held-out decision contexts (§4.4.1). Within a single context, the best candidate carries **3.04×** the median candidate’s goodput (p10 1.57×, p90 18.26×) and 7983 Mbit/s more than the worst; the coefficient of variation of goodput across candidates has median 1.43, and the worst candidate’s latency exceeds the best’s by 29.3 ms at the median, a factor of 3.5. The choice is also genuinely time-varying: the *identity* of the highest-goodput path changes between consecutive hours in 8.6% of cases, so a policy cannot memorize one answer per pair. At the network level, p90 link utilization is 0.76 with 7.8% of links oversubscribed, and only 0.09% of contexts have no usable bandwidth on any candidate. Path choice is therefore a first-order effect in this environment, which is the premise §4.1.1 needs.

Probing model. Fresh metrics are not free. The environment charges a latency probe 10 ms plus 0.5 ms per hop, and a full bandwidth probe 100 ms plus 20 ms per hop; it credits the agent with having probed only the path(s) it actually inspects. Because both costs grow with hop count, probing is cheaper on short paths – a fact §4.4.6 returns to. The reward charges the *mean* cost per probe issued in a step, normalized by a 500 ms reference. This turns “measure less” into a real objective rather than an afterthought, and lets us compare the learned selector’s probing footprint against baselines that probe exhaustively.

Charging the *mean* rather than the total cost is consequential and deserves justification, because it means a method issuing thirty cheap probes is penalized less by the reward than one issuing a single expensive probe. This is deliberate: the per-probe charge measures cost per unit of information gained, which keeps the reward a statement about *selection quality* and prevents the probing term from dominating the goodput and trust terms it is meant to modulate. The consequence is that the reward understates the probing gap – under the Balanced intent the learned selector is charged 135 ms per probe against exhaustive widest-path selection’s 180 ms, a near-tie, while their *total* measurement cost differs by a factor of 40. We therefore report total probing cost separately in §4.4.6 rather than relying on the reward to express it.

4.1.3 Decision Problem: State, Action, Reward

State. The agent observes a *global* context and a *per-path* feature matrix. The global context summarizes time and aggregate conditions – normalized day-of-week and hour-of-day, mean path utilization, mean trust, and the fraction of paths whose utilization exceeds 0.7 – plus a compact encoding of the source and destination. The three aggregate terms are computed over the current flow’s own candidate path set, not over the whole network. Each candidate path contributes a seven-dimensional feature row: latency normalized by 100 ms, loss rate, hop count over 20, the path’s bandwidth relative to the best candidate at that hour, utilization, a static-bandwidth term, and a trust score. Because the number of rows N varies, the state is naturally a (global, path-matrix) pair rather than a flat vector.

Action. The action is the choice of one path, i.e. an index into the current candidate set. Because N varies, the action is realized as an $\arg \max$ over per-path scores (§4.1.4) with invalid entries masked, rather than as a fixed categorical head.

Reward. We adopt and generalize the goodput–trust reward of Bourak et al. [3]. For the chosen path, let $\text{goodput} \in [0, 1]$ be the achieved bandwidth normalized by the *best available bandwidth among that pair’s candidate paths at that hour*, and define a trust term from loss and delay,

$$\text{trust} = \max(0, 1 - (w_3 \text{loss} + w_4 \text{delay})), \quad (4.1)$$

with delay the latency capped at 100 ms and normalized. The base reward trades goodput against trust and is shifted to $[-1, 1]$, and a probing penalty is subtracted:

$$r = \underbrace{2(w_1 \text{ goodput} + w_2 \text{ trust}) - 1}_{\text{base reward}} - w_{\text{probe}} \cdot \text{probe_penalty}, \quad (4.2)$$

clamped to $[-1, 1]$, where probe_penalty is the probing cost normalized to $[0, 1]$. We take $w_1 + w_2 = 1$ throughout, which is what places the base reward in $[-1, 1]$ before clamping. The five weights $\mathbf{w} = (w_1, w_2, w_3, w_4, w_{\text{probe}})$ are exactly the *application intent*: default balanced values are $(0.7, 0.3, 0.5, 0.5, 0.05)$, but a throughput-hungry intent pushes w_1 toward 1, a loss-averse intent raises w_2 and w_3 , a latency-sensitive intent raises w_4 , and a probe-frugal intent raises w_{probe} . The design goal of §4.2 is a *single* policy that behaves correctly for *any* setting of $\mathbf{w}$.

Two properties of this reward shape every result in the chapter, and we state them explicitly because both are easy to misread. First, goodput is normalized against the *best candidate path of that pair at that hour*, so it is a relative score rather than an absolute rate: a policy that always picks the best available path scores near 1 whether that path carries 9 Gbit/s or 2 Gbit/s. The reward therefore measures selection quality and is blind by construction to how much capacity exists to select from – which is why, in §4.5, the learned selector’s reward stays flat across the congestion range while its goodput falls by a quarter. Second, latency enters the objective only *inside* the trust term, through w_4 ; there is no standalone latency weight. A latency-sensitive intent is expressed by raising w_2 and w_4 together, which is how the *Low-Latency* profile of Table A.1 is defined.

4.1.4 Handling a Variable Path Set: The Scoring Architecture

To make the policy independent of the path count, we score each path with a *shared* network and select by comparison, following the permutation-equivariant design of §2.3.4. The global context is broadcast and concatenated to every path’s feature row, and a shared multilayer perceptron maps each $(\text{global}, \text{path}_i)$ pair to a scalar Q -value. Selection is $\arg \max_i Q_i$; absent or invalid paths are masked to -10^9 so they can never be chosen and do not distort training targets.

We use a *dueling* scoring head (§2.3): a value stream reads the global context alone to produce $V(s)$, an advantage stream reads the per-path features to produce $A(s, i)$, and the two combine as $Q_i = V(s) + (A(s, i) - \text{mean}_{j \in \text{valid}} A(s, j))$ with the mean taken only over valid paths. Because many paths are often comparably good, separating “how good is this situation” from “which path is marginally better” stabilizes learning. Double DQN targets and prioritized replay (§2.3) further stabilize and accelerate training. The scoring design also delivers the dynamic-action property directly: the same weights serve a pair with three paths and a pair with thirty, and are invariant to how the paths happen to be ordered.

4.2 Where Intent Conditioning Must Enter

The central contribution of this chapter is making one scoring policy serve every intent $\mathbf{w}$. The finding is that this is not a matter of feeding the policy its intent: *where* the intent enters decides whether it can influence the choice at all.

4.2.1 Where conditioning cancels

The obvious approach is to append $\mathbf{w}$ to the state. Whether that works depends entirely on *which* part of the state it is appended to, and the reason is a one-line argument about the comparison that defines the action.

Proposition 4.1 (Common-mode cancellation). *Let a scoring policy assign path i the score $Q_i(s, \mathbf{w}) = f(s, f_i) + c(s, \mathbf{w})$, where f_i is path i 's feature row and the conditioning enters only through the term c , which does not depend on i . Then $\arg \max_i Q_i(s, \mathbf{w})$ is independent of $\mathbf{w}$: the policy consumes the intent but its action is invariant to it.*

The proof is immediate — $c(s, \mathbf{w})$ is a constant offset across the compared alternatives, so it cancels in every pairwise comparison $Q_i - Q_j$ and cannot change which index attains the maximum. The point is that this is not a statement about optimization difficulty or sample efficiency: no amount of training can overcome it, because the intent-dependent term is not in the image of the decision rule at all.

The dueling scoring head of §4.1.4 instantiates exactly this hypothesis when the intent is routed into the value stream. There $Q_i = V(s, \mathbf{w}) + (A(s, i) - \text{mean}_j A(s, j))$, so V plays the role of c : conditioning the value stream is provably inert. We verify this directly at the network level – holding path features fixed and varying only $\mathbf{w}$ leaves every pairwise Q -difference constant to within 10^{-5} – and §4.4.2 confirms the behavioral consequence over five training seeds.

One qualification matters, because it is the difference between “cannot re-rank” and “re-ranks only through a channel it does not control”. Proposition 4.1 is a statement about the *architecture*, and in the deployed system its premise is mildly violated: the seventh per-path feature is the trust score, which is itself computed from w_3 and w_4 . Intent therefore does leak into f_i through one feature, and this is precisely why the value-stream variant of §4.4.2 attains a small but nonzero behavioral divergence rather than exactly zero. The measured residual is what the leak predicts, which sharpens the proposition rather than weakening it.

4.2.2 Conditioning the per-path comparison

The fix follows directly from Proposition 4.1: the intent must reach the term that distinguishes one candidate from another. The scoring architecture of §4.1.4 already broadcasts the global context to every path row before the advantage

stream, so it suffices to make the intent part of that context. Writing $\tilde{s} = [s \parallel \mathbf{w}]$ for the widened global vector, with $[\cdot \parallel \cdot]$ denoting concatenation, the scores become

$$Q_i = V(\tilde{s}) + \left(A([\tilde{s} \parallel f_i]) - \text{mean}_{j \in \text{valid}} A([\tilde{s} \parallel f_j]) \right). \quad (4.3)$$

The advantage stream now evaluates $\mathbf{w}$ jointly with each path’s own feature row, and because the shared scorer is nonlinear, the same $\mathbf{w}$ shifts two candidates by different amounts, so the ordering can change. This is not of the form Proposition 4.1 assumes and the cancellation argument does not apply. The intent also reaches the value stream, which is why we call the agent *Two-Stream-Concat*; by the proposition that half is inert for the choice, and it is the advantage stream that does the work.

We adopt this mechanism because it is the least the proposition permits. It introduces no conditioning sub-network, no new hyperparameters, and no new architectural component: it widens the existing global context by the five intent weights and lets the existing broadcast do the rest, for $5 \times 128 \times 2 = 1280$ additional parameters – one input weight per hidden unit of each stream. It also inherits the architecture’s independence of the path count unchanged, since the intent is broadcast rather than replicated per candidate. Section 4.4.2 shows this captures 93% of the re-ranking a per-context oracle achieves, which leaves little room for a more elaborate mechanism to improve on. The finding this chapter reports is about *where* the intent enters, not about the machinery that carries it once it is in the right place.

4.2.3 Training across intents

To learn a policy that spans the intent space we train across six *reward profiles* – named weight vectors spanning a throughput extreme, a loss-averse extreme, a latency-averse extreme, a balanced midpoint, and two probe-frugality settings (Table A.1). Each training episode is assigned a profile, sets the environment’s reward weights to it, and feeds the same weights to the scorer, so the agent simultaneously experiences the reward for an intent and the encoding of that intent. Profiles are scheduled in a stratified fashion – each profile repeated to fill the episode budget, then shuffled – so every intent receives an equal number of episodes. The intent is presented to the network in a rescaled encoding rather than as raw weights, which keeps the conditioning inputs on a comparable scale to the path features.

Training on a finite set of profiles is a means, not the goal. What we want is a policy that maps the *intent space* to behavior, so that an unseen $\mathbf{w}$ presented at inference – in particular one interpolating between two trained profiles – yields correspondingly interpolated behavior without retraining. This is the zero-shot generalization property required by §3.2, and §4.4.4 tests it directly.

4.2.4 Relation to Mortise

This is the path-selection instance of Mortise’s principle [16]. Where Mortise maps an application’s revealed QoS preference to congestion-control *parameters*, our scorer makes an explicit intent vector an input to the per-path comparison that determines which path is chosen. In both, intent is a first-class input that reshapes a network mechanism, not a constant compiled into it. Chapter 5 pushes the same principle one step further, letting intent shape not only the network’s choice but the application’s own output.

4.3 Implementation

This section describes how the design is realized: the agent family and their architectures (§4.3.1), the end-to-end evaluation pipeline that turns a topology into trained models and comparable results (§4.3.2), and the baselines and probing accounting against which the learned selectors are measured (§4.3.3). The implementation is in Python (PyTorch for the agents) and is organized so that each pipeline stage passes data to the next through timestamped run directories.

4.3.1 Agent Family

The codebase implements a family of agents of increasing capability, from which the ablation ladder of §4.4.2 is drawn.

Flat DQN (simple / enhanced). A conventional fixed-action DQN over the 5-dimensional global state. The *simple* variant is a small multilayer perceptron with ϵ -greedy exploration and a periodic target network; the *enhanced* variant adds Double DQN [18], a dueling head [19], prioritized experience replay [15], soft target updates, and action masking. These agents cannot handle a variable path set and stand in for the original design [3].

Path-scoring DQN (simple / enhanced). The permutation-equivariant design of §4.1.4: a shared per-path scorer producing one Q -value per path, with variable N handled by padding to the batch maximum and masking. The enhanced variant uses the dueling scoring head, Double DQN, and prioritized replay. This family delivers the dynamic-action property.

Conditional path-scoring DQN (Two-Stream-Concat). The recommended agent, adding the conditioning of §4.2.2: the five-dimensional encoded intent widens the global context, which the scoring head already broadcasts to both streams, so the advantage stream evaluates the intent jointly with each candidate’s features. Two intent encodings are supported and recorded in the checkpoint – the raw weights, or a policy-friendly rescaling used in all reported runs.

Value-Concat. The control that isolates Proposition 4.1. It is identical to the above except that the intent is routed to the value stream alone, leaving the advantage stream on the unwidened context and the raw path features; by the proposition it cannot re-rank.

All five rungs were trained on the same environment, the same 28-day demand trace, and the same optimizer settings, so the ladder isolates one architectural change at a time: flat to scoring (per-path features and a variable action set), scoring to Value-Concat (intent present but path-independent), and Value-Concat to Two-Stream-Concat (intent reaching the per-path comparison).

Training details. All agents use two hidden layers of width 128, learning rate 10^{-3} , discount $\gamma = 0.95$, batch size 32, a replay buffer of 10^4 transitions, and soft (Polyak) target updates with $\tau = 0.05$. Prioritized replay uses priority exponent $\alpha = 0.6$ with the importance-sampling exponent β annealed from 0.4 to 1.0. Exploration is ϵ -greedy, decayed multiplicatively per episode. Invalid actions are masked to a large negative value both when acting and when forming bootstrap targets, so masked paths never contribute to the loss. Episodes are 24 steps (one simulated day); the flat agent trains for 533 episodes and the conditional agents for 666, with one gradient step every four environment steps. The conditional agents are trained across reward profiles on a stratified schedule (§4.2), so that within a single training run the network sees every intent an equal number of times.

This budget is short, and we state its consequences rather than leave them implicit. At one gradient step per four environment steps, 533 and 666 episodes amount to roughly 3200 and 4000 optimizer updates in total, and ϵ decays only to 0.069 (flat) and 0.036 (conditional), never reaching the stated 0.01 floor. The natural worry is that the weakest rung of the ladder is undertrained rather than architecturally limited, which would inflate the gap the ablation attributes to the architecture. The training curves (Appendix A.4) settle it: the flat DQN plateaus at a mean episode reward of +0.049 with a training loss that *rises* over the run, while the unconditioned scoring agent, given the identical 533 episodes on the identical data with the identical optimizer, reaches +0.809 – sixteen times the final reward. The flat agent’s deficit is a property of its architecture, a fixed 30-way action head over a five-dimensional global state with no per-path features, not of its budget. The conditional agents converge to +0.814 (Two-Stream-Concat) and +0.816 (Value-Concat) over 666 episodes. Every agent in the ladder was retrained under five random seeds, and §4.4.2 reports the resulting spread.

4.3.2 Evaluation Pipeline

The system is driven by a six-stage pipeline, each stage writing artifacts consumed by the next:

1. **Topology generation** – BRITE produces an AS graph, converted into SCION roles, ISDs, and links.

2. **Beaconing** – PCB propagation discovers up/core/down segments and composes candidate paths per source–destination pair.
3. **Traffic simulation** – 28 days of hourly demand with per-link aggregation, producing the time-varying link states the environment replays.
4. **Training** – all agent variants are trained on the resulting environment; checkpoints and per-episode statistics are saved.
5. **Evaluation** – every method (learned and baseline) is run over held-out pairs, hours, and intents, logging reward, achieved latency and bandwidth, probing overhead, and selection time.
6. **Figures** – results are rendered in a consistent style for the report.

Running the whole pipeline is a single command; intermediate artifacts (topology, path store, link states, checkpoints, results tables) live under a timestamped run directory so that any stage can be re-run or inspected in isolation. Key knobs – topology size, number of training episodes, the cap on training pairs, and learning hyperparameters – are exposed as configuration so that experiments can be scaled up or down.

4.3.3 Baselines and Probing Accounting

Baselines. Six heuristic selectors provide the comparison points. *Shortest-path* takes the fewest AS hops and *lowest-latency* the smallest measured delay. *Widest-path* takes the largest measured available bandwidth, and is the only baseline requiring a full bandwidth probe on every candidate. *ECMP* restricts to the minimum-hop set and pins the flow to one member by hashing its five-tuple, and the *SCION default* takes the minimum hop count with ties broken by latency. Finally *random* fixes one pseudo-randomly chosen path per source–destination pair and keeps it for the whole evaluation, which makes it a static-misconfiguration reference rather than a per-decision coin flip. Together they span the spectrum from naive to strong single-metric heuristics (widest for throughput, lowest-latency for delay), so that the learned selectors must beat not just a weak baseline but the *right* heuristic for each intent.

Oracle. None of these heuristics is an upper bound. Widest-path maximizes bandwidth, which is the correct target only under the Throughput intent; under Low-Latency it is actively wrong. We therefore also evaluate a per-context *oracle*: for each decision it enumerates the candidate set and selects the path maximizing the *true reward under the intent being scored*. It is charged one full probe, so its probing cost is comparable to the learned selector’s, and its goodput is normalized identically – by that pair’s best candidate at that hour. The oracle is not a deployable policy, since it requires perfect knowledge of every candidate’s realized metrics

before choosing; it is the per-intent upper bound that turns Table 4.2 into a normalized optimality gap and lets us say how much of the available quality each method captures.

Probing accounting. A fair comparison must charge each method for the measurements it needs. The environment therefore records, per decision, which paths a method probed and at what cost – latency probes being cheap and full bandwidth probes expensive (§4.1.2). Heuristics that require a metric on every path (e.g. widest-path) incur the cost of probing all paths, whereas the learned selector is charged only for the paths it actually inspects. This makes the probing-overhead reduction a directly measurable quantity rather than an assumed benefit, and is the mechanism by which we test the high-quality-at-low-probing claim of Bourak et al. [3].

Concretely, and since this drives the whole of Fig. 4.5: widest-path is charged a full bandwidth probe on all 30 candidates, 5388 ms per selection; every other heuristic a latency probe on all 30, 360 ms; and the learned selector a full bandwidth probe on the single path it selects, 132–139 ms depending on that path’s hop count.

One label needs correcting. Our ECMP baseline is evaluated with a constant flow identifier, so it hashes every flow of a pair onto the same member of the minimum-hop set and exhibits none of the load spreading its name implies. It is *hash-pinned shortest path* here, and we call it that where the distinction matters; we retain the ECMP label in tables for comparability with the literature.

4.4 Experimental Evaluation and Intent Alignment

This section evaluates the selectors on held-out hours. We first fix the experimental setup and metrics (§4.4.1). We then answer five questions the design raises, in turn: where must intent enter for conditioning to have any effect (§4.4.2)? Does the conditioned policy re-rank paths in the *right* direction as the intent changes (§4.4.3)? Does it generalize to intents it never trained on (§4.4.4)? Does the scoring architecture in fact deliver the dynamic-action property it was chosen for (§4.4.5)? And does it retain the high-quality-at-low-probing property of the selector it builds on (§4.4.6)? Section 4.5 then turns to the limit that motivates the next chapter.

4.4.1 Experimental Setup

Environment. Experiments use one BRITE-generated SCION topology of 90 ASes and 528 inter-AS links, partitioned into 2 ISDs with 6 core ASes. Beaconing (§4.3.2) discovers 1004 core segments and 1127 up/down segments, which compose into candidate paths for 8010 routable source–destination pairs, each with up to 30 paths. The network is driven by the 28-day hourly demand model with per-link aggregation (§4.1.2).

Train/test split. The main tables use a *temporal* split: training uses the first 14 days and every number is measured on the last 14 days (336 held-out hours), a window the agents never trained on. Unless noted, each cell aggregates 32 pairs across those 336 hours, i.e. 10752 independent decision contexts. We are explicit that this is a temporal split rather than a pair split, because the 32 evaluation pairs are drawn from the same pool the agents train on. Training episodes sample a pair uniformly from all 8010 routable pairs and visit 644 distinct ones over the run, so only 2 of the 32 evaluation pairs were ever seen; the weakness of this set is not contamination but homogeneity, since all 32 share the same source AS.

Structurally held-out pairs. To test generalization across the topology rather than across time, we additionally construct an exactly disjoint evaluation set: replaying the training RNG identifies the 644 visited pairs, and we sample 32 pairs from the remaining 7366, chosen to span 32 distinct source ASes. No agent has seen any of them. Table 4.4 reports the ablation on this set.

Intents. The conditional agent is trained across the six reward profiles of §4.2 and evaluated on the four *selection-relevant* ones: throughput-maximizing (*Throughput*), latency-averse (*Low-Latency*), loss-averse (*Low-Loss*), and a balanced midpoint (*Balanced*); their exact weight vectors are given in Table A.1. The two probe-frugality profiles are excluded from the conditioning studies because a single-path selector issues one probe per decision regardless of w_{probe} , so the weight has little leverage over the choice; probing cost is instead the subject of §4.4.6.

This exclusion needs a measured justification rather than an assertion, because probe cost is charged *per hop* (§4.1.2) and w_{probe} is therefore not strictly path-independent: a probe-frugal intent does have some incentive to prefer short paths. The incentive is real but negligible. Evaluating all six profiles as intents, the mean chosen hop count of the *oracle* – which bounds what any policy could do – moves from 1.899 under *probe_minimal* ($w_{\text{probe}} = 0.001$) to 1.881 under *probe_averse* ($w_{\text{probe}} = 0.35$), a shift of 0.018 hops across a 350-fold change in the weight. For scale, the same oracle moves 0.333 hops between *Throughput* and *Low-Latency*. Probe frugality thus commands roughly 5% of the leverage of a genuine intent axis, which is too little to discriminate between paths, and the two profiles are near-duplicates of *Balanced* by construction. They remain in the training schedule, where they cost nothing and broaden the intent space the policy is exposed to, and are excluded from the conditioning studies.

[TODO: state the seed protocol for this chapter as plainly as §5.3.1 states it for the next, because right now the asymmetry is the thesis’s own methodological lesson applied to only one of its two systems. Five seeds cover Table 4.3 and nothing else: the intent-alignment matrix (Table 4.5), the zero-shot sweep (Fig. 4.3), the path-count study (Fig. 4.4), the probing comparison (Table 4.6), and the ceiling figure (Fig. 4.6) are all read from a single reference run. Section 5.3.8 identifies exactly that practice as the source of four retractions and says the spread over training runs exceeds the evaluation-seed interval by an order of

magnitude on this class of problem. A reader who takes that lesson seriously will apply it here, and the chapter should get there first. Two routes: re-run the five affected results over the five seeds that already exist — these agents train in 1–3 minutes and the checkpoints are on disk, so this is an evaluation-only sweep of well under an hour, and it is the cheapest credibility purchase available anywhere in the thesis — or mark each affected claim as single-seed in place. The first is strongly preferable; the second is the minimum.]

Metrics. For each method we report the intent-weighted reward of Eq. (4.2) and the achieved per-flow goodput and latency. Intent alignment is assessed two ways: by reward under each intent, and by *behavioral divergence* – the degree to which the chosen path changes as the intent vector changes, holding the decision context fixed. Concretely, writing d for the number of distinct paths a method selects across the $K = 4$ intents in a given context and N for that context’s candidate count, the divergence is

$$\text{div} = \frac{d - 1}{\min(K, N) - 1} \in [0, 1], \quad (4.4)$$

averaged over contexts, so a method that ignores intent scores 0 and one that picks a different path for every intent scores 1. We also report the *choice entropy*, the Shannon entropy in bits of the distribution of choices across the four intents, which credits a method for spreading its choices evenly rather than merely for making more than one. Probing cost is measured by the per-selection probe accounting of §4.3.3.

4.4.2 Where Must Intent Enter? The Conditioning Ablation

Proposition 4.1 predicts that conditioning routed only through path-independent terms cannot change the selected path. We test that prediction, and separate it from the effect of the scoring architecture, by walking up the four-rung agent ladder of §4.3.1: a Flat DQN with a single baked-in objective; an *unconditioned* path-scoring DQN, which sees no intent at all and isolates what the architecture alone buys; a Value-Concat agent that receives the intent in its value stream only; and Two-Stream-Concat, which routes it to the per-path comparison. Table 4.2 reports the mean reward each attains under each intent, together with its adaptivity, choice entropy, and distance from the per-context oracle.

Per-path scoring buys the goodput; conditioning buys the re-ranking. The two contributions separate cleanly, and the unconditioned scoring agent is what separates them. It reaches 8.21 Gbit/s of goodput against the flat agent’s 5.62 Gbit/s, *with no intent input whatsoever* – so the large absolute gap between the flat and conditional agents belongs to the per-path architecture, not to conditioning. Over five seeds the effect is 8208 Mbit/s [8196, 8220] against 5808 Mbit/s [5711, 5906], an improvement of 41% with the intervals far apart. What conditioning adds is

Table 4.2: Intent-conditioning ablation. Mean intent-weighted reward under each of the four intents; *Adapt.* is the mean behavioral divergence of Eq. (4.4) (0 = intent-invariant path choice, 1 = a different path per intent), *Ent.* the mean choice entropy in bits, and *Params* the *Q*-network’s parameter count in thousands. The oracle is the per-context reward arg max under the intent being scored, so each column reads as a normalized optimality gap. Each reward cell aggregates 10752 decisions over 336 held-out hours from the single reference run that the rest of the chapter is also anchored on; Table 4.3 gives every method’s spread over five independent training seeds.

Method	Mean reward by intent				Intent sensitivity		
	Thpt.	Low-Lat.	Low-Loss	Bal.	Adapt.	Ent.	Params
Flat DQN	0.333	0.701	0.868	0.561	0.011	0.029	85.3 k
Scoring DQN (uncond.)	0.979	0.679	0.937	0.898	0.003	0.009	36.2 k
Value-Concat	0.979	0.708	0.937	0.894	0.050	0.130	36.9 k
Two-Stream-Concat	0.986	0.731	0.940	0.897	0.149	0.375	37.5 k
<i>Oracle</i>	<i>0.986</i>	<i>0.733</i>	<i>0.941</i>	<i>0.898</i>	<i>0.161</i>	<i>0.419</i>	–

narrower and precisely targeted. On Throughput, Low-Loss, and Balanced the unconditioned scorer is already statistically indistinguishable from the conditioned one, because those three objectives are all served by the same high-bandwidth paths. On Low-Latency – the one evaluated intent that must steer *away* from the bandwidth-maximizing default – it falls to 0.679 while Two-Stream-Concat reaches 0.731, cutting mean chosen latency from 17.0 ms to 12.7 ms at a cost of roughly 960 Mbit/s of goodput. Its adaptivity of 0.003 confirms it is exactly the intent-blind agent it should be.

The flat architecture is the wrong shape. The Flat DQN collapses onto a single trade-off: it scores 0.868 on Low-Loss but only 0.333 on Throughput and 0.561 on Balanced, and its adaptivity of 0.011 confirms it selects essentially the same path whatever intent it is asked to serve. Its reward variance under Throughput is 0.74, against 0.03 for the unconditioned scorer and 0.003 for the conditioned one – it is not merely worse on average but wildly inconsistent. As §4.3.1 establishes, this is architectural rather than a training artifact.

None of this is a capacity effect, and the direction it runs in is worth stating because it inverts the trade one would expect. The flat agent carries 85.3 k parameters; the scoring agents carry 36.2–37.5 k (Table 4.2). The architecture that handles a variable path set, serves every intent from one set of weights, and lifts goodput by 41% is *less than half the size* of the fixed-action one it replaces — because a shared per-path scorer is reused across candidates where a fixed head must allocate output capacity to each. The dynamic-input design is therefore not a

flexibility that is paid for in model size; on this problem it is cheaper on both axes at once, which is the rare case where the deployability argument and the performance argument point the same way.

Where the intent enters decides whether it acts. Both conditional agents receive the same intent vector and both recover strong reward on every intent. The decisive difference is how much they *act* on it: Two-Stream-Concat attains an adaptivity of 0.149 and a choice entropy of 0.375 bits against Value-Concat’s 0.050 and 0.130 – a factor of three in re-ranking. This is the predicted signature of Proposition 4.1: an intent entering only the value stream shifts every path’s score by the same amount and cancels under the arg max, so Value-Concat can influence its own choice only through the indirect effect of training and through the single trust feature that leaks w_3, w_4 into f_i . The behavioral consequence shows up as reward exactly where re-ranking is required – on Low-Latency, 0.731 against 0.708.

How much re-ranking is available? Adaptivity lives on $[0, 1]$, so 0.149 looks small in isolation. The oracle sets the scale: a policy with perfect knowledge, re-optimizing per context under each intent, attains an adaptivity of only 0.161 in this environment, because most contexts have no better answer to give a different intent. Two-Stream-Concat therefore captures 93% of the achievable re-ranking, and Value-Concat 31%.

Distance from optimal. Reading Table 4.2 against its oracle row, Two-Stream-Concat’s shortfall is 0.0002 on Throughput, 0.0018 on Low-Latency, 0.0012 on Low-Loss, and 0.0005 on Balanced – at most **0.25%** of the oracle’s own reward on any evaluated intent. The conditioned selector is thus near-optimal in the strict sense of being close to a per-context arg max that knows every candidate’s realized metrics in advance, while itself inspecting one path.

[TODO: lead with the absolute shortfalls, which this paragraph already has, and demote the percentage, which is a ratio on Eq. (4.2) — shifted into $[-1, 1]$ and then clamped, so its zero is an artifact of the shift and 0.25% of it denotes nothing physical. Then add the sentence that makes the result robust rather than merely impressive: state how much room there was. The oracle’s own adaptivity is 0.161 (Table 4.2), so in the large majority of contexts no policy could have done better under any intent, and three of the four evaluated intents are served by the same high-bandwidth paths. “Near-optimal on a task with little headroom” is what the data support and it is a claim nobody can attack; “within 0.25% of the oracle” invites a reader to ask what fraction of contexts were winnable at all, and the chapter should answer that before it is asked. The number — the share of contexts in which the oracle’s choice differs from the unconditioned scorer’s — is computable from artifacts already on disk.]

Reproducibility across seeds. Several of the comparisons above are small reward differences, so we retrained every agent in the ladder under five random seeds and

re-evaluated each on the identical 10752 contexts. The environment’s pair, hour, and profile streams are held fixed, so every seed sees the same stream of training contexts and only the learning process varies. Table 4.3 reports means with 95% confidence intervals on the two intents that discriminate, plus adaptivity.

Both structural claims survive with room to spare. The architectural one: on Throughput the flat agent’s interval [0.3518, 0.3942] is nowhere near the scoring agent’s [0.9754, 0.9813]. The conditioning one: on Low-Latency, Value-Concat’s [0.6996, 0.7119] and Two-Stream-Concat’s [0.7279, 0.7312] are disjoint, as are their adaptivity intervals [0.0309, 0.0541] and [0.1293, 0.1503]. So is the comparison that isolates what conditioning buys – Two-Stream-Concat’s [0.7279, 0.7312] against the unconditioned scorer’s [0.6755, 0.6841] on the one intent that conflicts with the bandwidth-maximizing default.

One ordering in Table 4.2 does *not* reproduce, and we flag it rather than leave it to be discovered: on Throughput, the reference run has Value-Concat marginally ahead of the unconditioned scorer (0.9788 against 0.9787), while over five seeds the scorer leads (0.9784 against 0.9760) with overlapping intervals. The two are indistinguishable on that intent, which is what the surrounding text claims; neither the single-run nor the five-seed ordering should be read as a result.

A paired Wilcoxon signed-rank test over the 10752 shared contexts agrees with the seed analysis on every comparison above ($p < 10^{-12}$ throughout). We note that with $n = 10752$ paired contexts almost any consistent difference reaches significance, which is why we report effect sizes, confidence intervals, and the oracle-normalized gap alongside the p -values rather than instead of them.

Generalization to unseen pairs. Table 4.4 repeats the ablation on the 32 structurally held-out pairs of §4.4.1, spanning 32 source ASes none of the agents trained on.

Two findings matter here. First, the scoring agents generalize structurally: rewards fall on Low-Latency (0.731 to 0.607), but the *oracle* falls with them (0.733 to 0.610), and Two-Stream-Concat’s optimality gap grows only from 0.0018 to 0.0033. The absolute drop is a property of what these pairs make available, not of the policy. Adaptivity in fact rises, from 0.149 to 0.190, tracking the oracle’s own rise to 0.181. Second, the flat agent does not generalize at all: its Throughput reward goes *negative*, from 0.333 to -0.085 . Its fixed, index-addressed action space is tied to the pairs it trained on, which is direct evidence for abandoning it (§4.1.1).

4.4.3 Intent Alignment

Adaptivity establishes that the conditioned policy changes its choice with intent; this section shows that it changes it in the *right* direction. We evaluate the single trained checkpoint under each intent and measure both the reward it earns and the metrics of the paths it selects.

Figure 4.1 and Table 4.5 report the reward $R(i, j)$ obtained when the agent is told intent i but its choice is scored under objective j . Reading down a column

Table 4.3: Seed variance over five training runs per method, evaluated on the same 10752 held-out contexts. Cells are mean [95% confidence interval]. Both of the chapter’s structural comparisons reproduce with disjoint intervals: flat against per-path scoring on Throughput, and value-stream against per-path conditioning on Low-Latency and on adaptivity.

Method	Throughput reward	Low-Latency reward	Adaptivity
Flat DQN	0.3730 [0.3518, 0.3942]	0.7026 [0.6918, 0.7134]	0.0134 [0.0032, 0.0235]
Scoring DQN (uncond.)	0.9784 [0.9754, 0.9813]	0.6798 [0.6755, 0.6841]	0.0056 [0.0033, 0.0078]
Value-Concat	0.9760 [0.9722, 0.9798]	0.7058 [0.6996, 0.7119]	0.0425 [0.0309, 0.0541]
Two-Stream-Concat	0.9839 [0.9815, 0.9864]	0.7296 [0.7279, 0.7312]	0.1398 [0.1293, 0.1503]
<i>Oracle</i>	<i>0.9861</i>	<i>0.7330</i>	<i>0.1609</i>

Table 4.4: The same ablation on 32 pairs across 32 distinct source ASes, none visited during training. Absolute rewards fall on Low-Latency, but so does the oracle’s, so the drop reflects these pairs’ available headroom rather than a generalization failure. The flat agent is the exception: its Throughput reward goes negative.

Method	Thpt.	Low-Lat.	Low-Loss	Bal.	Adapt.
Flat DQN	−0.085	0.517	0.652	0.246	0.008
Scoring DQN (uncond.)	0.978	0.557	0.887	0.857	0.010
Value-Concat	0.980	0.592	0.888	0.854	0.070
Two-Stream-Concat	0.984	0.607	0.889	0.857	0.190
<i>Oracle</i>	<i>0.985</i>	<i>0.610</i>	<i>0.892</i>	<i>0.858</i>	<i>0.181</i>

asks: to do well under this objective, which intent should the agent be given? The answer is the matching one in all four columns, which is the Intent Alignment property of §3.2. For Throughput the margin is decisive – being told “Throughput” scores 0.986, against 0.734 when told Low-Latency. For Low-Latency the matched intent again wins, and the mismatch penalty is largest relative to the column’s range, precisely because latency is the objective that most demands re-ranking.

The margins on the remaining two columns are small, and we report that plainly: the Low-Loss column spans 0.011 across all four intents, so while the diagonal wins it, no reader should take that as strong evidence. The Balanced column spans more (0.115), but almost all of it comes from the Low-Latency intent scoring badly there rather than from the other intents separating.

Why the loss axis is degenerate, and why better profiles would not fix it. The reason Low-Loss discriminates so weakly is not a poor choice of profile but the environment: across all 43008 selections, **98.85%** of chosen paths have exactly zero loss, so there is almost nothing for a loss-averse intent to avoid. Consistently, the

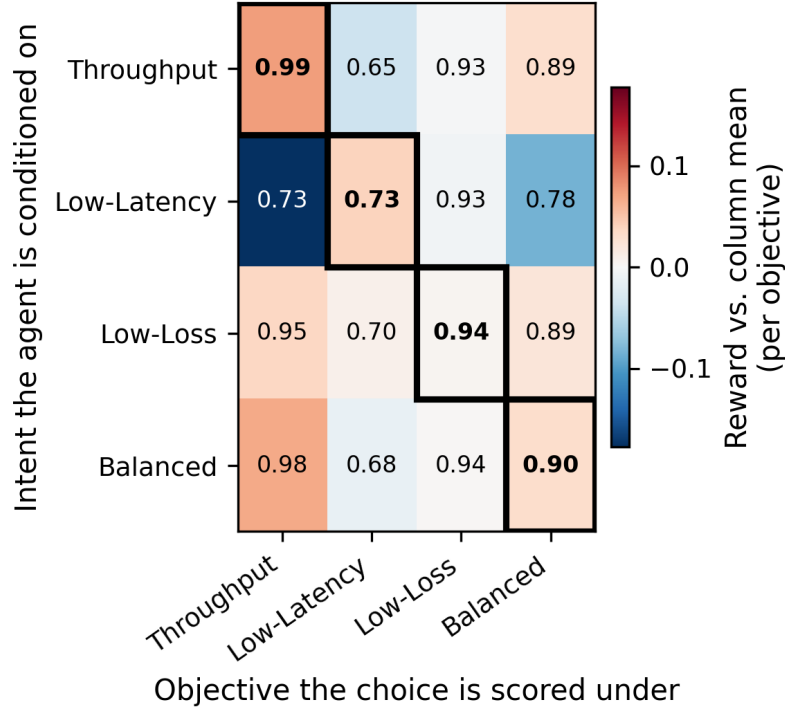


Figure 4.1: Reward matrix $R(\text{intent told}, \text{objective scored})$ for the conditioned policy. Cell color is the reward relative to its column mean, so a red diagonal means that conditioning on an intent is the best way to score well under that same objective. The diagonal wins every column; the margin is large on the two objectives that conflict with the high-bandwidth default and small on the two that do not. Table 4.5 gives the underlying numbers.

latency-optimal and loss-optimal paths coincide in 82.8% of contexts, whereas a genuine bandwidth/latency conflict exists in 59%. It is tempting to propose replacing Low-Loss with a profile whose optimum conflicts with high bandwidth – a hop-count-averse or trust-dominant intent – but the reward function of Eq. (4.2) exposes only three levers, and hop count enters solely through the probe charge. We tested two such candidates against the *oracle*, which bounds what any policy could achieve: a `probe_extreme` profile at sixteen times the `probe_averse` weight moves the oracle’s chosen hop count by 0.009 hops, and a pure loss-aversion profile with the latency term removed collapses onto the throughput optimum, matching it to within 2 Mbit/s of goodput. Since the oracle bounds every policy, no reweighting of this reward can produce a fourth discriminating intent. The honest statement is therefore that *this environment offers one genuine behavioral trade-off axis, goodput against latency*, that intent alignment is demonstrated on it, and that making loss or hop count discriminating requires changing the reward or the environment – adding an explicit hop-count term, or raising offered load

Table 4.5: The reward matrix behind Fig. 4.1: mean reward when the policy is told the row intent and scored under the column objective, over 10752 contexts. Column maxima in bold; the diagonal attains all four. Note the very different column ranges, which the figure’s column-relative coloring deliberately hides.

Told	Scored under			
	Throughput	Low-Latency	Low-Loss	Balanced
Throughput	0.9860	0.6549	0.9333	0.8949
Low-Latency	0.7341	0.7312	0.9293	0.7824
Low-Loss	0.9478	0.7010	0.9398	0.8877
Balanced	0.9781	0.6786	0.9371	0.8974
Column range	0.252	0.076	0.011	0.115

until loss is common, given that only 7.8% of links are oversubscribed today. We return to this in Chapter 8.

Figure 4.2 shows the same behavior in the physical metrics of the chosen paths. Holding the decision contexts fixed and varying only the intent, the Low-Latency intent shifts the chosen-latency distribution down markedly (mean 12.73 ms against 18.40 ms under Throughput and roughly 16 ms under the other two), and pays for it in goodput – 7.25 Gbit/s on average against 8.24 Gbit/s. The Throughput intent shifts the chosen-goodput distribution up, though only slightly, because three of the four intents already converge on the widest paths.

The trust panel does *not* behave as the intent labels would suggest, and the discrepancy is instructive rather than a defect. The highest-trust selections come from the *Low-Latency* intent (mean 0.934), not the Low-Loss intent (0.922, barely above Balanced’s 0.915 and Throughput’s 0.908). This follows directly from Eq. (4.1): we score trust with canonical weights that penalize loss and delay equally, so the short paths a latency-averse intent selects maximize it, whereas the Low-Loss profile weights loss heavily and delay barely and therefore optimizes a quantity the canonical trust score largely ignores. The fourth panel makes the point directly: the Low-Latency intent is the only one that moves loss exposure at all, and it moves it *up*, to 4.91% of selections incurring any loss against the 0.48% floor that the other three intents all sit on. Buying 5.7 ms of latency means accepting shorter, more heavily loaded paths. The mechanism works; it is the loss axis that has nothing to give.

4.4.4 Zero-Shot Generalization to Unseen Intents

Every intent evaluated so far is one of the six profiles the agent trained on. That establishes only that a single network can store six behaviors – not that it has learned a *mapping* from the intent space to behavior. This section tests the stronger

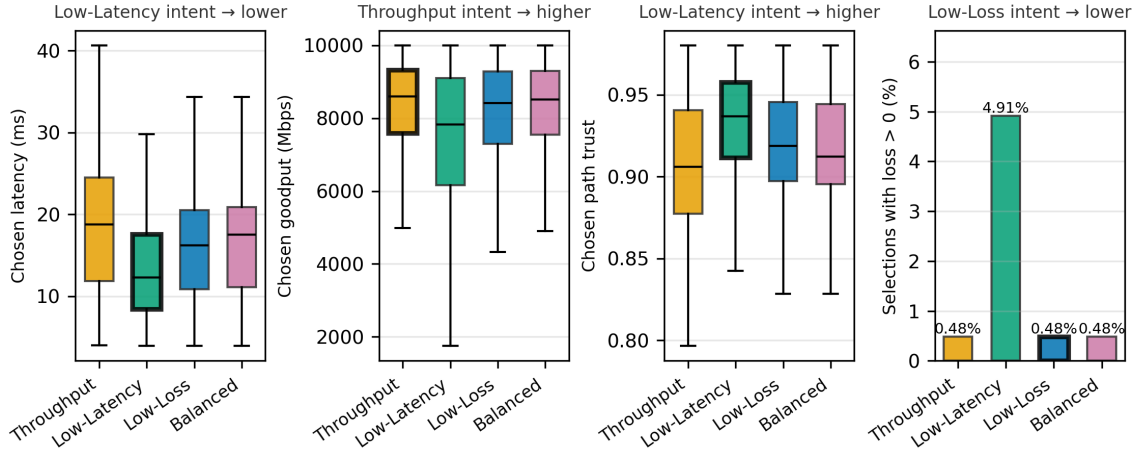


Figure 4.2: Distributions of the chosen path’s metrics as the conditioned policy is told each intent, over a fixed set of decision contexts. The Low-Latency intent lowers the chosen latency and the Throughput intent raises the chosen goodput. Trust is scored with the canonical weights of Eq. (4.1) and is therefore maximized by the Low-Latency intent, not the Low-Loss one; the fourth panel reports loss exposure directly, since a boxplot of loss rate is a flat line at zero.

property that §3.2 requires: presented with a weight vector it has never seen, does the policy behave in the correspondingly interpolated way, without retraining?

We sweep the intent along the segment between the two profiles that genuinely conflict,

$$\mathbf{w}(t) = (1-t) \cdot \mathbf{w}_{\text{Throughput}} + t \cdot \mathbf{w}_{\text{Low-Latency}}, \quad t \in \{-0.2, -0.1, 0.0, 0.1, \dots, 1.1, 1.2\}, \quad (4.5)$$

and evaluate each of the 15 resulting weight vectors on the same 10752 held-out contexts. Only $t = 0$ and $t = 1$ are vectors any agent has ever seen; the eleven interior points are unseen interpolations and the two outside $[0, 1]$ are extrapolations beyond both trained profiles.

Figure 4.3 shows the result, and it is unambiguous. The conditioned policy’s mean chosen latency falls monotonically from 18.40 ms at $t = 0$ to 12.73 ms at $t = 1$, with a Spearman rank correlation against t of exactly -1.000 . Monotonicity alone is a weak test, so we also ask how the change is distributed: no single step of the sweep carries more than 24.7% of the total span. A policy that had memorized six points would instead sit at one endpoint behavior until the weight vector crossed some threshold and then jump, concentrating essentially the whole span in one step. The endpoints reproduce the trained-profile behavior exactly, matching the Throughput and Low-Latency rows of §4.4.3, and the extrapolated points continue the trend rather than diverging – 12.05 ms at $t = 1.2$, beyond any trained intent.

Two further observations. First, the same sweep on the 32 structurally held-out pairs of §4.4.1 reproduces the result: 24.76 ms to 19.16 ms, again monotone with

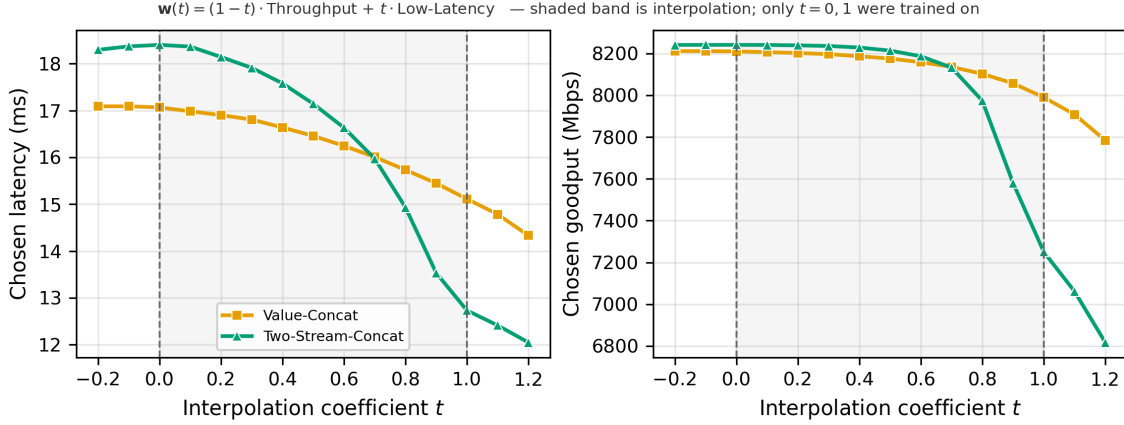


Figure 4.3: Behavior of the conditioned policies along the unseen intent segment of Eq. (4.5). Only the two dashed verticals were trained on; the shaded band is interpolation and the flanks are extrapolation. Chosen latency falls monotonically and smoothly, which is the signature of a learned mapping rather than of memorized endpoints. Value-Concat interpolates smoothly too, but traverses less than half as much behavior, as Proposition 4.1 predicts.

$\rho = -1.000$. This is *unseen intents on unseen pairs simultaneously*, and it is the strongest generalization evidence in the chapter. Second, the Value-Concat control interpolates smoothly as well, but over a span of only 2.0 ms against Two-Stream-Concat’s 5.7 ms. This is Proposition 4.1 showing up from a third direction: a policy whose conditioning cannot re-rank directly can still shift behavior through the indirect effects of training, but it covers less than half as much of the behavior space.

We therefore claim zero-shot generalization in a specific and bounded sense: interpolation, and modest extrapolation, along the segment between two trained profiles, on one topology. We have not tested weight vectors far outside the trained convex hull, nor a second topology.

4.4.5 Variable Path Sets

The scoring architecture of §4.1.4 was adopted so that one policy could serve pairs with any number of candidate paths, in any order, without padding or retraining. So far that claim rests on the architecture’s construction. This section measures it.

Binning the held-out decisions by their natural candidate count is uninformative here, because 7988 of the 8010 routable pairs have exactly 30 paths – the topology simply does not vary N . We therefore restrict the candidate set *at decision time* to $N \in \{2, 4, 8, 16, 30\}$, presenting every method the identical subsets, and measure each method’s regret against the best path available within that restricted set.

This is a harder test than natural variation would give, since the agents trained exclusively on 30-candidate sets and every smaller value of N is off-distribution.

Figure 4.4 reports the result. Every scoring agent holds mean regret between 0.0001 and 0.013 across the whole range, while the flat DQN sits between 0.24 and 0.50 – two orders of magnitude worse, and worst in the middle of its range, where the padded action head is least well matched to the number of real alternatives. One model, no retraining, no padding, across a 15-fold change in the action count.

One caveat is worth stating rather than smoothing over: the scoring agents are slightly *better* at $N = 30$ (0.0001–0.004) than at small N (0.005–0.013). They trained only on 30-candidate sets, and the per-path bandwidth-ratio feature is normalized by the maximum within the presented set, so subsampling shifts the observation distribution off what they saw. The effect is real but bounded at about 0.01 reward: the architecture handles unseen N gracefully rather than perfectly.

Order invariance. The second half of the property is that the choice must not depend on the order in which candidates happen to be enumerated. We re-scored each of the 10752 contexts under three random permutations of the path ordering, 32256 trials per agent, and checked whether each agent selected the same *underlying path*. All four scoring and conditional agents did so in **100.0000%** of trials – exact invariance, not approximate. The flat DQN is excluded by construction: its action is an index into a padded vector, so permuting the candidates changes what it selects, which is precisely the architectural point.

4.4.6 Probing Overhead

A path selector is only useful if its selection quality does not come at the price of measuring every path on every decision. Using the probing accounting of §4.3.3, we compare the conditioned selector against the six heuristics – first under the Balanced intent, where goodput and reward are directly comparable, then under all four.

Figure 4.5 places each method in the quality–cost plane. Widest-path attains the highest goodput (8.24 Gbit/s) but must issue a full bandwidth probe on all 30 candidate paths, at 5388 ms of probing per selection. The learned selector reaches 8.21 Gbit/s – within 0.5% of widest-path – while inspecting only the single path it selects, at 137 ms per selection: a reduction in probing cost of roughly $39\times$ at matched goodput, and *one probe per decision rather than thirty*.

The learned selector is in fact the cheapest method in the comparison, not merely cheaper than the strongest one. Every load-blind heuristic needs its metric on all 30 candidates before it can apply its rule, so each pays 360 ms of latency probing per selection – about $2.6\times$ the learned selector’s cost – and then chooses worse: ECMP, the strongest of them, reaches only 6.6 Gbit/s, and the SCION default, shortest-path, and lowest-latency deliver 5.8–6.1 Gbit/s, roughly 20–40% less goodput, because none of them inspects bandwidth.

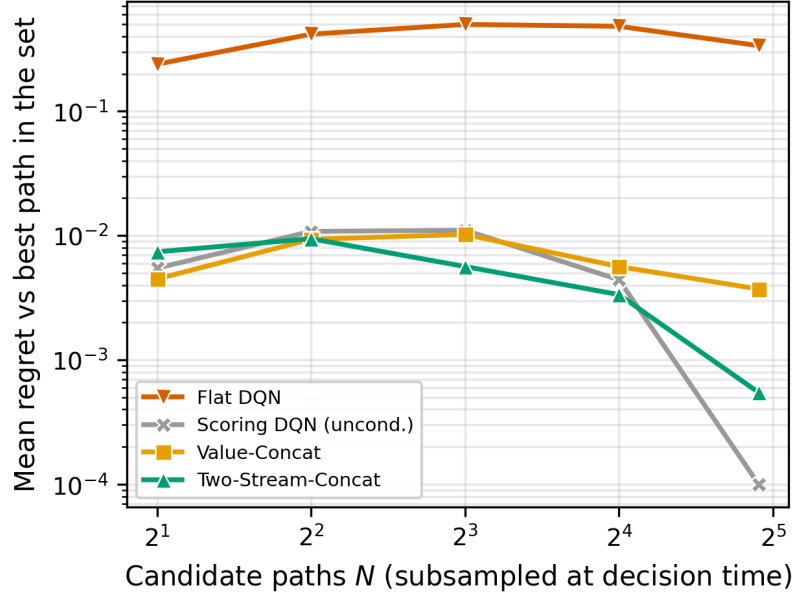


Figure 4.4: Mean regret against the best path in the presented candidate set, by set size (log scale, 10752 decisions per point, Balanced intent). One model per method, no retraining and no padding, across a 15-fold range in N . Every scoring agent stays two orders of magnitude below the flat DQN, which must address a fixed 30-way action head and degrades as N moves into the middle of its range.

The strongest heuristic changes with the intent. Running the comparison only under Balanced would understate the result, because Balanced is the one intent for which a single heuristic – widest-path – is close to the right rule. Table 4.6 repeats it under all four intents on the same contexts.

The pattern is the chapter’s strongest practical result. One conditioned policy *beats every heuristic* on three of the four intents – Throughput (0.986 against widest-path’s 0.982), Low-Loss (0.940 against 0.930), and Balanced (0.897 against 0.891) – and on the fourth it is within 0.0015 of lowest-latency, the heuristic purpose-built for that objective. Meanwhile each heuristic collapses under the other’s objective: widest-path falls to 0.652 under Low-Latency, and lowest-latency to 0.449 under Throughput. Selecting the right heuristic per intent would require knowing the intent *and* switching mechanism; the learned selector absorbs both into one policy, at one probe instead of thirty and at a lower absolute measurement cost than any of them. For the conditioned selector we therefore recover, and extend across intents, the high-quality-at-low-probing property of Bourak et al. [3].

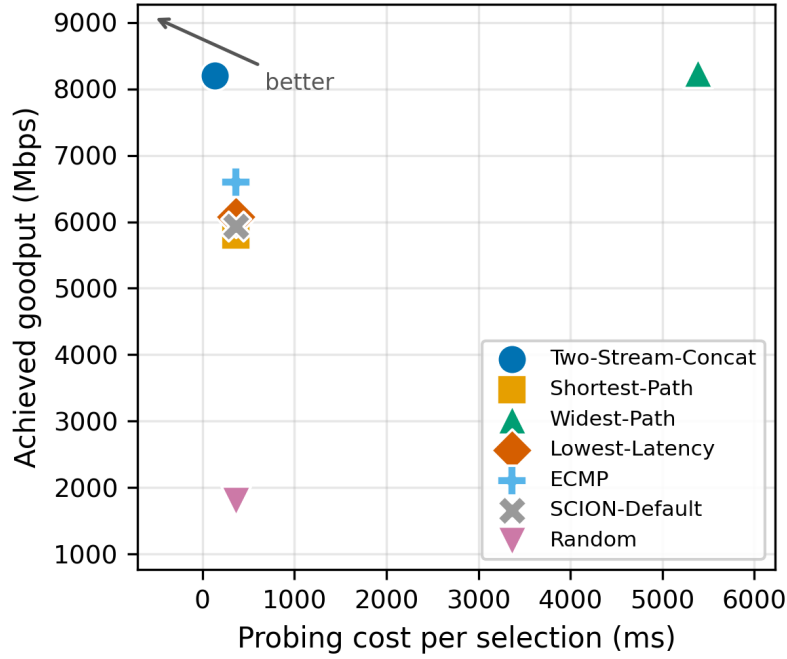


Figure 4.5: Achieved goodput against probing cost per selection under the Balanced intent (top-left is better). The learned selector reaches the goodput of exhaustive widest-path selection at roughly 40× less probing, because it inspects the one path it selects rather than all 30. It is also the cheapest method on the plot: every load-blind heuristic must measure all 30 candidates to apply its rule, and so probes about 2.6× more than the learned selector while choosing worse.

4.5 The Single-Path Performance Ceiling

The results so far show that one conditioned policy can serve the evaluated intents at low probing cost. This final experiment asks the question that motivates the next chapter: how far does the best single-path choice carry an application as the network fills up? We bin every held-out decision into six equal-count bins by its realized congestion – the mean utilization across the flow’s candidate paths at that hour – and plot the goodput each method achieves against it (Fig. 4.6). Each bin holds 1792 decisions per method. Because the environment allows links to be oversubscribed, capping utilization at 1.5 rather than 1.0 (§4.1.2), the upper three bins sit above 1.0: they are hours in which the aggregate demand offered to a flow’s candidate paths exceeds their capacity.

Two effects appear in the goodput panel. First, the learned selector tracks the widest-path ceiling across the entire congestion range, confirming that it makes near-optimal use of whatever single-path capacity exists, while the load-blind heuristics fall away sharply – shortest-path, lowest-latency, and the SCION default lose roughly 55–58% of their goodput from the lightest to the heaviest

Table 4.6: Mean intent-weighted reward by method and intent, with measurement cost, over the same 10752 contexts. Column maxima in bold. No heuristic is strong under more than one objective: each collapses under the other’s. The learned selector is the only method that is at or near the top of every column, and it is the cheapest method in both probes and milliseconds.

Method	Measurement cost		Mean reward by intent			
	Probes	ms/sel.	Thpt.	Low-Lat.	Low-Loss	Bal.
Two-Stream-Concat	1	132–139	0.986	0.731	0.940	0.897
Widest-Path	30	5388	0.982	0.652	0.930	0.891
Lowest-Latency	30	360	0.449	0.733	0.899	0.629
ECMP	30	360	0.577	0.711	0.912	0.693
SCION-Default	30	360	0.417	0.726	0.893	0.610
Shortest-Path	30	360	0.383	0.721	0.888	0.590
Random	30	360	−0.569	0.247	0.558	−0.088

congestion bin, ECMP degrades more gracefully but still loses about 39%, and random collapses entirely. Second, and crucially, *the ceiling itself descends*: both the learned selector and widest-path drop from about 9.1 Gbit/s at light load to about 6.9 Gbit/s at heavy load, a loss of 24%. This is the single-path performance ceiling. When the shared bottleneck is congested, the best available path is still one path, and one path can carry only so much; no amount of intent-aware selection recovers capacity that a single path cannot provide. Escaping this ceiling requires spreading a flow across several paths at once – the problem Chapter 5 takes up.

[TODO: this is the thesis’s structural bridge and it is currently an argument presented in the register of a result. Three things weaken it and all three are fixable in text. (i) The ceiling is measured in a model that cannot represent the remedy: the link model is open-loop (§4.1.2), so a multipath flow’s own contention is not representable, and nothing in this environment demonstrates that splitting recovers any of the lost 24%. (ii) Chapter 5 then answers the question on a different data plane with a different topology, different traffic, and a different reward, so no quantity crosses the chapter boundary — the bridge is conceptual, not empirical. (iii) The descent is in residual bottleneck capacity, not in delivered throughput. Say all three plainly and the bridge holds as what it is: a well-motivated argument that the action space, not the policy, is the binding constraint, which is exactly what the right panel of Fig. 4.6 shows. The experiment that would convert it into a result is worth naming even if it is not run: replay the same heavy-congestion contexts allowing the flow to be split across the top-k candidates under the same link model, and report the aggregate residual capacity available to a split against a single choice. It needs no learning and no new environment — only the existing path store and link states — and it would let this chapter close its own argument rather than deferring it.]

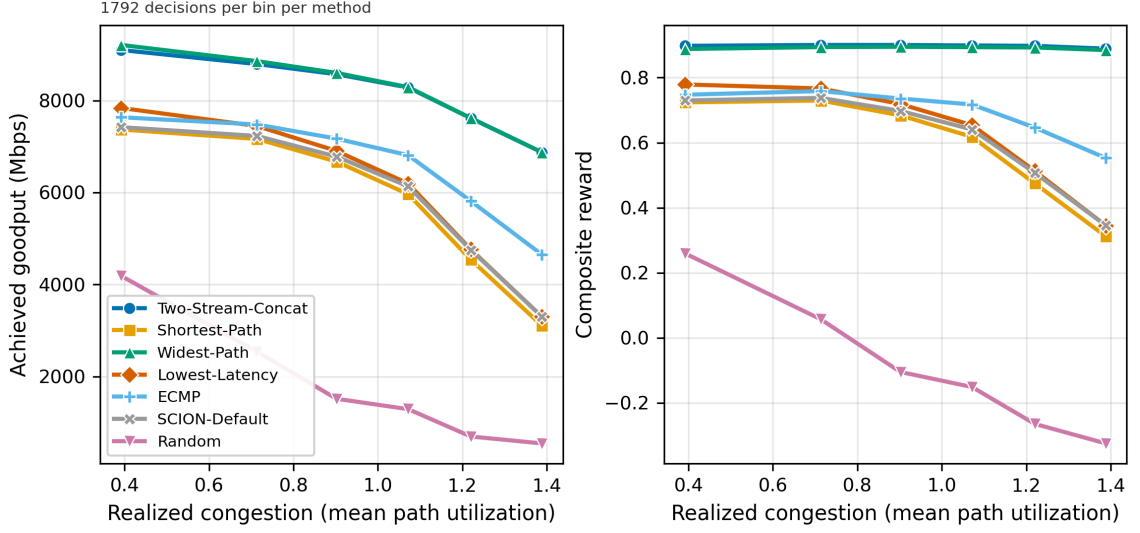


Figure 4.6: Achieved goodput (left) and intent-weighted reward (right) against realized congestion, 1792 decisions per bin per method. The learned selector tracks the widest-path ceiling at every load, but that ceiling itself falls by about 25% from light to heavy congestion: past a point, no single path can carry the demand. The right panel is the bridging argument of this thesis in one picture – the selector’s *reward* stays flat while its goodput descends, because the reward measures selection quality and is blind to the ceiling by construction. The policy is doing its job; the action space is the limit.

Why the reward does not see the ceiling. The right panel of Fig. 4.6 tells the complementary story, and it is worth stating explicitly because a reader who takes the reward as an absolute performance measure will find the two panels contradictory. The learned selector’s reward is essentially flat across the congestion range, 0.898 to 0.889, while its goodput falls by a quarter. This is a direct consequence of Eq. (4.2), in which goodput is normalized against the best candidate path at the same hour (§4.1.3): a policy that keeps choosing the best available path scores near the top whatever that path can carry. The heuristics, which do not keep choosing the best available path, collapse on reward as well – shortest-path from 0.72 to 0.31, and the fixed-random baseline from 0.26 to -0.32 . The ceiling is therefore invisible to the objective the agent optimizes, which is precisely why breaking it takes a change of *action space* rather than a better policy or a better reward.

Summary. This chapter established four claims. First, a fixed-objective, fixed-action DQN is the wrong shape for this problem: replacing it with a permutation-equivariant per-path scorer raises goodput by 41% over five seeds, from 5.81 Gbit/s to 8.21 Gbit/s with disjoint confidence intervals, gives exactly invariant selection

under 32256 permutation trials, and holds regret flat from $N = 2$ to $N = 30$, while the flat agent’s Throughput reward goes negative on pairs it never trained on. Second, making one policy serve many intents is not a matter of feeding it the intent: by Proposition 4.1, conditioning that enters only path-independent terms cancels under the $\arg \max$, and over five seeds the value-stream variant re-ranks roughly three times less than one whose conditioning reaches the per-path comparison, with disjoint confidence intervals. Third, the conditioned policy generalizes over the intent *space* rather than merely across trained points: behavior interpolates monotonically between trained intents ($\rho = -1.000$) and does so on pairs never seen in training. Fourth, it does this within 0.25% of a per-context reward oracle on every evaluated intent, beating every heuristic on three of four intents while issuing one probe instead of thirty.

Two scoping caveats belong with these claims. This environment offers exactly one genuine behavioral trade-off axis – goodput against latency – so intent alignment is demonstrated there rather than across all four evaluated profiles, and §4.4.3 shows why no reweighting of this reward could change that. And every result rests on a single topology. What survives both caveats is the chapter’s final finding: even a near-optimal single-path selector loses about a quarter of its goodput under heavy congestion, because a single path is a hard capacity bound – the ceiling that Chapter 5 sets out to break.

5 Joint Application–Network Optimization over Multipath

Chapter 4 chose a single path for a flow and found its ceiling: when the shared bottleneck saturates, even the best single path cannot carry the demand. This chapter starts from a more ambitious premise. SCION offers not one path but several, so why commit to one? And if Mortise [16] can tune a *transport mechanism* to the application, why not tune the *application itself* in the same breath? We design and evaluate a system that (i) uses multiple paths at once via Multipath QUIC [5] and (ii) jointly optimizes what the application sends – the video encoder’s target bitrate – and how the network spreads it – the per-path traffic split – to maximize the quality of experience of a real-time video call. Section 5.1 presents the design, §5.2 the testbed, and §5.3 the evaluation.

[TODO: declare the substrate here, in the first paragraph, because the thesis title promises SCION and this chapter does not contain any. There is no beaconing, no ISD structure, no path store, no segment composition, and no relationship to the topology of Chapter 4: the six paths are hand-specified per-path bottleneck rates and delays (Table A.4) and path “churn” is a hazard process, not a control-plane event. What the chapter actually uses is the path multiplicity a path-aware architecture makes routine, plus the assumption that an endpoint may send on several paths at once. That is a legitimate and ordinary abstraction for a transport study — the question here is scheduling and rate control, not path discovery — but it must be stated rather than left for a reader to infer from §5.2, and it should be stated in the sentence that opens the chapter rather than in a limitations list forty pages later. Two consequences worth naming with it: the results are portable to any multipath substrate, which is a strength, and the interaction between SCION path selection and multipath scheduling — the natural union of this thesis’s two halves — is consequently untested, which is the honest form of the future-work item in Chapter 8.]

Three findings organize the chapter. Neither decision can be made well alone: removing either agent forfeits most of the achievable quality. The value of making them jointly is governed not by whether the network is moving but by how unevenly capacity is distributed across the paths and by how strictly the application prices delay. And a *single* policy conditioned on the application’s intent does the work of three separately trained specialists, which is where this chapter rejoins the thesis’s central claim – and where it also finds that claim’s sharpest limit, since the conditioning result comes with a proposition whose premise the deployed architecture measurably violates.

5.1 System Design: Co-Optimizing Rate and Traffic Split

Section 5.1.1 motivates the joint, multipath framing and the strawmen it improves on. Section 5.1.2 presents the two-timescale hierarchical agent pair. Section 5.1.3 specifies the observations, actions, and rewards. Section 5.1.4 explains the coupling that keeps the two agents cooperating rather than working at cross purposes. Section 5.1.5 extends the scoring idea of Chapter 4 from a discrete choice to a continuous split over a variable, possibly changing, path set.

5.1.1 Motivation and Strawmen

Interactive video is a demanding, ubiquitous workload with no playback buffer to hide variance: a frame that misses its deadline is simply lost. Its QoE depends on two decisions that are usually made in isolation. The *application* chooses an encoding bitrate – too high and the network drops frames, too low and quality suffers. The *network* chooses how to carry the resulting bytes – over one path or spread across several. Optimizing either alone is a strawman:

Application-only. Adapt bitrate to the aggregate network but carry it over a fixed or naive path assignment. This wastes SCION’s path diversity and, as our results show, collapses when the paths it depends on degrade.

Network-only. Schedule cleverly across paths but treat the offered load as exogenous. The scheduler cannot relieve congestion it is itself creating by admitting too high a bitrate.

The two decisions are coupled through the same bottleneck: bitrate sets the load the scheduler must place, and the scheduler’s success sets the effective capacity the bitrate controller sees. Optimizing them jointly is the design’s premise. The challenge is that they naturally operate on *different timescales* – an encoder should not change bitrate every frame, but a scheduler must react per frame – which motivates a hierarchy rather than a single monolithic agent.

5.1.2 Two-Timescale Hierarchical Agents

We split control into two cooperating agents that act at different cadences, a “brain and body” decomposition.

Application agent (slow). Every 1 s (once per 30 frames at 30 fps) it observes an aggregate summary of the connection and outputs a single target encoder bitrate in the range 300–6000 kbit/s. It is deliberately *horizon-agnostic* – it never observes how far along the call is – so the same policy applies to a call of any length, an essential property for a controller meant to run in unbounded real sessions.

Transport agent (fast). Every frame (≈ 33 ms) it observes per-path transport state and the application agent’s current target bitrate, and outputs a split of that frame’s bytes across the available paths.

Both are trained with soft actor-critic (§2.3), whose continuous action support and sample efficiency suit these controls and the cost of simulator rollouts.

[TODO: this design has no diagram, and neither does anything else in the thesis — there is not one block diagram, architecture sketch, or schematic in eight chapters, which is unusual enough in a systems thesis that a reader will notice it before they notice any individual result. The two-timescale pair is the chapter’s central object and is currently conveyed entirely in prose across Sections 5.1.2–5.1.4. One figure carries all of it: the two agents with their cadences (1 s against ≈ 33 ms), what each observes, the bitrate arrow from application to transport that makes the pair hierarchical rather than merely concurrent, the two reward signals and what each is credited for, and the shared bottleneck that couples them. It is a half-day in TikZ and it is the difference between a reader who follows the design and one who reconstructs it. Two companions are worth the same effort if time allows: the shared per-path encoder and softmax of §5.1.5, which is the architectural claim the chapter’s largest measured effect rests on, and a SCION path-composition sketch in §2.1.]

5.1.3 Observations, Actions, and Rewards

Application observation and action. The application agent sees a small fixed-size vector: normalized current bitrate, smoothed RTT, jitter, loss, and throughput. Its action is one continuous scalar in $[-1, 1]$, mapped affinely to a target bitrate in 300 – 6000 kbit/s.

Transport observation and action. The transport agent sees a global block (target bitrate, RTT, loss, throughput) and, for each path, a feature vector (congestion window, smoothed RTT, buffer occupancy, per-path throughput, per-path loss). Its action is an N -vector passed through a softmax to yield a split $(w_1, \dots, w_N)$ that is non-negative and sums to one – a valid allocation by construction.

Rewards. The rewards encode the QoE objective and the division of responsibility between the agents. The application’s per-window QoE reward composes a quality term with penalties:

$$\text{QoE} = a \cdot \frac{\text{VMAF}(\cdot)}{100} - b \cdot \widetilde{\text{lat}} - c \cdot \widetilde{\text{jit}} - d \cdot \text{loss} + e \cdot (1 - \text{loss}) \cdot \widetilde{\text{util}}, \quad \text{clipped to } [-2, 1], \quad (5.1)$$

where $\widetilde{\text{lat}}$ and $\widetilde{\text{jit}}$ are latency and jitter normalized by lat_{norm} and jit_{norm} and clipped, and $\widetilde{\text{util}}$ is the achieved bitrate normalized by 3300 kbit/s – the knee of the quality curve. The utilization term restores the rate-control gradient under a quality

model that saturates in bitrate; it carries $e = 0.25$ in the dynamic scenario and is harmless under the default logarithmic curve. The quality term uses a VMAF surrogate (§5.2); when a learned surrogate that already accounts for loss is used, the explicit loss penalty is dropped to avoid double-counting.

The weight vector (a, b, c, d, e) together with the frame deadline is not a constant of the system but the *application intent* this thesis is about, and the three profiles evaluated in §5.3 instantiate it at three points of the ITU-T G.1010 use-case space (Table 5.1). The transport agent receives a per-frame reward that credits delivery without quality, since it does not control bitrate:

$$R_{\text{transport}} = (1 - \text{loss}) - b \cdot \widetilde{\text{lat}} - c \cdot \widetilde{\text{jit}}, \quad \text{clipped to } [-2, 1]. \quad (5.2)$$

[TODO: document the provenance of these weights, which is a methodological contribution the chapter currently omits entirely. The delay anchors come from ITU-T G.114 (150 ms preferred, 400 ms unacceptable) and the jitter normalizer from Y.1541 class-0 IPDV; the pair (b, d) is then fit rather than hand-set, by grid-correlating the linear reward of Eq. (5.1) against an ITU-T G.1070 opinion-model oracle over a trace corpus (`rl-mpquic/src/ns3env/g1070.py`, `scripts/calibrate_reward.py`; protocol in `docs/REWARD_TUNING.md` §5). Report two things with it: that the fit is corpus-insensitive (synthetic and real-trace corpora both land on $b = c = 0.25$, $d = 2.0$), and the stability gate of §5C – a weight set that changes which policy is optimal is rejected regardless of its correlation. That gate is what the static scenario fails (§5.3.2), and stating it before the results turns an awkward negative into a methodological point.]

5.1.4 Coupling the Two Agents

The design’s subtlety is keeping the agents cooperative. Two mechanisms couple them.

Observation coupling. The transport agent observes the application’s current target bitrate. It therefore schedules *knowing* how much load it must place, and can, for instance, spread an aggressive bitrate across more paths rather than overrunning one. This is what makes the pair hierarchical rather than two agents optimizing in ignorance of each other.

Reward decomposition. The application owns quality (the VMAF term) and is credited for the latency, jitter, and loss experienced over the window its bitrate governed; the transport agent is credited only for how well it delivered each frame. This assigns each decision the consequences it can actually control – the encoder is not blamed for a poor split, nor the scheduler for an over-ambitious bitrate – while the shared bottleneck still couples them: a bad split shows up as latency/loss in the application’s reward, and an over-high bitrate shows up as loss the transport agent cannot schedule away.

Table 5.1: The three application intents evaluated in §5.3, each a bundle of QoE weights (Eq. (5.1)), normalizers, and a frame deadline, keyed to ITU-T G.1010 use-case classes. Source: `rl-mpquic/configs/profiles/*.yaml`. The delay weight b falls and the deadline widens as the role relaxes; loss stays dominant for the two content-critical roles. These play the part the reward profiles of Table A.1 play in Chapter 4.

Intent	G.1010 class	Deadline	b	c	d	lat_{norm}
interactive	conversational video	180 ms	0.25	0.25	2.0	200 ms
presenter	one-way video	400 ms	0.20	0.20	2.0	400 ms
passive	background/streaming	800 ms	0.10	0.30	0.7	800 ms

5.1.5 Continuous Scoring over a Variable Path Set

The transport agent inherits the single-path selector’s problem – a variable, possibly changing number of paths – and its solution, adapted from a discrete choice to a continuous allocation. A shared per-path encoder maps each (global, path_i) pair to a logit; the logits are softmaxed into the split, so the policy is permutation-equivariant and independent of the path count. To handle paths that appear and disappear, each path carries a *liveness* flag and the state includes a binary mask; the softmax excludes dead paths, so traffic is never allocated to a path that is gone. The critic aggregates per-path embeddings with a masked mean-pool – a Deep Sets [22] operation – yielding a value estimate invariant to path ordering and count. This continuous scoring agent is the natural sibling of the discrete scoring selector of Chapter 4: the same architectural idea, once for “pick a path” and once for “divide across paths.”

5.1.6 Where Intent Must Enter a Continuous Allocation

Proposition 4.1 showed that conditioning entering only path-independent terms cancels under the $\arg \max$ and cannot change a discrete selection. The allocation here is not an $\arg \max$ but a softmax, so the question must be asked again – and the answer is sharper, because softmax is exactly invariant to a constant added to every logit.

Proposition 5.1 (Cancellation under a softmax allocation). *Let a transport policy allocate traffic by $\mathbf{p} = \text{softmax}(z_1, \dots, z_N)$ with logits $z_i = f(s, f_i) + c(s, \mathbf{w})$, where f_i is path i ’s feature row and the intent $\mathbf{w}$ enters only through the term c , which does not depend on i . Then $\mathbf{p}$ is independent of $\mathbf{w}$: the policy consumes the intent and emits an identical split.*

The proof is one line. Writing $z_i = \hat{z}_i + c$ with $\hat{z}_i = f(s, f_i)$,

$$\text{softmax}(\hat{\mathbf{z}} + c\mathbf{1})_i = \frac{e^{\hat{z}_i + c}}{\sum_j e^{\hat{z}_j + c}} = \frac{e^c e^{\hat{z}_i}}{e^c \sum_j e^{\hat{z}_j}} = \text{softmax}(\hat{\mathbf{z}})_i, \quad (5.3)$$

so the intent-dependent factor cancels between numerator and denominator. The implementation already relies on this identity for a different reason: `PathAgent.to_split` subtracts $\max_i z_i$ before exponentiating for numerical stability, which is only sound because the subtraction provably does not change the output.

[TODO: say directly what this proposition is, before the section spends two pages on it. It is the observation that softmax is invariant to a constant added to every logit — an identity so standard that, as the paragraph above notes, the implementation already relies on it for numerical stability. The build-up currently invites the reader to weigh it as a theoretical result, and weighed that way it does not survive: it is a line of algebra, its premise is violated three ways by the very architecture this chapter deploys, and §5.3.5 shows its behavioral prediction is false on trained policies. Weighed as what it is — a design lemma that identifies which conditioning routings carry a guarantee, together with a measurement of exactly how real architectures escape it — it is a genuine and unusual contribution, because almost nobody publishes the boundary of their own principle. Open the subsection with that framing rather than closing with it. The material is already here and in the right order; only the register needs changing, and the honest register is the stronger one.]

Two things make this statement stronger than its discrete counterpart. It is an equality on the *whole action*, not merely on which index wins: the split is numerically identical in every component, so there is no tie-breaking caveat of the kind that qualifies Proposition 4.1. And it is checkable to machine precision rather than behaviorally — hold the path rows fixed, sweep the intent through the global block alone, and the resulting splits must agree to floating-point tolerance. We measure exactly this, and the measurement is where the interesting part begins.

The premise is narrow, and the deployed architecture violates it three times over.

Proposition 5.1 requires the intent to be additive *on the logits the softmax consumes* — which is a materially stronger requirement than “the intent enters through a path-independent block”. Note first what it does *not* require: separability is a statement about the decomposition, not about linearity, and both f and c may be arbitrary nonlinear networks. Table 5.2 reports the behavioral divergence of the split as the intent is swept, for a ladder of scorers running from the proposition’s exact hypothesis up to the policy this chapter actually deploys.

The first row is the proposition, confirmed to 3.8×10^{-9} — float32 noise. The next three rows each remove one of its assumptions, and each is a mechanism worth naming, because together they say something the proposition alone does not.

1. **The squash.** The per-path latent is tanh-squashed before the softmax, as the Gaussian policy’s bounded action mapping requires, and $\tanh(z+c) - \tanh(z)$

Table 5.2: Behavioral divergence of the traffic split as the intent is swept with the decision context frozen, over a 21-point sweep on six paths. Divergence is the mean pairwise total-variation distance between the resulting splits, in $[0,1]$ — the continuous analogue of Eq. (4.4), which counts distinct discrete choices. A scorer obeying Proposition 5.1 must read exactly zero under global-only conditioning. Source: `rl-mpquic/tests/test_intent_conditioning.py`.

Scorer	Intent routed to	Divergence	
Separable, raw logits	global only	3.8×10^{-9}	exact cancellation
Separable, raw logits	per-path rows	9.5×10^{-4}	
Separable, + tanh squash	global only	2.7×10^{-5}	breaker 1
Deployed shared encoder	global only	6.8×10^{-4}	breaker 2
Deployed, rebuilt observation	global only	4.0×10^{-4}	breaker 3
Deployed, rebuilt, fixed scales	global only	6.8×10^{-4}	breaker 3 closed

depends on z . A constant added upstream of a saturating nonlinearity is no longer a constant downstream of it. This is the sharpest of the three and it is architectural: *no* tanh-Gaussian policy satisfies Proposition 5.1’s premise, whatever it does with its conditioning.

2. **The shared encoder is not separable.** The deployed scorer is an MLP over the *concatenation* $[s \parallel f_i]$, so the global block mixes with each path’s own features inside the hidden layers. Its first layer *is* affine on that concatenation, so the intent enters the first pre-activation exactly common-mode — verified to 1.2×10^{-7} — and it is the first ReLU that destroys the invariance. The proposition therefore describes the architecture for exactly one layer.
3. **The observation normalizer carries the intent.** Each path’s smoothed RTT is divided by a latency normalizer that is itself part of the intent bundle (200 ms for interactive, 800 ms for passive), so switching intent rescales a per-path feature column by $4\times$ even when the conditioning is nominally global-only: the measured per-path row span is 0.300 rather than 0. This is precisely the trust-feature leak that qualifies Proposition 4.1 in §4.2.1, reappearing through a normalizer instead of a feature. Fixing the normalizers across intents closes it exactly (row span 0.000, last row).

What survives is both weaker and more useful than what we set out to prove. The exact claim is Proposition 5.1 and it holds for additively separable scorers; it discharges, for that function class, the assertion in §7.4 that the cancellation covers “argmax or softmax” selectors. The design principle it licenses is the portable one and it is unchanged: *any* comparison-based selector must route conditioning into its per-alternative terms if the conditioning is to be guaranteed to do anything. But the corrected version says more than the original would have. A global slot is

not reliably *inert* either — three independent mechanisms leak intent past it — so per-path injection is not merely the better of two options but the only construction with a guarantee in either direction. Section 5.3.5 measures what that distinction is worth on trained policies, and the answer is not the one this section predicts.

5.2 Implementation: Multipath Real-Time Video Testbed

Realizing the design requires a testbed that is faithful enough to expose real multipath and congestion behavior yet fast enough for reinforcement-learning rollouts. We resolve this tension with a single data-plane abstraction served by two interchangeable backends (§5.2.1): a packet-level NS-3 simulator and a fast analytical mock. Section 5.2.2 describes the real-time video and multipath model, §5.2.3 the agent implementation and the VMAF surrogate, and §5.2.4 the known gaps that scope the current results.

5.2.1 Data-Plane Abstraction and Two Backends

Both backends implement one abstract `DataPlane` interface, so the RL code, observations, and rewards are written once and run unchanged against either. This lets us iterate quickly on the mock and validate on the higher-fidelity simulator without divergence.

NS-3 backend. A packet-level simulation in which the connection’s several paths are modeled as independent subflows over a topology of links, each with its own capacity, propagation delay, and active-queue management (FqCoDel). Realistic contention comes from OnOff UDP cross-traffic with exponentially distributed on/off periods, phase-shifted per path so the paths do not degrade in lockstep. Each video frame’s bytes are striped across the subflows according to the transport agent’s split, and delivery is tracked by a byte-watermark scheme – avoiding per-packet tagging – from which per-frame latency, jitter, and deadline-miss loss are measured. Python and NS-3 communicate through an ns3-ai shared-memory bridge.

Mock backend. A pure-Python, per-seed-deterministic model in which each path has a capacity envelope (a smooth sinusoidal component plus bursty cross-traffic and noise) and a standing-queue model that produces bufferbloat when a path is overdriven. It runs orders of magnitude faster than NS-3 and is used for rapid training.

Non-stationary dynamics. Both backends inject four mechanisms that repeatedly unbalance the capacity available on each path: *path churn* (paths appear and disappear, and bytes sent to a dead path are lost), *regime shifts* (abrupt changes

in the best path's capacity), *congestion bursts* (transient per-path capacity collapses), and *correlated failures* (shared-bottleneck groups that degrade together). The NS-3 body implements churn with drop-all receive error models rather than by collapsing the link rate, which would strand a packet mid-serialization, and tears down and reconnects the affected subflow; regime, burst, and correlation scale the bottleneck rate and the cross-traffic rate together. Behavioral parity between the two bodies is checked by a fixed-policy loss-regime guard (`rl-mpquic/scripts/parity_check.py`).

[TODO: report the parity check as a result rather than leaving it as a build step, and state the two places the bodies are deliberately not identical: they draw from independent random streams, so they are behaviorally equivalent rather than frame-identical, and the mock's sender queue is unbounded where NS-3's is not. The second asymmetry is not cosmetic – it is why the mock's failure model is optimistic, and it is the mechanism behind the single baseline inverting between the two bodies (Table 5.6). `rl-mpquic/docs/TUNING_DYNAMICS.md` §4 lists the structural differences.]

5.2.2 Real-Time Video and Multipath Model

The video source mirrors a conferencing encoder: a fixed frame rate (30 fps by default), variable frame sizes (with periodic larger I-frames and per-frame jitter), no playback buffer, and a deadline after which a late frame is discarded as lost. The aggregate target bitrate ranges over 300 – 6000 kbit/s. The canonical topology is deliberately *non-dominant*: no single path can carry full-quality video, and the aggregate usable capacity sits below the quality knee, so the system must genuinely combine paths and trade rate against delivery. For example, a four-path configuration mixes paths of differing capacity and RTT – including a higher-capacity but high-latency “trap” path – under heavy, path-specific cross-traffic, so that neither the widest nor the lowest-latency path is right on its own.

5.2.3 Agents and QoE Surrogate

Agents. Both agents are soft actor-critic learners with twin critics and automatic temperature tuning. The transport agent has two variants: a *flat* version whose actor and critic consume a fixed-length concatenation of the global and per-path features (a fixed path count), and a *scoring* version implementing the permutation-equivariant, mask-aware design of §5.1.5 with a shared per-path actor and a Deep Sets masked-mean-pool critic. The hierarchical training loop runs the two agents at their respective cadences, storing each agent's transitions in its own replay buffer and crediting the application agent over the window of frames its bitrate governed.

VMAF surrogate. The quality term of Eq. (5.1) needs a mapping from an encoding (and network conditions) to perceptual quality. Two are supported: a default

bitrate-only logarithmic curve fit through published quality anchors, and a learned surrogate that interpolates a quality-of-service→VMAF table over bitrate, loss, delay, and jitter (produced by a sibling data-generation project). When the learned surrogate is active, the explicit loss penalty in Eq. (5.1) is dropped to avoid double-counting the loss it already reflects.

5.2.4 Known Gaps

The current testbed has four limitations that scope the results of §5.3 and define concrete follow-up work.

- **Transport realism.** Subflows are modeled with TCP-like congestion control rather than true Multipath QUIC; this preserves congestion, queueing, and loss dynamics but masks some network loss as latency, and forgoes QUIC’s stream multiplexing, connection migration, and unreliable datagrams. Swapping in a real QUIC module is future work.
- **Surrogate coverage.** The learned VMAF surrogate’s loss, delay, and jitter axes are currently under-sampled, so the quality term is effectively bitrate-dominated; a richer surrogate would activate those axes and let quality respond to delivery conditions directly. Every result in §5.3 therefore uses the bitrate-only logarithmic curve.
- **Training body.** All reported policies are trained on the mock backend and evaluated on both. Whether a mock-trained policy transfers is measured, and the answer is that it depends on the training seed in a way mock evaluation cannot predict (§5.3.6); that the system *learns* on the packet-level body is not yet established.
- **Episode independence.** Episodes reset in-band to keep the shared-memory bridge alive, which leaves congestion state warm across resets, so episodes are not independent draws.

[TODO: one further scoping item belongs here once measured: the application agent is horizon-agnostic by construction (§5.1.2), but every episode is 30 s long, so the property is a design claim rather than an evaluated one. A single long-call rollout – several minutes, policy unchanged – would convert it into a measurement, and it costs one evaluation run.]

5.3 Experimental Evaluation: Regimes of Value

This section evaluates the multipath system and answers six questions in turn. Section 5.3.1 fixes the setup, the eight comparison points, and the statistical protocol. Section 5.3.2 asks whether the learned pair beats every baseline, and under which application intents. Section 5.3.3 asks the question the chapter is

named for – *when* does learned scheduling earn its keep – and answers it with a mechanism rather than a scenario label. Section 5.3.4 asks whether the three intents produce genuinely different policies or merely relabel the reward, and §5.3.5 asks the thesis’s own question of this system: whether one policy conditioned on the intent can replace the three that bake it in. Section 5.3.6 asks whether any of it survives on a packet-level transport stack, and §5.3.7 isolates the individual design choices, including the ones that did not pay off. Section 5.3.8 draws together what the section’s four retractions have in common, which is a result in its own right.

[TODO: this chapter has *no figures* — zero against Chapter 4’s six, in the chapter the thesis calls its core. It is the most visible deficiency in the document and the cheapest to fix: nothing below needs a new training run, because --figures was simply never passed on any current evaluation and `rl-mpquic/evaluation/generate_figures.py` renders from any stored `evaluation_results.json`. Six are specified in `rl-mpquic/docs/THESIS_BRIEF.md` §9; in priority order they are the within-episode split trace (the chapter’s mechanism claim, currently asserted in prose — see the gap at §5.3.2), a grouped bar chart replacing Table 5.3, the intent sweep of §5.3.5 as the counterpart to Fig. 4.3, the decision-time distribution that closes the overhead gap in §6.2.1, per-intent training curves for the appendix, and the two-dimensional regime map of §5.3.3. All but the last are evaluation-only against checkpoints already on disk and total well under an hour. Name them `p2eval_*/p2app_*` to match Chapter 4’s convention.]

5.3.1 Setup, Baselines, and Statistical Protocol

Scenarios. The *dynamic* scenario is the headline test: six candidate paths, none individually sufficient, with a latency trap, a shared-bottleneck pair, and all four non-stationary mechanisms of §5.2.1 enabled, so that the identity and quality of the best paths change during a call. The *static* scenario is a three-path stationary topology retained as a contrast case; §5.3.3 explains why it is the less informative of the two.

Metric. The primary metric is mean per-window QoE (Eq. (5.1)) under the intent being evaluated; we also report the underlying achieved bitrate, latency, loss, and VMAF. Because each intent carries its own reward, QoE is comparable *within* an intent and never across intents — all comparisons below are read down a column.

Baselines and ablations. Eight policies are compared. Four are fixed scheduling rules driving a common reactive bitrate controller, so that the comparison isolates *scheduling*: *even* split over live paths, *single* highest-throughput path, *proportional* allocation by recent per-path throughput, and *random* (a fresh Dirichlet draw per frame). Under dynamics all four are mask-aware, operating only over active paths, so they are not handicapped by churn they cannot see. A fifth, *WebRTC*, is the strongest external comparison point: a stateful Google Congestion Control bitrate estimator driving a proportional split, i.e. what a deployed real-time video stack

does today. Two ablations decompose the learned pair itself: *app-only* keeps the learned bitrate controller and replaces the transport agent with an even split, and *path-only* keeps the learned split and replaces the application agent with the same GCC estimator. Together the two ablations answer which half of the hierarchy carries the result.

[TODO: *no MPQUIC-native scheduler appears among the eight comparison points, and this is the first thing a transport reviewer will look for. minRTT is the default scheduler in essentially every Multipath TCP and Multipath QUIC implementation, and BLEST and ECF are the standard research comparisons that a paper on learned multipath scheduling is expected to beat; proportional allocation by recent throughput is not a substitute for any of them, because none of them allocates proportionally. minRTT in particular is nearly free to add: it is a fixed rule over the smoothed RTT already present in the per-path observation, it needs no training, and it slots into the same fixed-rule harness as even and proportional. Adding it converts “beats the rules we wrote” into “beats the rule the field actually deploys”. If it is not added, state its absence explicitly here as a limitation rather than leaving the comparison set to speak for itself, and note in §7.2 that ReLeS and Peekaboo are discussed but not compared against.*]

Statistical protocol. Every comparison is graded over a grid of training checkpoints $\times$ evaluation seeds, five episodes per cell, with all policies seeing identical conditions within a cell. Three quantities accompany every claim: the 95% confidence interval over evaluation seeds, the *training-seed spread* SD_{train} (the standard deviation of per-checkpoint mean QoE), and a per-cell ranking-stability check that reports whether the learned pair is first in *every* cell rather than merely on average. The middle quantity is the one that matters, and it deserves stating as a methodological position rather than a footnote: a margin smaller than SD_{train} is a statement about which training seed was drawn, not about the method. Adopting it forced us to retract or narrow four verdicts we had previously drawn from single-checkpoint evaluations; every such reversal is reported in place below, and §5.3.8 draws out what they have in common.

[TODO: *declare “margin larger than SD_{train} ” as the decision rule it is, and bound it. It gates every verdict in this section, it is not a significance test, and an SD estimated from three samples is itself so uncertain that the rule’s threshold moves substantially between arms — which is why §5.3.5 can report the spread “moving noisily and in both directions” when the seed count doubles. None of that undermines the rule: it is deliberately conservative, it is the right instinct, and adopting it is what caught four bad results. It should be presented that way — a stated, conservative screening heuristic with a known weakness — rather than as a statistic, because a reader who mistakes it for one will correctly object that $n = 3$ supports no such inference. Two additions make it checkable rather than authoritative: give per-seed means for the headline arms, as Table 5.7 already does for the transfer experiment, so a reader can apply their own threshold; and state the minimum detectable effect this design achieves, which is the number §5.3.5’s parity claim silently depends on.*]

5.3.2 Does the Learned Pair Win, and Under Which Intent?

Table 5.3 reports the dynamic scenario under each of the three application intents of Table 5.1.

The learned pair leads every column. Its margin over *WebRTC*, the strongest policy in the table that is genuinely hand-written, is 11.5%, 19.8%, and 21.4% across the three intents; over *path-only*, which is the stronger comparison point but is an ablation of our own system rather than a baseline, it is 10.7%, 18.4%, and 19.6%. Every one of those margins clears the training-seed spread in its column several times over, and the ranking is stable per cell as well as on the mean: the learned pair is first in all nine evaluation cells under *presenter* and *passive* and in eight of nine under *interactive*, where the *path-only* ablation takes one cell. This is the best-evidenced claim in the chapter.

[TODO: the evidence here is the strongest in the thesis and the way it is quoted is the weakest part of it. “11.5%, 19.8%, and 21.4%” are ratios taken on Eq. (5.1), a composite of a quality term and three penalties, clipped to $[-2, 1]$: its zero is wherever the weights happen to put it, so $0.635/0.570$ is a ratio of two points on a scale with no origin, and the same underlying improvement would read as any percentage one liked under an affine re-parameterization of the reward. Lead with the absolute deltas — $+0.065$, $+0.118$, $+0.143$ — against the training-seed spreads of 0.018 – 0.023 that make them meaningful, since “three to six times the spread between training runs” is both true and unattackable where a percentage is neither. Then give the physical metrics, which are what a reader can actually price and which this section already has two paragraphs below: VMAF, delivered latency, achieved bitrate, loss. Keep the percentages only if Eq. (5.1)’s zero point is defended somewhere. The same correction applies to the chapter summary in §5.3.9; the abstract and §1.2 have already been rewritten to lead with the absolute deltas.] [TODO: a second correction has been applied to the paragraph above and should be checked wherever the phrase recurs: these margins were previously described as “an order of magnitude above the training-seed spread”. They are not. Against the spreads in Table 5.3 the ratios are $0.065/0.019 = 3.4\times$, $0.118/0.018 = 6.6\times$ and $0.143/0.023 = 6.2\times$ — comfortably significant under this chapter’s own decision rule, but three to seven times rather than ten. The claim never needed the exaggeration and is stronger without it, since any reader who checks the arithmetic against the table will find it. Sweep the chapter for other uses of “order of magnitude” and verify each against its table. Note the phrase is used correctly in §5.3.8, of the ratio between evaluation-seed intervals and training-seed spread, which genuinely is tenfold.]

The distinction between the two margins is worth keeping straight, because the literature’s question and the design’s question are different. Against what a deployed real-time video stack does today — a Google Congestion Control bitrate estimator over a proportional split — the learned pair is worth 11–21%. Against a system that keeps our learned scheduler and swaps only the bitrate controller, it is worth 11–20%, which is a statement about the application half of the hierarchy rather than about the system as a whole. Section 5.3.7 decomposes it properly.

Table 5.3: Mean QoE (Eq. (5.1)) on the dynamic six-path scenario, by policy and application intent, over three training seeds $\times$ three evaluation seeds $\times$ five episodes. SD_{train} in brackets, omitted for the deterministic baselines that carry no learned component. Each column is evaluated under its own reward and deadline, so columns are not comparable to one another. Source: `rl-mpquic/runs/ab-prof2-ev*-tr*/ab_summary.json`.

Policy	interactive	presenter	passive
Learned pair	0.635 [0.019]	0.719 [0.018]	0.810 [0.023]
Path-only (GCC rate), <i>ablation</i>	0.574 [0.000]	0.607 [0.009]	0.677 [0.006]
WebRTC (GCC + proportional)	0.570	0.601	0.667
Single best-active path	0.456	0.496	0.512
App-only (learned rate, even split), <i>ablation</i>	0.220 [0.044]	0.441 [0.033]	0.682 [0.018]
Proportional	0.238	0.275	0.332
Even split	0.126	0.158	0.223
Random split	0.118	0.154	0.217

The policies also *behave* differently under different intents, which is what the intent mechanism is for. Delivered latency rises from 49.0 ms (interactive) to 57.6 ms (presenter) to 93.1 ms (passive), and the achieved bitrate rises with it from 2626 to 3304 to 3400 kbit/s, lifting VMAF from 79.5 to 85.3 to 86.1. As the intent relaxes its delay budget, the policy spends the slack on quality. Section 5.3.5 shows the conditioned policy reproducing that same ordering *continuously*, as a function of the intent it is handed, rather than in three discrete steps.

[TODO: the three physical-metric triples in the paragraph above come from the earlier single-training-seed run (`runs/ab-prof-*`), not from the three-seed table above it, and the *interactive* checkpoint there had twice the training budget of the other two. The QoE numbers have been re-derived at three seeds; these have not. The ordering is qualitative and is independently corroborated by the continuous sweep of §5.3.5, so the claim is safe — but the specific values should be re-read from `runs/ab-prof2-ev*-tr*/ab_summary.json`, which already records them, or the sentence should be rewritten to quote the sweep instead and drop the triples.]

Figure 5.1 shows the behavior behind these numbers within a single episode. The reallocation claim is visible directly: each interval in which a path’s loss saturates at 1.0 — the signature of churn taking it down — is an interval in which its allocated share is zero, and the share returns when the path does.

[TODO: two qualifications on the figure above, and one figure still missing. (i) It is one episode of one evaluation seed of one training seed, chosen as representative and not by any stated criterion — say so in the caption, since every other number in this chapter is graded over a 3×3 grid and a reader is entitled to know this one is not. (ii) The claim the figure supports is qualitative co-occurrence; the quantitative version — mean allocated share to a path in the windows where it is down, which should be indistinguishable from

zero, against its share when live — is computable from the same traces and is what would make this a measurement rather than an illustration. (iii) Still missing is the grouped bar or slope chart of Table 5.3, which reads poorly as an eight-row table; build it from the three-seed `ab_summary.json` aggregates rather than from `figure1_qoe` of a single cell, which would understate the evidence behind the table it replaces.]

5.3.3 When Does Splitting Help? The Governing Quantity

The chapter’s title promises regimes of value, and the honest form of that claim is not “learned scheduling helps when the network is dynamic.” It is sharper, and it is mechanical. The quantity that governs whether a scheduler has anything to do is the relationship between the *share an even split places on each path* and the *residual capacity of the weakest path*. Where every path can absorb its even share, any allocation works and the learned agent has nothing to earn. Where some path cannot, the split decides whether the frame arrives. Volatility matters because churn, regime shifts, and bursts are mechanisms that *destroy* that headroom homogeneity transiently — but they are not the only ones, and naming them as the axis mistakes a cause for the variable.

Two observations support the reframing, and both cut against the simpler story.

A static network is not a safe network. On the static three-path scenario, the app-only ablation does not merely lose — it collapses to -0.396 QoE at 696 ms latency and 52% loss, because the topology is *asymmetric* (8/4/2 Mbit/s) and an even split overdrives the weakest path continuously. Staticness bought it nothing. Conversely, a static topology of four *near-equal* paths puts app-only within 2% of the full system. It is the homogeneity of headroom, not the stationarity of the network, that decides.

The static scenario cannot discriminate at all. On that same three-path scenario the learned pair scores 0.700 against 0.691 for the single-best-path baseline — a margin of 0.009 against an SD_{train} of 0.017. The learned pair is first in six of nine cells and loses all three cells of one training seed. The correct reading is not that scheduling is unnecessary when paths are stable; it is that this scenario *cannot tell*, because the margin is smaller than the spread between training runs of the same method. A practitioner benchmarking a multipath scheduler only on a stable topology would conclude it does not work, and would be measuring their own seed draw.

Intent strictness is the second axis. The app-only ablation recovers 35%, 61%, and 84% of the full system’s QoE under interactive, presenter, and passive respectively (Table 5.3). Holding the network fixed and varying only what the application asks for, the scheduler’s contribution more than doubles as the delay budget tightens. This is a mechanism-level result rather than an average: the

stricter the intent, the less slack there is to absorb a bad allocation, so the more the allocation matters. It is also the point at which this chapter rejoins Chapter 4 — the value of the control machinery is itself a function of the intent. The trend is one of the few in this chapter that reproduced without qualification when the experiment was rerun at three training seeds: the single-seed version of the same measurement read 39/55/81%.

[TODO: turn the two paragraphs above into a measured two-dimensional map, which would be this chapter’s counterpart to Fig. 4.6 and its strongest single figure. Rows: headroom heterogeneity, swept by scaling the per-path capacity spread and the dynamics hazard rates (`rl-mpquic/configs/dynamic.yaml`, `dynamics: block`, and the held-out `dynamic_ood_{harsh,mild}.yaml`). Columns: the three intents of Table 5.1. Cell value: the app-only recovery ratio, which is the cleanest scalar summary of “how much did the scheduler contribute here”. Nine reference points already exist in `rl-mpquic/docs/TUNING_DYNAMICS.md` §4 and can seed the grid; the full sweep is evaluation-only against checkpoints already on disk. Specified as work item W4 in `rl-mpquic/docs/THESIS_BRIEF.md`.]

5.3.4 Do the Intents Produce Different Policies?

Table 5.3 shows that policies trained under different intents behave differently, but that is also consistent with the three intents simply being easier or harder to satisfy. The discriminating test is to evaluate every intent-trained policy under every intent’s reward and read down each column: if the intents produce genuinely specialized policies, each column should be won by its own diagonal entry. Table 5.4 reports it. This is the multipath counterpart of the intent-alignment matrix of Fig. 4.1, and unlike that one it does not come out clean — in a way that is more interesting than a clean failure would have been.

Read naively, the table says the interactive-trained policy is the best policy for every intent, winning all three columns and beating two diagonals on their own terms. Read against the training-seed spread, it says something considerably more deflationary. The interactive diagonal wins its column by +0.025 against spreads of 0.019 and 0.012, about 1.1σ . The presenter diagonal loses its column by 0.016, about 0.6σ . The passive diagonal loses by 0.009, about 0.3σ . *Every margin in the matrix is inside the spread between training runs of the same method.* The honest conclusion is not that one specialist dominates but that, at three training seeds and this budget, **which intent a policy was trained for does not measurably determine which intent it is best at.**

This overturns a cleaner-looking result we had previously drawn from the same experiment run at one training seed per cell, where two of three diagonals held and the third failed conspicuously — the passive policy being dominated by the presenter policy on passive’s own terms, which invited an appealing mechanistic story about a weight ($b = 0.1$) too small to generate the gradient that would train against it. The passive diagonal does still lose. But so does

Table 5.4: Intent-transfer matrix: mean QoE of each intent-trained policy (rows) evaluated under each intent’s own reward and deadline (columns), on the dynamic scenario, over three training seeds $\times$ three evaluation seeds. SD_{train} in brackets. Read *down* a column, which fixes the evaluation intent. The diagonal is set in bold and the column winner underlined; they coincide only for interactive, and no column’s margin clears its own training-seed spread. Source: `rl-mpquic/runs/ab-prof2-ev*-tr*/ab_summary.json`.

Trained under	Evaluated under		
	interactive	presenter	passive
interactive	<u>0.635</u> [0.019]	<u>0.735</u> [0.021]	<u>0.818</u> [0.017]
presenter	0.611 [0.012]	0.719 [0.018]	0.803 [0.023]
passive	0.552 [0.076]	0.704 [0.040]	0.810 [0.023]

presenter’s, and neither loss clears noise, so the story is not supported: what looked like a specific pathology of relaxed objectives is one of three statistically indistinguishable outcomes. Section 5.3.8 returns to this, because it is the third time in this chapter that a single-seed result did not survive being run three times.

The consequence for the design is the one worth carrying forward, and it cuts in an unexpected direction. The per-intent model zoo that Chapter 4 argues against is, on this evidence, not even buying what it charges for: three separately trained specialists at three times the training cost are not measurably specialized. That *weakens* the motivation for specialists and correspondingly *strengthens* the case for a single conditioned policy, which now has less to lose than the single-seed reading suggested. It also fixes the bar. A conditioned policy must clear not the diagonal but the best entry in each column — 0.635, 0.735, 0.818 — since those are the best policies actually available per intent, whichever intent produced them.

5.3.5 One Conditioned Policy, or Three Specialists?

This is the question the thesis is about, asked in the multipath setting: can one policy that takes the intent as an input replace the family of policies that bake it in? We train two conditioned arms, both over intents randomized per episode, differing only in *where* the intent is routed — §5.1.6’s question made experimental. The *per-path* arm supplies the intent to the global block and to every per-path feature row; the *global-only* arm supplies it to the global block alone, and is the control that Proposition 5.1 predicts should be inert. Each arm is one policy graded at all three intents against the Table 5.4 bar.

One policy does the work of three, at no measurable cost. The global-only arm matches the best specialist under all three intents, with no margin distinguishable

Table 5.5: A single intent-conditioned policy against the best specialist per intent. Bar = column maxima of Table 5.4. Both conditioned arms are single policies evaluated at all three intents; specialists are three policies. Six training seeds per conditioned arm, SD_{train} in brackets; σ is the pooled training-seed spread of the two arms being differenced. Source: `rl-mpquic/runs/ab-intent6-{pp,gl}-*/ab_summary.json`.

Evaluated under	Best specialist	Conditioned, per-path		Conditioned, global-only	
	QoE	QoE	Δ	QoE	Δ
interactive	0.635 [0.019]	0.587 [0.028]	−0.048 (−1.4 σ)	0.639 [0.041]	+0.004 (+0.1 σ)
presenter	0.735 [0.021]	0.707 [0.030]	−0.028 (−0.8 σ)	0.693 [0.057]	−0.042 (−0.7 σ)
passive	0.818 [0.017]	0.816 [0.035]	−0.002 (−0.1 σ)	0.823 [0.059]	+0.005 (+0.1 σ)

from zero and two of the three nominally positive. The per-path arm matches under passive and presenter and carries a persistent shortfall under interactive. This is the multipath analogue of §4.4.3’s result and it is the chapter’s answer to the thesis’s central question — but it must be stated as *parity*, not superiority. A failure to detect a difference at this spread is not a demonstration of equivalence, and the claim the chapter makes is the weaker, defensible one: *a single conditioned policy replaces three specialists without measurable loss*, at a third of the training cost and with one artifact to deploy rather than three.

[TODO: this paragraph does the right thing — it states the result as parity rather than superiority — and then omits the one number that makes parity a claim instead of an absence. Report the *minimum detectable effect*: at the pooled training-seed spreads in Table 5.5 (0.028–0.059) and six seeds per arm, what size of QoE difference would this design have detected? It is a two-line computation from numbers already in the table, and it converts “we found no difference” into “we would have found a difference larger than X , and there was none”, which is the only form of a null a reviewer can accept. Quote X against the effects the chapter cares about: the specialist margins it is being compared to are 0.002–0.048, so if X lands above them the honest reading is that this experiment could not have resolved the question either way — which is itself worth stating, and is consistent with Table 5.4 having failed to separate the specialists at all. Two sentences, and they are the difference between a defensible null and a claim that reads as wishful.]

Two caveats belong with it rather than after it. First, the bar itself is noise-limited: the specialists were not measurably specialized (Table 5.4), so “matches the specialists” is a weaker statement here than it would be against a strongly separated set. Second, doubling from three to six training seeds — the numbers above are the six-seed grading — did not shrink the spread. The global-only arm’s SD_{train} moved from 0.062/0.087/0.055 to 0.041/0.057/0.059, that is, noisily and in both directions. More seeds bought a better estimate of the mean and did not make the learner more reliable, which is a property of this learner on this scenario and not a defect of the experiment.

The control arm did not behave like a control. Proposition 5.1 predicts that the global-only policy consumes the intent and emits an identical split. It does not. Sweeping the intent with the decision context frozen, over 200 on-distribution contexts per training seed, the global-only arm’s behavioral divergence is 0.146 [0.015] against the per-path arm’s 0.157 [0.048] — a difference well inside the per-path arm’s own spread, where the prediction was that the first number would be zero. Only 29% of the global-only arm’s logit perturbation is the common mode the softmax discards; the predicted figure was essentially all of it.

This is the empirical completion of §5.1.6’s second breaker, and it is the more informative half of the chapter’s theory. At random initialization the leak through the shared encoder’s nonlinearity is of order 10^{-4} and looks negligible. *Training turns it into the primary channel:* a network free to route the intent around the cancellation learns to do so, and the softmax’s shift-invariance stops constraining it at all. The proposition is not wrong — it is exact, for the function class it names — but the function class is narrow enough that a deep encoder escapes it without being designed to.

The practical consequence is that per-path conditioning is *not* empirically justified in this system. It costs 30 additional input dimensions, scored no better on divergence, scored worse on QoE at two of three intents, and, as the next paragraph shows, interpolates less smoothly.

[TODO: the chapter reports per-path conditioning losing on every axis it measures and never says why, which leaves its own recommended design mechanism unexplained at the moment it is abandoned. There is a specific and testable suspect already documented: breaker 3 of §5.1.6, the observation normalizer that carries the intent into the per-path rows (Appendix A.3, “Variant A”). Under per-path conditioning the intent reaches each path row twice — once as a broadcast feature and once as a $4\times$ rescaling of that row’s sRTT column — and the network must learn to reconcile two correlated channels rather than one, which is the mechanism one would expect to cost exactly what is observed: no gain in divergence, a loss in QoE, and a non-monotone latency curve. `intent.obs_norm`: fixed closes that channel exactly (row span 0.000, last row of Table 5.2), is implemented and test-guarded, and has never been trained — it is listed as unmeasured in `r1-mpquic/docs/RESULTS.md` §10.5. Three seeds is ≈ 4.5 h and settles it either way: if the shortfall closes, the chapter gains a mechanism for its own negative result and the design principle is vindicated rather than merely defended on guarantees; if it does not, the chapter can say the leak is not the explanation and attribute the cost to the 30 extra input dimensions, which is a cleaner statement than the current silence. Run it, and note it must not run concurrently with an NS-3 job (shared-memory segment collision, see the gap at §5.3.6).] What survives is an argument about guarantees rather than performance: per-path injection is the only routing that is guaranteed to work, and global routing happened to work here through a mechanism nobody specified and nobody can rely on. That is a design principle with an honest boundary on it, which is worth more than the unqualified version the chapter set out to prove.

The conditioned policy interpolates across a continuum of intents. Sweeping the intent linearly from interactive to passive in eleven steps, both arms trade delay for quality monotonically and smoothly: achieved bitrate rises from ≈ 2500 to 3440 kbit/s (per-path) and ≈ 2890 to 3830 kbit/s (global-only), with no single step of the sweep carrying the trend. Because the intent bundle contains the deadline, the raw metrics move even for a policy that ignores the intent entirely, so the de-confounded statistic is the ratio of the learned policy’s span to a non-learning control’s span over the same rollouts: the conditioned policies adapt by $17\text{--}24\times$ the control on bitrate and VMAF. Spearman ρ against λ is 0.80 and 0.71 for the global-only arm on bitrate and VMAF, and 0.96 on latency against the per-path arm’s 0.41 — the per-path arm’s latency curve is non-monotone, climbing then falling back, while the global-only arm’s rises cleanly. Per-path conditioning has now scored equal-or-worse on all three axes measured.

[TODO: state the scope of the interpolation claim explicitly in the text above rather than only here, and choose deliberately, because the natural phrasing overclaims. Training sampled the intent space continuously (`configs/dynamic_intent.yaml`, `continuous: true`, `anchor_frac: 0.5`), so interior λ are in-distribution. What is licensed is “the policy generalizes across a continuum of intents rather than memorizing three operating points” — a real claim, and the counterpart of §4.4.4’s. What is not licensed is a zero-shot claim about the interior, which is what Chapter 4 demonstrates and what a reader will assume is meant here unless told otherwise. Only $\lambda = -0.2$ and $\lambda = 1.2$ are genuinely unseen, and they fail asymmetrically: at $\lambda = 1.2$ (deadline 924 ms) both arms hold QoE at 0.82 , while at $\lambda = -0.2$ (deadline 56 ms) both collapse to ≈ 0.16 . That asymmetry is worth reporting — the policies extrapolate toward looser intents and not toward stricter ones, which is the expected direction, since $\lambda < 0$ asks for a deadline below anything the network can deliver rather than merely an unseen one. Training a third arm at `anchor_frac: 1.0` (anchors only, no interior sampling) is what would license the strong zero-shot claim, and it is one training run.]

[TODO: the operational argument for a conditioned policy is one this chapter never makes and could make cheaply: changing intent mid-call. Everything measured here holds the intent fixed for an entire episode, which is precisely the regime in which three specialists and one conditioned policy are interchangeable — a deployment can load the right specialist at call setup and pay nothing. The conditioned policy’s real advantage appears only when the intent moves while the connection is up: a call that starts as screen-share and becomes conversational, or an application that tightens its deadline under user interaction. Swapping model weights mid-call is a bad answer; taking the intent as an input is the good one, and no experiment in this thesis exercises it. One evaluation run does: step or ramp λ at mid-episode and show the split, target bitrate, and delivered latency tracking the change within some settling time, against a specialist that cannot follow it at all. It needs no training — the six `ab-intent6-*` checkpoints already exist and `evaluate.py` already accepts an intent override — and it produces both the chapter’s most persuasive figure and the answer to “why not just train three models”. It is named as the wanted bonus in `rl-mpquic/docs/THESIS_BRIEF.md` §3.7.]

5.3.6 Does It Survive a Packet-Level Transport Stack?

Every result so far is measured on the analytical mock. Table 5.6 evaluates six independently trained mock policies, without retraining or fine-tuning, on the NS-3 body — real TCP subflows, FqCoDel queues, and packet-level cross traffic.

The ranking does not transfer. On the mock the learned pair leads every column by an order of magnitude more than its own spread; here it is second on the mean, tops only 8 of 18 cells, and sits 0.032 behind the path-only ablation against a training-seed spread of 0.081. The correct statement is not that the heuristic wins — that margin is well inside noise — but that *on a packet-level transport stack the learned pair is no longer distinguishable from the best available scheduler that does not learn its bitrate*. Two of the three things this section previously claimed do survive, and they survive in stronger form. Absolute QoE is not portable: 0.348 here against 0.617 on the mock. And *single inverts* — mid-table on the mock, worst of eight here at -0.131 with 41% loss — because one real TCP subflow cannot absorb the offered rate whereas the mock’s unbounded analytical queue lets it. That inversion is direct evidence that the mock’s failure model is optimistic.

Transfer is a property of the training seed, not of the method. The variance in Table 5.6 is almost entirely between training runs, and it is not a spread but a split.

Three of six checkpoints keep their first-place ranking on NS-3 and three fall behind a GCC-driven scheduler, in two clean clusters with nothing in between. The same six policies are *indistinguishable* on the mock: a spread of 0.008, smaller than a single arm’s evaluation-seed confidence interval. The tenfold variance inflation appears only on the realistic body, and — this is the part with teeth — it is not a rescaling of the same ordering:

Spearman ρ between mock rank and NS-3 rank, over six independently trained checkpoints, is -0.086 . The correlation is not weak; it is absent. The *best* mock checkpoint is the *worst* on NS-3.

This is the strongest form of the sim-to-real caveat this thesis can state, and it is a methodological result rather than a caveat on a number. Any procedure that trains N policies on a cheap surrogate, selects the best by surrogate score, and reports it on the realistic body is — on this evidence — selecting at random, and in the one case we tested would have selected the worst of six. Our own earlier, single-cell version of this section is exactly that failure mode caught in the act: the cell it reported was the best evaluation seed of a checkpoint that we can now see is the worst of the six. Checkpoint selection must happen on the backend the claim is about, or the honest alternative is to report the distribution over seeds, as Tables 5.6 and 5.7 do.

Table 5.6: Zero-shot transfer to the packet-level NS-3 backend: six mock-trained checkpoints evaluated on the NS-3 body without retraining, dynamic scenario, TCP subflows, 6×3 cells. SD_{train} is the spread across training seeds; it is the column that decides the reading. Note the inversion of single, mid-table on the mock and last here. Source: `rl-mpquic/runs/ab-ns3-6seed/ab_summary.json`.

Policy	QoE	SD_{train}	VMAF	Latency	Loss	Bitrate
Path-only (GCC rate)	0.380	0.024	71.3	73.0 ms	0.107	2079
Learned pair	0.348	0.081	77.0	86.4 ms	0.155	2546
WebRTC	0.298	0.000	57.2	57.9 ms	0.089	1404
App-only	0.279	0.033	76.1	94.0 ms	0.190	2401
Proportional	0.146	0.000	46.1	62.2 ms	0.124	1276
Even split	0.132	0.000	25.0	40.9 ms	0.003	301
Random split	0.127	0.000	25.2	40.2 ms	0.002	303
Single best path	−0.131	0.000	67.3	145.1 ms	0.407	2848

Table 5.7: Per-training-seed mean QoE on both backends, over three evaluation seeds each. The NS-3 column is bimodal with the path-only ablation (0.380) sitting in the gap, and that gap is what decides each seed’s verdict. The mock does not resolve the difference at all. Source: `rl-mpquic/runs/ab-ns3-6seed/`, `runs/ab-mock-6seed/`.

Checkpoint	Mock QoE	NS-3 QoE	Transfers?
dyn-s1	0.628 (best)	0.262 (worst)	no
dyn-s2	0.603 (worst)	0.289	no
dyn-s5	0.615	0.273	no
dyn-s4	0.621	0.402	yes
dyn-s6	0.617	0.426	yes
dyn-s3	0.620	0.434	yes
spread	$SD = 0.008$	$SD = 0.081$	3 of 6

The mechanism: the transferring policies are the ones that keep the network out of queue. The split is explicable, and explaining it identifies what the mock is missing. Re-reading the per-frame traces already stored in the eighteen NS-3 cells — no new simulation — the statistic that separates the two clusters is *delay-weighted allocation*, $\sum_i w_i \cdot \text{sRTT}_i$: where the bytes actually go, priced by what that path costs at the time. It rank-separates the clusters perfectly, the three transferring seeds taking the three lowest values, which for a 3–3 split is the strongest separation six points admit (exact one-tailed permutation $p = 0.050$), and it correlates with NS-3 QoE at Pearson $r = -0.918$ — roughly 84% of the between-seed variance the aggregate table cannot explain. It factors into two terms that point the same

way: the non-transferring policies both run every path hotter (mean path sRTT 75.8 ms against 70.5 ms, self-inflicted, since they drive the higher bitrate into real transport) and aim worse within that (weighting paths slower than the current average, ratio 1.058 against 0.997).

The natural competing hypothesis — that the clusters differ in loss tolerance — is refuted by the same traces: loss-weighted allocation is 0.0137 against 0.0162, a gap of 0.28 standard deviations and in the wrong direction for two of the six seeds. Split entropy, active-path count, and split volatility are flat too. Of six behavioral statistics, only the two delay-weighted ones separate.

And this is precisely why the mock cannot predict NS-3. Running the identical analysis on the mock cells, mean path sRTT is 48 ms for all six policies, spanning 1.9 ms, where on NS-3 the same six span 9.7 ms. The mock's analytical queue does not charge these policies differently for the queueing they cause, so the one variable that decides the NS-3 outcome is invisible in mock evaluation, and a mock ranking is free to order the seeds however noise happens to. The $\rho = -0.086$ is not a mysterious variance problem: it is the queue model. The fix is a higher-fidelity queue in the surrogate, not more surrogate seeds.

[TODO: the mechanism result rests on $n = 3$ per cluster, and $p = 0.050$ is the floor attainable at a 3–3 split, reached only because the rank separation is perfect — so a single added seed that breaks the ordering drops it to $p \approx 0.10$. Two more training seeds graded on NS-3 (dyn- $s\{7, 8\}$, specified as D7b in `rl-mpquic/docs/RESULTS.md` §9) is what makes this load-bearing rather than suggestive, at ≈ 9 h training plus 15 min of NS-3 evaluation. Two further limits should be stated in the text once it is: sRTT is partly endogenous, since loading a path lengthens its own queue, which is why the statistic is decomposed rather than quoted alone — and the least endogenous of the three terms, the ratio, is also the weakest ($p = 0.100$); and this is a behavioral correlation across six policies, not an intervention, so it does not show that forcing a lower delay-weighted allocation would make a non-transferring seed transfer. The intervention that would — retrain with a delay-aware split prior, or raise b and re-run — is not done.]

[TODO: whether the system learns on the packet-level body, as opposed to transferring to it, is still open and is now the chapter's most valuable remaining experiment, because Table 5.7 makes the mock's status as a training surrogate the live question rather than a footnote. NS-3-native training correct the cost figure before quoting it: the ≈ 4 h per 100-episode seed recorded in `rl-mpquic/docs/RESULTS.md` §9 is extrapolated from a 5-episode probe (441 s against the mock's 436 s) and does not survive contact with a full run, because per-episode cost grows with replay-buffer occupancy. Measured on the two seeds actually executed, an NS-3-native seed takes ≈ 25 h: seed 2 logged 86 episodes in 21.1 h, or 14.7 min per episode. Runs are also strictly sequential, because both `ns3ai_utils.Experiment` and `ns3-ai-msg-interface.h` default the shared-memory segment name to "My Seg" and nothing overrides it, so two concurrent runs deadlock. A 3-seed NS-3 arm is therefore ≈ 75 h wall-clock rather than the 12 h previously stated, against ≈ 4.5 h for the mock arm run three-up — a factor of roughly 17. That ratio, not a fidelity judgment, is the honest reason every policy in this chapter is mock-trained,

and it belongs in §5.2.4 as such. This is D5 in `r1-mpquic/docs/RESULTS.md` §9 and was executing at the time of writing.]

5.3.7 Ablations

The permutation-equivariant path head is what makes the system work. Replacing the scoring transport agent of §5.1.5 with the legacy flat MLP over a fixed-length concatenation — the two configurations are field-identical apart from that one architectural key — costs 0.231 ± 0.029 QoE, positive in all nine evaluation cells, against training-seed spreads of 0.013 and 0.037. This is the largest effect measured anywhere in this chapter, an order of magnitude above the noise floor, and its importance is not the size of the gap but where the gap lands: at 0.386 the flat agent is beaten by WebRTC and by the single-path baseline in *every one* of the nine cells, so **with the flat head, the headline claim of this chapter is false** — the learned pair ranks first in 0/9 cells rather than 9/9. The dynamic-input architecture is therefore not a convenience for handling a variable path count; it is load-bearing. Isolating the scheduler makes the mechanism cleaner still: the path-only ablation, which is exactly this head under a fixed GCC bitrate driver, moves by 0.401 ± 0.039 .

Two limits bound it honestly. This is the architecture’s home turf — the dynamic scenario has churn, so the live path set changes within an episode and the flat MLP must encode a varying-cardinality set into fixed slots, which means the result establishes necessity *under churn* and says nothing about the static scenarios. And the flat arm’s own training-seed spread is roughly three times the scoring arm’s, so part of the gap is that the flat learner is less consistent rather than only worse on average; at three seeds those are not separable.

The gap is also not bought with parameters, which is the same inversion §4.4.2 reports for the discrete selector: there the fixed-action head carries 85.3 k parameters against the scoring agents’ 36.2–37.5 k, less than half the size for more than double the goodput. A shared per-path encoder is evaluated once per candidate and reused across all of them, where a fixed head must spend output capacity on every slot it might ever need. In both settings the dynamic-input design is cheaper to store as well as better to run, and its cost is paid instead in inference time, which grows linearly in the path count and which Fig. 5.2 shows to be far inside the frame budget. **[TODO: confirm the multipath parameter counts directly and quote them here rather than borrowing Chapter 4’s, which are for a different network in a different framework. The two configurations are field-identical apart from the architectural key, so the comparison is clean; a single load of each checkpoint reports it.]**

Which half of the hierarchy carries the result? Both, and in a way that depends on the intent — on the mock. Removing the transport agent (app-only) costs 65% of the QoE under interactive and 16% under passive; removing the application agent (path-only) costs 10% and 16% respectively. Neither half is sufficient, and the difference is what the joint premise of §5.1.1 buys.

On the packet-level body the decomposition does not hold, and this is the sharpest instance of the sim-to-sim gap of §5.3.6. There the app-only ablation recovers 80% of the full system rather than 35%, and the path-only ablation does not cost anything at all — it *outscores* the full pair, 0.380 against 0.348, within noise. The two-sided ablation is therefore a claim about the analytical backend and must be stated as one: what the packet-level body supports is that the learned scheduler is worth having, not that the learned bitrate controller on top of it is.

Domain randomization does not help here, on either axis. Randomizing the dynamics hazard rates per training episode costs 0.027 QoE on the nominal environment and loses in all nine evaluation cells, against training-seed spreads of 0.024 and 0.013. That measures randomization’s *cost*; its intended *benefit* is generalization to networks outside the training distribution, which requires grading both arms on held-out environments the randomization was supposed to prepare for. Doing so costs a further 0.033 QoE, again in 9 of 9 cells. The treatment is negative on the axis it was adopted for as well as on the one it was previously measured on. The interpretation is mechanical: the observation carries no signal about *which* dynamics draw is in force, so widening the training distribution adds variance the policy cannot condition on. Consistent with this, the one place randomization clearly helps is the app-only fallback (0.299 against 0.155), which is what one expects from a treatment that flattens a policy rather than sharpening it. One held-out environment is uninformative rather than supportive: on the harsh draw both arms score negative, the ranking is unstable, and the randomized arm’s training-seed spread is 0.445 — larger than any effect in this chapter.

An attention-pool critic does not help either, and destabilizes training. Replacing the masked mean-pool of §5.1.5 with a Set-Transformer attention pool — targeted at the shared-bottleneck structure the mean-pool is blind to — is not significant in the mean (0.083 in the mean pool’s favor, inside its confidence interval, won in four of nine cells). The decisive number is the spread: the attention arm’s is 0.151, twelve times the mean-pool’s 0.013 and larger than the entire margin the system claims over any baseline, because one of its three training seeds collapsed outright. The mean pool remains the default. The reason to report this rather than drop it is that evaluated on its healthy seed alone the attention pool looked like a small win — a concrete instance of the methodological point of §5.3.1, caught in our own data.

[TODO: two pieces of architectural evidence that Chapter 4 carries and this chapter still does not, both cheap now that the flat-versus-scoring arm above exists. First, permutation invariance of the split under reordering of the path set — asserted in §5.1.5 as a property of the architecture and verified in `rl-mpquic/tests/test_scoring_agent.py`, but never reported as a measurement, where Chapter 4 reports a 32256-trial invariance test. Second, QoE against live path count, the continuous counterpart of Fig. 4.4, which is

what would show the architecture’s independence of path count empirically rather than by construction. Both are evaluation-only against checkpoints already on disk.]

The per-frame inference cost fits the frame budget with three orders of magnitude to spare. The transport agent is the design’s one hard real-time component: it runs once per frame, so its inference must complete well inside 33 ms at 30 fps. Figure 5.2 measures it. The learned split decision costs 0.507 ms on average on a 16-core CPU host with no GPU, tightly distributed across frames — about 1.5% of the frame budget, and roughly 23× the cost of the cheapest fixed rule (even split, at 0.0123 ms) which is nonetheless an irrelevant difference at this scale. The application agent’s bitrate decision is cheaper still and runs 30 times less often. Because the shared per-path encoder is evaluated once per candidate, the cost grows linearly in the path count, so the margin here corresponds to roughly 200 paths before the frame budget binds.

[TODO: three fixes on the figure and the paragraph above. (i) The rendered panel (b) y-axis label collides with panel (a)’s tick labels — regenerate with a wider figure or a shorter label before submission. (ii) The measurement is from one evaluation cell; decision time is a property of the machine rather than of the training seed, so this is defensible, but state the host specification in the caption as §6.2.1 does for the single-path selector, and give the 99th percentile alongside the mean, since a real-time claim is about the tail and the box plot already carries it. (iii) The linear-scaling extrapolation to ≈ 200 paths above is arithmetic, not a measurement — either measure the cost at a few path counts, which is cheap and would also supply the missing counterpart to Fig. 4.4, or soften the sentence to a scaling statement without a number. Finally, propagate the headline figure into §6.2.1, which still carries an open gap asking for exactly this number.]

DroQ and prioritized replay buy nothing at convergence, and little before it. The critic LayerNorm with a raised update-to-data ratio, and prioritized experience replay, are enabled in every configuration reported in this chapter. Turning all three off costs -0.005 ± 0.010 QoE — not significant, positive in two of nine cells — and both arms rank the learned pair first in every cell, so no result in this chapter depends on them. Since these are *sample-efficiency* mechanisms and that comparison is at convergence, the stored training curves test the claim they actually make: the flags are ahead by +0.138 mean training reward over the first ten episodes and reach an early threshold one to three episodes sooner, but the advantage is gone by episode 25. A real but transient head start, not a sample-efficiency win — and a weak version of the claim at that, since the shipped update-to-data ratio is 2 rather than the 10–20 the DroQ recipe is about, so what is refuted is DroQ as configured here and not DroQ as published.

5.3.8 What the Retractions Have in Common

Four results in this chapter looked clean when read from a single training run and did not survive being run three times. The static scenario's reward comparison was stable and became unstable (§5.3.3). Intent specialization held in two of three columns and became detectable in none (§5.3.4). The learned pair ranked first on the packet-level backend and became one of two clusters (§5.3.6). Global-only conditioning was predicted to be inert and turned out to be a working policy (§5.3.5). We report the pattern rather than four separate corrections, because the pattern is the more useful finding and it is the one result in this chapter that generalizes beyond multipath scheduling.

Two of the four were not merely optimistic — they were the most favorable cell available, without anyone choosing it. Neither was cherry-picked; both are what one gets by running a configuration, looking at the number, and writing it down.

The mechanism is a confusion between two spreads that measure different things and that differ by an order of magnitude on this problem. Confidence intervals over *evaluation* seeds are routinely ± 0.001 – 0.01 here; the spread over *training* seeds ranges 0.012 – 0.151 across the arms in this chapter. A margin reported against the first and not the second looks significant at roughly ten times its true resolution, and every claim above that broke, broke inside that gap. The recommendation follows directly and is the methodological position of §5.3.1 stated as a result: a policy is a draw from a learner, the learner is the thing under test, so report the spread over training runs and not only over evaluation episodes.

Two corollaries are less comfortable and belong with it. First, more seeds buy resolution, not reliability: doubling from three to six training seeds improved the estimates and did not reduce the spread (§5.3.5). On this problem the learner is genuinely inconsistent run to run, and §5.3.6 shows what that costs where it matters most. Second, and sharper, the sampling remedy does not always exist. The mock-NS-3 decorrelation of Table 5.7 is not insufficient sampling: over six seeds the mock's own spread is 0.008 and its rank correlation with the packet-level body is -0.086 , so *no* amount of mock evaluation identifies which checkpoint transfers. The lesson upgrades accordingly — seeds must be sampled *and* graded on the backend the claim is about.

5.3.9 Summary

Five claims survive this evaluation, and three do not.

The learned pair beats every one of seven comparison points under all three application intents on the analytical backend, with stable rankings and margins of 11.5 – 21.4% over the strongest hand-written policy, a WebRTC-style deployed stack. The permutation-equivariant path head is what makes that possible: substituting a flat one costs 0.231 QoE and drops the learned pair from first place in nine of nine cells to none. A single policy conditioned on the application's intent matches the best specialist under all three intents, at a third of the training cost, and

adapts continuously across the space between them. The scheduler’s contribution scales with both the heterogeneity of path headroom and the strictness of the application’s intent — the second being the axis this chapter shares with Chapter 4. And Proposition 5.1 holds exactly for additively separable scorers, discharging for that function class the claim that the cancellation of Chapter 4 covers softmax as well as argmax selectors.

Three claims do not survive. Intents do not produce measurably intent-specific policies: no margin in the transfer matrix clears its own training-seed spread. The ranking does not transfer zero-shot to the packet-level backend: three of six checkpoints keep first place and three do not, and mock score does not predict which. And global-only conditioning is not inert, as Proposition 5.1 would suggest for a system whose scorer met its premise: a trained nonlinear encoder routes the intent past the cancellation, so the proposition constrains the architecture for exactly one layer.

The last two are the more valuable of the negatives. Together they say that the theory identifies where conditioning must enter for a *guarantee* while the experiment shows a deep encoder arriving there without one, and that a cheap surrogate can be an adequate training environment while being useless for selecting among the policies it produces. Both are stronger statements than the results they replaced.

Per-Path Network State & Allocation — Hierarchical RL (Ours)

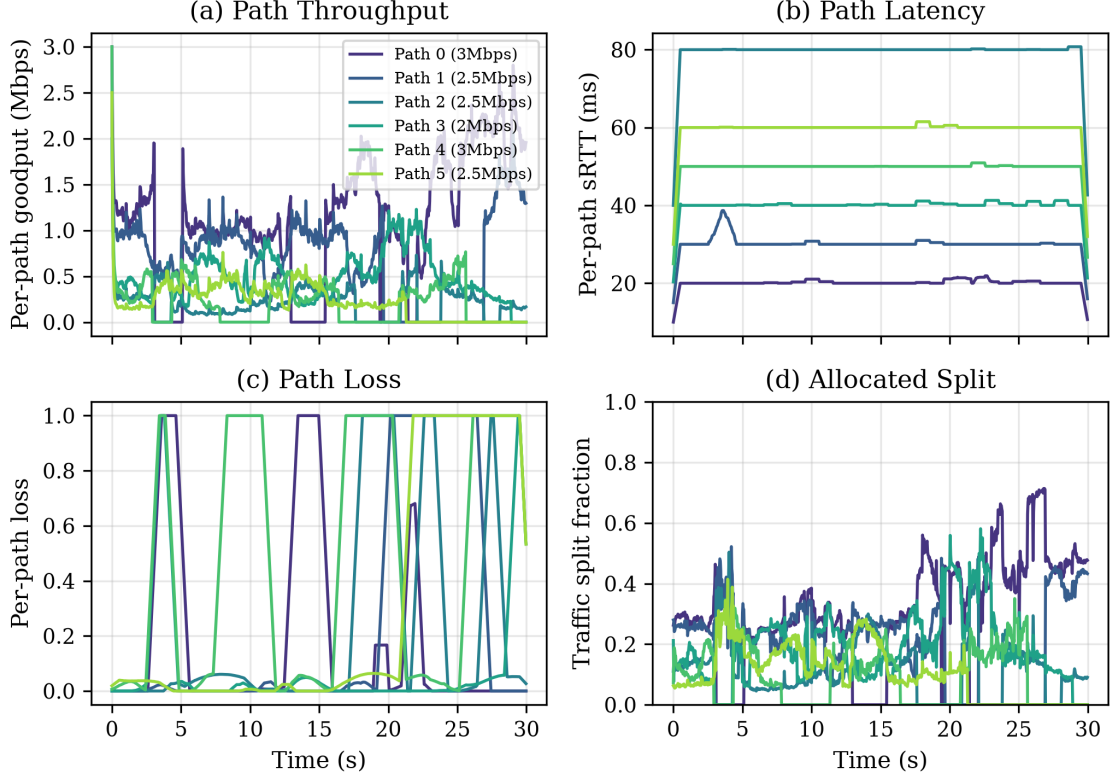


Figure 5.1: The mechanism, within a single 30 s episode of the dynamic six-path scenario under interactive. Per-path goodput (a), smoothed RTT (b), and loss (c) are the state the transport agent observes; (d) is the split it produces. A path taken down by churn appears in (c) as loss saturating at 1.0 — path 0 at ≈ 3 –5.5 s and again at ≈ 13 –15.5 s, path 4 at ≈ 7 –11 s — and in every case the corresponding trace in (d) falls to zero over the same window and recovers when the path does. The agent also shades allocation away from the 40 ms latency trap (path 2, the upper curve in (b)) without abandoning it. Source: `rl-mpquic/runs/ab-prof2-evinteractive-trinteractive/interactive/seed-1000-t1`.

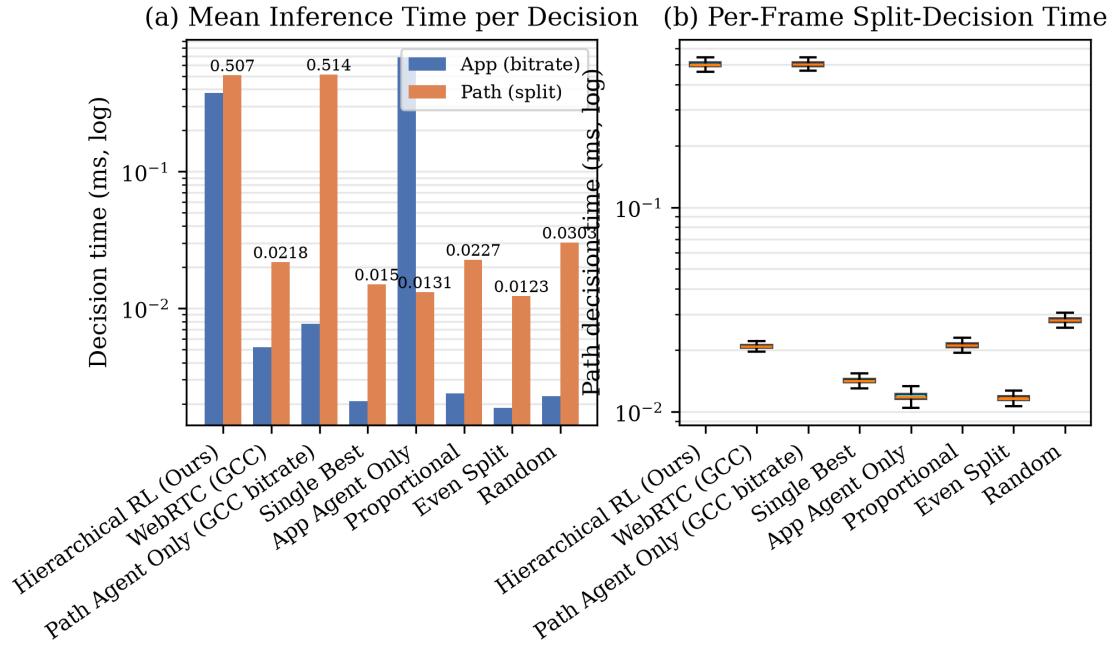


Figure 5.2: Inference cost per decision, by method, on a 16-core CPU host with no GPU and with warm-up calls discarded. (a) Mean cost of the bitrate decision and of the split decision, on a log scale. (b) Distribution of the per-frame split decision, the design’s binding real-time constraint. The learned split costs 0.507 ms against a 33ms frame budget at 30 fps. Source: `rl-mpquic/runs/ab-prof2-evinteractive-trinteractive/interactive/seed-1000-t1`.

6 Discussion and System Synthesis

This chapter steps back from the two systems to their joint meaning. Section 6.1 synthesizes what intent-conditioned single-path selection and joint multipath co-optimization say together about intent-driven network control. Section 6.2 then analyzes both systems along the axes that decide whether learned path control is practical – overhead, scalability, and generalization – and §6.3 states the deployment gaps and trust assumptions under which the controllers operate. Where a claim depends on measurements still in progress, it is marked.

6.1 Synthesis of Findings

Read together, the two systems tell a consistent story about *when* additional control machinery earns its keep.

6.1.1 Interpretation

Intent conditioning turns a family of policies into one. The central lesson of Chapter 4 is that the right way to serve many objectives is not many models but one model that takes the objective as input – provided the objective reaches the part of the network that actually decides. Proposition 4.1 is the crisp version of this point: a conditioning signal that enters identically across the alternatives cancels under the comparison that picks the action, so the policy can look conditioned while behaving identically. Supplying the objective to the per-path terms avoids this, because the objective can then interact with each candidate’s own feature values and change which one wins. This is a design principle, not a SCION-specific trick: any comparison-based selector conditioned on an objective must route that objective into its per-alternative terms. It holds in both settings this thesis studies, and Proposition 5.1 makes it sharper in the second — under a softmax allocation the cancellation is exact on the entire action, not merely on which alternative wins.

But the guarantee is narrower than the behavior. The multipath system supplies the qualification, and it is the more useful half of the result. Proposition 5.1’s premise is that the conditioning be additive *on the terms the comparison consumes*, and three independent mechanisms in a perfectly ordinary architecture break it: a saturating nonlinearity between the conditioning and the comparison, a shared encoder that mixes the conditioning into each alternative’s hidden representation,

and an observation normalizer that is itself a function of the objective. Measured on trained policies, a globally conditioned agent that theory predicts should be inert is instead a working one, indistinguishable in behavioral divergence from the per-alternative variant (§5.3.5). The reconciliation is that the theory says where conditioning must enter to be *guaranteed* to act, while a deep encoder learns to get there without a guarantee. Both statements are needed. A designer who has only the first over-engineers; one who has only the second is relying on an accident of the function class, and Chapter 4’s value-stream agent — which did re-rank three times less — is what that accident looks like when it does not happen.

The two decisions are not separable. The first lesson of Chapter 5 is the one the design was built to test: removing either agent forfeits most of the achievable QoE. An application agent over a naive split recovers as little as 35% of the joint system, and a learned split under a conventional bitrate controller as little as 84% (§5.3.7). Rate and split are coupled through the same bottleneck, and a controller that fixes either one is optimizing against a constraint it could have moved. The decomposition is a claim about the analytical backend: on the packet-level one the application-only ablation recovers 80% rather than 35%, and the network-only ablation costs nothing measurable at all. What the packet-level evidence supports is that the learned *scheduler* is worth having; that the learned bitrate controller on top of it is worth having remains a mock-backend result.

What governs the payoff is headroom, not motion. The second lesson is a correction to the intuition we began with. It is tempting to say that learned multipath scheduling matters when the network is volatile, and the correction matters because that framing would misdirect anyone deciding whether to deploy it. The governing quantity is the relationship between the load an even split places on each path and that path’s residual capacity (§5.3.3). Churn, regime shifts, and bursts matter because they destroy that evenness transiently — but they are not the variable. A stationary but *asymmetric* topology defeats application-only adaptation as thoroughly as a churning one, while a stationary topology of near-equal paths leaves the scheduler almost nothing to earn. Two practical implications follow. One should not benchmark a multipath scheduler on a well-balanced topology, stationary or not, because that is the regime in which any allocation works — and, as §5.3.3 shows, the regime in which the measured margin can fall below the spread between training runs of the same method, so the benchmark reports its own seed draw. And the payoff scales with the application’s intent as well as with the network: the stricter the delay budget, the less slack there is to absorb a poor allocation, so the more the allocation matters.

A surrogate can be good enough to train in and useless to choose in. The last lesson is methodological and was not one we set out to learn. Six policies trained identically on the analytical backend are indistinguishable there — a spread of

0.008, smaller than a single arm’s evaluation-seed interval — and separate into two clean clusters on the packet-level backend, three retaining their first-place ranking and three not. The rank correlation between the two backends is -0.086 : absent, not weak, with the best surrogate checkpoint the worst on the realistic body (§5.3.6). The mechanism is identifiable and mundane. The variable that decides the packet-level outcome is delay-weighted allocation — where the bytes go, priced by what each path costs at that moment — and the surrogate’s analytical queue compresses it into a 1.9 ms band where the realistic body spans 9.7 ms. A model can therefore be faithful enough to learn a good policy in and blind to the dimension that separates good policies from bad ones. The practical rule is narrow and firm: select checkpoints on the backend the claim is about, or report the distribution over training seeds and select nothing.

One principle, two layers. Both systems instantiate the Mortise principle [16] – intent as a first-class input that reshapes a network mechanism – at successively deeper points of control. The single-path selector reshapes *which path* is chosen; the multipath system reshapes both *how the paths are used* and *what the application produces*. In both, one conditioned policy replaces a family of per-objective ones. The progression suggests a general recipe: expose the application’s intent, and let it steer every controllable decision between the application and the wire.

6.1.2 Implications

The results matter for how we build controllers on path-aware architectures. Prior learned path selectors [3] and learned congestion or scheduling controllers [9, 23, 20] each optimize a fixed objective; our work argues that on a path-aware substrate the objective should be a knob, and that a single conditioned policy can replace a zoo of per-objective models. For real-time media specifically, the joint bitrate-and-split result reinforces a theme from learned ABR [11] – that application and network decisions are better made together – and extends it to the multipath setting that SCION makes routine. More broadly, the two systems are evidence that SCION’s endpoint path choice is not merely a routing convenience but an enabler of application-aware optimization that is awkward or impossible in a destination-based Internet.

6.1.3 Limitations

The results are bounded in ways that the analysis (§6.2) details and that fairness requires restating here. Both systems are evaluated in simulation. The multipath system’s headline numbers are measured on the analytical mock, and on the packet-level backend they hold for three of six independently trained policies and not for the other three, with absolute QoE not portable in any case (§5.3.6). All of its policies are trained on the mock, so whether the system *learns* on the realistic

body is untested (§5.2.4); and since mock score does not predict packet-level rank, the mock’s standing as a training surrogate is an open question rather than a technicality. The parity of the intent-conditioned policy with the per-intent specialists is a failure to detect a difference at the measured training-seed spread, not a demonstration of equivalence, and the specialists it matches were themselves not measurably specialized (Sections 5.3.4 and 5.3.5). The single-path study rests on one topology and one training seed, with a purely temporal train/test split, so its quantitative margins should be read as indicative rather than as established effect sizes (Sections 4.4.1 and 4.4.2). Intent in this thesis is *declared*, not *inferred*: we assume the application can state its objective, and we do not study how to recover a latent intent from traffic – a natural but separate problem. The transport model is TCP-like rather than true Multipath QUIC, and the VMAF surrogate is bitrate-dominated, so absolute QoE figures should be read as relative comparisons rather than calibrated user scores. Finally, we have not addressed online, safe learning against live traffic, which any deployment would require. None of these undercut the two qualitative findings – conditioning that reaches the per-alternative comparison is the only kind guaranteed to act, and joint rate-and-split control pays off in proportion to how unevenly path capacity is distributed and how strictly the application prices delay – but they do delimit how far the quantitative results should be pushed.

[TODO: two limitations are missing from a list that is otherwise exemplary, and both are ones a reader will find on their own if this section does not state them. (i) Chapter 5 does not run on SCION. There is no control plane, no beaconing, no path store; the six paths are a hand-specified configuration (Table A.4) and the study assumes only path multiplicity. Given the thesis title, this belongs in the limitations list explicitly rather than being inferable from §5.2. (ii) Chapter 4’s seed coverage is partial. The paragraph above says “one topology and one training seed”, which understates the situation in one direction and overstates it in another: the ablation ladder is at five seeds with disjoint intervals (Table 4.3), while the intent matrix, zero-shot sweep, path-count study, probing comparison and ceiling figure are all single-run. Say which is which. The asymmetry matters because §5.3.8 makes single-run reporting the thesis’s own cautionary tale, and a limitations section that blurs the distinction invites the reader to discount the results that are properly seeded along with those that are not.]

6.2 System Analysis

This section analyzes the two systems along the dimensions that determine whether learned path control is practical: overhead (§6.2.1), scalability (§6.2.2), and generalization (§6.2.3).

6.2.1 Overhead

Two kinds of overhead matter: measurement and computation.

Measurement (probing) overhead. In the single-path selector (Chapter 4), probing is a first-class cost charged in the reward (Eq. (4.2)) and accounted per decision (§4.3.3). The learned selector probes only the paths it inspects, in contrast to heuristics that require a metric on every path, so its measurement footprint is a fraction of an exhaustive scan at comparable selection quality – and, in fact, smaller than that of every heuristic baseline, since each of those must measure all candidates before applying its rule (§4.4.6). In the multipath system (Chapter 5), per-path state is derived from ordinary transport feedback (RTT samples, acknowledgements, loss signals) rather than dedicated probes, so the marginal measurement cost of the split decision is low.

Computational overhead. Both policies are small neural networks evaluated once per decision. The single-path selector runs at path-selection granularity (seconds to minutes) and is comfortably cheap. The multipath transport agent is the tighter constraint: it runs *per frame* (≈ 33 ms at 30 fps), so its inference must fit well inside a frame interval. The scoring architecture keeps this affordable – a shared per-path encoder evaluated N times and a softmax – but the per-frame budget is the binding real-time constraint. For the single-path selector the margin is ample: timing only the decision itself, over 1280 held-out contexts of 30 candidate paths each on a 22-core CPU host with no GPU, the conditioned selector decides in under 0.3 ms on average and under 2 ms at the 99th percentile across repeated runs. That is three orders of magnitude inside a path-selection budget of seconds to minutes, and comfortably inside a 33 ms video frame. The multipath transport agent, whose per-frame budget is the tighter of the two, has the same shape of answer: it decides in 0.507 ms on average on a 16-core CPU host with no GPU, about 1.5% of the frame interval it must fit inside (Fig. 5.2). Both controllers are therefore compute-bound nowhere near their deadlines, and the cost grows linearly in the candidate count in each case, since the shared per-path encoder is evaluated once per path. *[TODO: add the 99th percentile for the transport agent alongside the mean once Fig. 5.2 is regenerated — a real-time claim is a claim about the tail, and the single-path figures above are quoted with both. The distribution is already recorded in every `evaluation_results.json`.]*

6.2.2 Scalability

The scoring architecture is what makes both systems scale in the number of paths. Because a shared network scores each path independently and decisions are made by comparison (§4.1.4) or softmax (§5.1.5), the parameter count is independent of N , and inference is linear in N . This lets one trained model serve source–destination pairs with any number of paths and topologies of any size, without retraining or padding to a fixed maximum – the limitation of the flat agents. The remaining scalability question is training coverage: the environment must expose enough variety of path counts, topologies, and conditions for the shared scorer to

generalize. *[TODO: report how selection quality holds as topology size and path count grow, ideally on topologies larger than those seen in training.]*

6.2.3 Generalization

Generalization is required along three axes. **Across intents:** the conditioning of Chapter 4 is meant to interpolate to intents between (and, ideally, beyond) the trained reward profiles, so that a novel weight vector yields sensible behavior without retraining. Section 4.4.4 confirms it: sweeping the intent along the segment between two trained profiles moves the chosen path’s latency monotonically, with a Spearman correlation of -1.000 and no single step carrying more than a quarter of the span, and modest extrapolation beyond both endpoints continues the trend. The multipath conditioning generalizes along the same axis but with a weaker warrant: sweeping the intent between two profiles moves bitrate and quality smoothly and monotonically, by 17–24 times the span a non-learning control shows over the same rollouts, but training there sampled the intent space continuously, so the interior of the segment is in-distribution rather than unseen (§5.3.5). Extrapolation past the endpoints fails asymmetrically in both systems, and in the multipath case informatively so: loosening the intent beyond anything trained holds QoE, tightening it beyond the trained range collapses it – which is the expected direction, since the latter asks for a deadline below what the network can deliver rather than merely an unfamiliar one. **Across topologies and path counts:** the permutation-equivariant design gives structural generalization, and §4.4.5 confirms both halves of it empirically – exact order invariance over 32256 permutation trials, and regret held two orders of magnitude below the fixed-action agent across a 15-fold range in candidate count. Section 4.4.2 adds a structurally disjoint set of 32 pairs spanning 32 source ASes never visited in training, on which the conditioned selector’s optimality gap grows only from 0.0018 to 0.0033 while the flat agent’s Throughput reward goes negative. What remains untested is scale: all of this is one topology of 90 ASes. **Across conditions:** the dynamic scenario of Chapter 5 (§5.3.2) is itself a generalization stress test, since the policy must cope with churn and bursts rather than a stationary environment; the collapse of the fixed-schedule baselines there shows that a fixed allocation rule does *not* survive a network whose headroom keeps shifting underneath it, which is the gap learned control fills. For the single-path system, off-distribution degradation is bounded on two of these three axes: unseen intents cost nothing measurable (§4.4.4), and unseen pairs and candidate-set sizes cost at most about 0.01 reward (Sections 4.4.2 and 4.4.5). The held-out-dynamics axis is now measured for the multipath system and the answer is a clean negative: graded on two configurations whose hazard rates lie outside anything either arm trained over, domain randomization costs a further 0.033 QoE in nine of nine cells, on top of the 0.027 it costs on the nominal environment (§5.3.7). The more permissive reading that was available before this run – that randomization trades nominal

performance for robustness – is not supported: it loses on both axes here. One of the two held-out environments is uninformative rather than contrary, both arms scoring negative under rankings that do not hold.

[TODO: what remains unquantified on this axis is scale, for both systems: the single-path results are one topology of 90 ASes and the multipath results one six-path configuration, so nothing here bounds decay as topology size grows. This is the more expensive of the two remaining generalization questions and the only one still open.]

6.3 Deployment Considerations and Trust Assumptions

6.3.1 Deployment Gaps

Several gaps still separate these prototypes from a deployment.

- **From simulation to reality.** Both systems are trained in simulation. The single-path study relies on a synthetic traffic and beaconing environment; the multipath system on NS-3 and a mock transport. Bridging the gap to deployment would require sim-to-real transfer – for example through domain randomization or online fine-tuning against live measurements – and the packet-level NS-3 backend is the intended stepping stone (§5.2.4).
- **Real transport.** The multipath system models subflows with TCP-like congestion control rather than true Multipath QUIC [5]. A real QUIC integration is therefore needed to validate the per-frame scheduling under an actual multipath transport.
- **Online and safe learning.** A deployed agent would learn or adapt against live traffic, where exploration has real cost. Bounding worst-case behavior – for example by falling back to a safe heuristic when the policy is uncertain, or by constraining exploration – is a prerequisite that this work does not address.

Two properties help on the positive side. First, the endpoint-driven nature of SCION path selection means these controllers require no network-internal changes. Second, the horizon-agnostic application agent (§5.1.2) is designed for unbounded real sessions rather than fixed-length episodes.

6.3.2 Trust Assumptions

This thesis is a performance and control study, not a security study, so in place of a classical attacker model we state the trust assumptions under which the learned controllers operate and note where security considerations would enter a real deployment.

- **Honest measurements.** We assume that the per-path measurements an agent acts on – whether obtained by active probing (Chapter 4) or transport feedback (Chapter 5) – are honest. A malicious on-path element that fabricates favorable metrics could steer the agent toward a bad path; defending against that is out of scope, although the trust term below offers partial mitigation.
- **SCION path authenticity and trust.** SCION’s cryptographically authenticated paths mean an agent knows *which* ASes a path traverses and cannot be silently redirected. The single-path reward includes a *trust* component derived from path quality, allowing the agent to prefer higher-confidence paths and discount those with anomalous loss or latency – a lightweight hedge against unreliable or adversarial path advertisement, not a full defense.
- **No adversarial control of the learner.** We assume the agent’s own training and inference pipeline is trusted; adversarial manipulation of the RL policy (e.g. reward poisoning or observation attacks) is beyond this work.

A production deployment would need to revisit these assumptions – in particular by verifying measurement integrity and by bounding the damage a single misreported path can do.

7 Related Work

Where Chapter 2 covered the systems this thesis *builds on*, this chapter surveys work that attacks the *same or similar problems* and positions our contributions against it. We group it into learned routing and traffic engineering (§7.1), learned congestion control and multipath scheduling (§7.2), adaptive bitrate and QoE-driven control (§7.3), goal-conditioned and modulated policies (§7.4), and intent-based networking (§7.5).

7.1 Learned Routing and Traffic Engineering

A line of work applies reinforcement learning to routing and traffic engineering. *Learning to Route* [17] studies RL and learned representations for routing, and experience-driven networking [21] uses deep RL for traffic-engineering decisions. These target network-operator objectives (e.g. minimizing congestion globally) from a control-plane vantage point. Our single-path selector differs in both the actor and the objective: the decision is made by an *endpoint* choosing among cryptographically authenticated SCION paths, and the objective is the *application's* intent rather than a global network utility. Most directly related is the DQN SCION path selector of Bourak et al. [3], which we extend; the differences – a variable-action scoring architecture and per-path intent conditioning that removes the single-objective limitation – are the substance of Chapter 4.

[TODO: add how SCION endpoints select paths today, so the learned selector has a stated status quo rather than only a learned predecessor: the path policy language and path-service interface, and the path-aware transport APIs that expose selection to applications (PANAPI and related work). This is the deployed alternative to everything in Chapter 4, and a Network Security Group reader will know it well enough to notice its absence. It also gives §4.3.3's "SCION default" heuristic a citation, which it currently lacks — the baseline is defined in the implementation section by description alone. Pairs with the background gap at §2.1.]

7.2 Learned Congestion Control and Multipath Scheduling

Learned transport controllers include Aurora [9], which casts congestion control as RL, and Orca [1], which combines a learned layer with a classic controller. Mortise [16] is the closest in spirit, and we treat it separately below. For multipath

scheduling, ReLeS [23] learns a Multipath QUIC scheduler and Peekaboo [20] learns to schedule across heterogeneous paths in dynamic environments; both optimize a scheduling policy for a given, exogenous load. The multipath system’s distinction is that it does *not* treat the offered load as fixed: it co-optimizes the application’s bitrate with the split, so the controller can relieve congestion it would otherwise create – a coupling those schedulers, and the Multipath TCP mechanisms of RFC 8684 [6], do not address.

[TODO: this section discusses the two learned multipath schedulers and omits the non-learned ones the field actually deploys and benchmarks against — minRTT, the default scheduler in every Multipath TCP and Multipath QUIC implementation, and the BLEST and ECF refinements that address its head-of-line-blocking failure mode. They belong here for two reasons: they are the status quo the learned scheduler of Chapter 5 improves on, and their absence from Table 5.3 is the first thing a transport reviewer will check (see the baselines gap at §5.3.1). A second omission is larger: learned real-time video transport is the direct neighbour of this thesis’s second half and no work in it is cited — Salsify’s joint codec–transport control is essentially Chapter 5’s premise on a single path, and the deployed-scale RL video controllers (OnRL, Loki, Concerto) are the closest published systems to what is built here. Citing them costs the chapter nothing — none does multipath and rate jointly, which is precisely the delta to state — and not citing them makes the contribution look unaware of its own neighbourhood.]

7.2.1 Mortise: intent-driven tuning of a transport mechanism

The conceptual template for both parts is Mortise [16]. Mortise observes that a single congestion-control algorithm cannot be optimal for every application, and instead *auto-tunes* the algorithm’s parameters to maximize the running application’s QoE, using the application’s revealed preferences over throughput, latency, and loss to steer the tuning. The essential idea – treat application intent as a first-class input that reshapes a network mechanism rather than a constant compiled into it – is exactly what we transplant. The single-path selector applies it to *path selection* (which path to use), and the multipath system applies it to *joint rate and multipath scheduling* (what the application sends and how it is spread across paths).

7.3 Adaptive Bitrate and QoE-Driven Control

Learned adaptive bitrate control, exemplified by Pensieve [11], showed RL beating hand-crafted ABR heuristics for buffered video streaming, using QoE-shaped rewards and quality metrics such as VMAF [10]. Our multipath system shares the QoE-driven, learned-bitrate stance but targets *real-time* video (no playback buffer, deadline-based loss) and, crucially, couples the bitrate decision to a simultaneous multipath scheduling decision. Where Pensieve adapts rate to a single, exogenous

network path, we adapt rate *and* decide how to spread it across several paths, jointly.

7.4 Goal-Conditioned and Modulated Policies

The machinery for serving many objectives with one policy comes from goal-conditioned RL – universal value function approximators [14] and hindsight relabeling [2], which learn a value function jointly over states and goals. That literature is largely silent on *where* in a network the goal should enter, because its policies are not comparison-based. Ours are, and our contribution is a specific insight about that gap: for a comparison-based selector, conditioning must reach the per-alternative terms, because an additive, symmetric signal cancels under the comparison. We establish this for both forms the comparison takes here — under an arg max over discrete candidates (Proposition 4.1) and under a softmax over continuous allocations (Proposition 5.1), where the cancellation is exact on the whole action rather than only on the winner. We also establish its limit, which is the more practically useful half: the premise is additivity *on the terms the comparison consumes*, and a deep encoder that mixes the conditioning with each alternative’s features escapes it without being designed to (§5.3.5). The principle is therefore a statement about which routings carry a guarantee, not about which ones happen to work. This connects a general representation-learning idea to a concrete failure mode in networked decision-making.

*[TODO: the section positions the contribution against goal-conditioned RL and never mentions **multi-objective RL**, which is the literature this thesis is actually a member of. The intent vector is a preference weighting over a linear scalarization of a multi-objective reward, and the claim that one network can span the family of optima induced by sweeping those weights is the founding result of weight-conditioned MORL — not a consequence of UVFAs, and certainly not of hindsight relabeling, which does not apply to preference goals at all. Cite it, and then state the delta, which is genuinely there and is sharper than the current framing: MORL policies emit actions from a fixed, enumerated action space, so nothing in that literature asks where in a comparison-based scorer the preference must enter, and Propositions 4.1 and 5.1 are exactly that question. Framed against MORL the contribution is narrow, correct and defensible; framed against goal-conditioned RL alone it will read to anyone in the area as a rediscovery. Relatedly, Perez2018 (FiLM) sits unused in the bibliography since feature-wise modulation was measured and dropped on 2026-07-26 — this is the section where that belongs, as one sentence: the obvious alternative mechanism was tried, tied within seed noise, and cost more parameters.]*

7.5 Intent-Based Networking

Finally, “intent” also names a body of network-management work concerned with translating high-level operator goals into configuration, surveyed in RFC 9315 [4].

That community’s “intent” is an operator’s declarative policy for a whole network; ours is an *application’s* per-flow QoS preference consumed by a learned endpoint controller. The two are complementary rather than competing – one could imagine operator intent bounding the space in which application intent is served – but they operate at different layers and timescales, and our techniques address the fine-grained, per-flow end of the spectrum.

8 Conclusion

This thesis asked how an endpoint on a path-aware Internet should choose the best use of its paths given the intention of the application and the state of the network. It answered in two movements that share one idea, drawn from Mortise [16]: treat application intent as a first-class input that reshapes a network mechanism, not as a constant compiled into it.

Chapter 4 turned a fixed-objective DQN path selector for SCION [3] into an intent-conditioned one. Two design decisions carried the work: a permutation-equivariant *scoring* architecture that handles a variable, changing set of candidate paths, and the routing of the intent vector into the *per-path comparison* itself, which lets a single policy serve a range of objectives – including ones it never trained on – without retraining. Chapter 5 asked what happens if we use several paths at once and optimize the application itself, and built a two-timescale hierarchical agent pair that co-optimizes video bitrate and per-path traffic split over Multipath QUIC, coupled so that each agent accounts for the other – then conditioned that pair on the intent as well, so that one policy serves the range of application objectives three separately trained ones had covered.

What we learned. The best-supported finding is the one the second system was built to test: *the two decisions are not separable*. Rate and split are coupled through the same bottleneck, and a controller that fixes either is optimizing against a constraint it could have moved – removing either agent forfeits most of the achievable quality, and the joint pair leads every comparison point under every application intent at margins several times the spread between training runs of the same method. Close behind it is an architectural lesson we had expected to be a convenience and found to be load-bearing. Scoring each candidate with shared weights is normally adopted so that a policy can accommodate a variable action set; substituting a conventional fixed-dimension head costs more quality than any other single change we measured, and enough that the system’s headline claim reverses outright. It is also the smaller network of the two. Whatever a dynamic-input design costs, on these problems it is not parameters.

Second, that the payoff of joint rate-and-split control is *not* a fixed quantity, and that the variable governing it is not the one we expected. It is tempting to say such control matters when the network is volatile; what we measured is that it matters when path capacity is unevenly distributed relative to what an even split would place on each path, and that churn and bursts matter because they destroy that evenness rather than because motion is itself the relevant property. A stationary

but asymmetric network defeats application-only adaptation as thoroughly as a churning one. The payoff also scales with what the application asks for: the stricter its delay budget, the more a poor allocation costs.

Third, the thread that joins the two systems, stated with the boundary we found on it. *Where* intent is injected matters more than *that* it is injected: for a comparison-based selector, a conditioning signal reaching only the terms shared by every candidate cannot change the decision, because it cancels under the comparison – exactly and on the whole action when that comparison is a softmax. The intent has to reach each alternative’s own features to reorder them. But the guarantee turned out to be narrower than the behavior, and this is the correction we did not anticipate. The cancellation assumes the conditioning is additive on the terms the comparison consumes, and three mechanisms in an ordinary architecture break that assumption, so a trained nonlinear encoder routes intent past a cancellation the theory says should stop it. The principle survives as a statement about which routings carry a guarantee rather than which happen to work, which is the form a designer can actually use. Both settings also taught us how much such conditioning can be worth at all: it is cheap and it costs nothing to add, but it earns something only where the objectives genuinely conflict and there is headroom to exploit, and neither of our environments offered much of either.

A fourth lesson is about method rather than mechanism, and we learned it the expensive way. Four results in this thesis were reported from a single training run and did not survive being run three times, because on this problem the spread between training runs exceeds the confidence interval over evaluation episodes by an order of magnitude. Sampling training seeds is necessary and not sufficient: a cheap simulator can be faithful enough to train good policies in while being uncorrelated with the realistic body on which of them is good. We report the retractions as a result because the pattern is more useful than any of the four numbers it corrected, and because it is the one finding here that generalizes beyond multipath scheduling.

Future work. Several threads remain open, and much of the draft’s structure is deliberately laid out to guide them. On the experimental side, the sharpest question is the one §5.3.6 raises: whether the system *learns* on the packet-level body rather than merely transferring to it, which matters more now that we know surrogate score does not predict packet-level rank. Alongside it, extending the seed-level statistics of Chapter 5 back to the single-path study, which still rests on one training seed, and testing both systems on topologies larger than those they trained on. On the system side: replacing the TCP-like subflows with a real Multipath QUIC transport, and enriching the VMAF surrogate so quality responds to delivery conditions and not just bitrate – a change that would also close the queueing-fidelity gap that makes the analytical backend blind to the variable deciding packet-level outcomes. On the research side, three directions stand out: *inferring* intent from traffic rather than assuming it is declared; *online and*

safe learning against live traffic with bounded worst-case behavior; and unifying the two conditioned controllers, which this thesis builds and evaluates separately, into a single cross-layer policy in which one declared intent steers path selection, traffic split, and application output together.

Bibliography

- [1] S. Abbasloo, C.-Y. Yen, and H. J. Chao. Classic meets modern: A pragmatic learning-based congestion control for the Internet. In *Proceedings of the ACM SIGCOMM Conference*, pages 632–647, 2020.
- [2] M. Andrychowicz, F. Wolski, A. Ray, J. Schneider, R. Fong, P. Welinder, B. McGrew, J. Tobin, P. Abbeel, and W. Zaremba. Hindsight experience replay. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 30, 2017.
- [3] O. Bourak, T. Burman, T. John, H. Daltrophe, T. Shrot, and D. Hausheer. Path optimization using DQN in the SCION Internet architecture. In *Cyber Security, Cryptology, and Machine Learning (CSCML 2025)*, volume 16244 of *Lecture Notes in Computer Science*. Springer, Cham, 2025.
- [4] A. Clemm, L. Ciavaglia, L. Z. Granville, and J. Tantsura. Intent-based networking – concepts and definitions. RFC 9315, Internet Engineering Task Force (IETF), 2022.
- [5] Q. D. Coninck and O. Bonaventure. Multipath QUIC: Design and evaluation. In *Proceedings of the 13th International Conference on emerging Networking EXperiments and Technologies (CoNEXT)*, pages 160–166, 2017.
- [6] A. Ford, C. Raiciu, M. Handley, O. Bonaventure, and C. Paasch. TCP extensions for multipath operation with multiple addresses. RFC 8684, Internet Engineering Task Force (IETF), 2020.
- [7] T. Haarnoja, A. Zhou, P. Abbeel, and S. Levine. Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor. In *Proceedings of the 35th International Conference on Machine Learning (ICML)*, pages 1861–1870, 2018.
- [8] J. Iyengar and M. Thomson. QUIC: A UDP-based multiplexed and secure transport. RFC 9000, Internet Engineering Task Force (IETF), 2021.
- [9] N. Jay, N. H. Rotman, B. Godfrey, M. Schapira, and A. Tamar. A deep reinforcement learning perspective on Internet congestion control. In *Proceedings of the 36th International Conference on Machine Learning (ICML)*, pages 3050–3059, 2019.

- [10] Z. Li, A. Aaron, I. Katsavounidis, A. Moorthy, and M. Manohara. Toward a practical perceptual video quality metric. Netflix Technology Blog, 2016. <https://netflixtechblog.com/toward-a-practical-perceptual-video-quality-metric-653f208b9652>.
- [11] H. Mao, R. Netravali, and M. Alizadeh. Neural adaptive video streaming with pensieve. In *Proceedings of the ACM SIGCOMM Conference*, pages 197–210, 2017.
- [12] V. Mnih, K. Kavukcuoglu, D. Silver, A. A. Rusu, J. Veness, M. G. Bellemare, A. Graves, M. Riedmiller, A. K. Fidjeland, G. Ostrovski, et al. Human-level control through deep reinforcement learning. *Nature*, 518(7540):529–533, 2015.
- [13] A. Perrig, P. Szalachowski, R. M. Reischuk, and L. Chuat. *SCION: A Secure Internet Architecture*. Springer, 2017.
- [14] T. Schaul, D. Horgan, K. Gregor, and D. Silver. Universal value function approximators. In *Proceedings of the 32nd International Conference on Machine Learning (ICML)*, pages 1312–1320, 2015.
- [15] T. Schaul, J. Quan, I. Antonoglou, and D. Silver. Prioritized experience replay. In *International Conference on Learning Representations (ICLR)*, 2016.
- [16] Y. Shen, R. Chen, B. Wang, J. Chen, H. Zhang, M. Wang, Y. Liu, M. Xu, and Z. Meng. Mortise: Auto-tuning congestion control to optimize QoE via network-aware parameter optimization. In *22nd USENIX Symposium on Networked Systems Design and Implementation (NSDI 26)*. USENIX Association, 2026.
- [17] A. Valadarsky, M. Schapira, D. Shahaf, and A. Tamar. Learning to route. In *Proceedings of the 16th ACM Workshop on Hot Topics in Networks (HotNets)*, pages 185–191, 2017.
- [18] H. van Hasselt, A. Guez, and D. Silver. Deep reinforcement learning with double Q-learning. In *Proceedings of the 30th AAAI Conference on Artificial Intelligence (AAAI)*, pages 2094–2100, 2016.
- [19] Z. Wang, T. Schaul, M. Hessel, H. van Hasselt, M. Lanctot, and N. de Freitas. Dueling network architectures for deep reinforcement learning. In *Proceedings of the 33rd International Conference on Machine Learning (ICML)*, pages 1995–2003, 2016.
- [20] H. Wu, O. Alay, A. Brunstrom, S. Ferlin, and G. Caso. Peekaboo: Learning-based multipath scheduling for dynamic heterogeneous environments. *IEEE Journal on Selected Areas in Communications*, 38(10):2295–2310, 2020.

- [21] Z. Xu, J. Tang, J. Meng, W. Zhang, Y. Wang, C. H. Liu, and D. Yang. Experience-driven networking: A deep reinforcement learning based approach. In *IEEE INFOCOM 2018 – IEEE Conference on Computer Communications*, pages 1871–1879, 2018.
- [22] M. Zaheer, S. Kottur, S. Ravanbakhsh, B. Póczos, R. Salakhutdinov, and A. J. Smola. Deep sets. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 30, 2017.
- [23] H. Zhang, W. Li, S. Gao, X. Wang, and B. Ye. ReLeS: A neural adaptive multi-path scheduler based on deep reinforcement learning. In *IEEE INFOCOM 2019 – IEEE Conference on Computer Communications*, pages 1648–1656, 2019.

A Reproducibility Details

This appendix collects the hyperparameters, reward-profile definitions, and supplementary results that support the main text. Values still to be finalized from the experiment runs are marked.

A.1 Single-Path Selection: Reward Profiles

The conditional path-selection agent is trained and evaluated across the reward-weight vectors $\mathbf{w} = (w_1, w_2, w_3, w_4, w_{\text{probe}})$ of Eq. (4.2), spanning distinct application intents. Table A.1 lists the six profiles used, together with the label each carries in Chapter 4.

Table A.1: The six reward profiles (application intents) used in the single-path selection study (Chapter 4). w_1 weights goodput, w_2 trust, w_3 loss and w_4 delay within the trust term, and w_{probe} the probing penalty. All six are used in training on a stratified schedule; the four marked with a chapter label are also used in the evaluation of §4.4.

Profile	Label in Chapter 4	w_1	w_2	w_3	w_4	w_{probe}
bandwidth_max	Throughput	1.00	0.00	0.05	0.05	0.05
delay_averse	Low-Latency	0.10	0.90	0.15	1.00	0.05
loss_averse	Low-Loss	0.05	0.95	1.00	0.15	0.05
balanced_extreme	Balanced	0.55	0.45	0.55	0.55	0.05
probe_minimal	–	0.70	0.30	0.50	0.50	0.001
probe_averse	–	0.70	0.30	0.50	0.50	0.35

A.2 Single-Path Selection: Agent Hyperparameters

Table A.2 lists the hyperparameters shared by the flat, scoring, and conditional agents of §4.3.1; the flat agent differs only in training for 533 episodes rather than 666.

Two caveats on this table. The ε schedule is stated as its configured range, but a run is too short to traverse it: at 0.995 decay per episode, ε reaches only 0.036 after 666 episodes and 0.069 after 533, so the 0.01 floor is never attained and all reported agents retain a few percent of exploration at the end of training. Seeding

Table A.2: Hyperparameters for the single-path selection agents (Chapter 4).

Parameter	Value
Hidden layers / width	2 / 128
Learning rate (Adam)	10^{-3}
Discount γ	0.95
Batch size	32
Replay capacity	10^4 (minimum 500 before learning)
Target update	soft (Polyak), $\tau = 0.05$, every 100 gradient steps
Prioritized replay	$\alpha = 0.6$, β annealed $0.4 \rightarrow 1.0$
Exploration	ϵ -greedy, $1.0 \rightarrow 0.01$, decay 0.995/episode
Dropout	0.1
Gradient clipping	1.0
Episodes $\times$ steps	666×24 (flat: 533×24)
Gradient step frequency	every 4 environment steps
Invalid-action mask value	-10^9
Random seeds	5 (all agents)
Wall-clock per agent	1–3 min (22-core CPU, no GPU)

fixes network initialization, prioritized-replay sampling, and ϵ -greedy exploration; the environment’s pair, hour, and profile streams are deliberately left fixed across seeds, so every seed sees the identical stream of training contexts and only the learning process varies.

A.3 Multipath System: QoE Weights and SAC Hyperparameters

The QoE reward of Eq. (5.1) uses base weights $(a, b, c, d, e) = (1, 0.5, 0.5, 1, 0.25)$, overridden per intent by Table 5.1; the application agent acts every 1 s and the transport agent every frame (≈ 33 ms at 30 fps) over a bitrate range of 300–6000 kbit/s. Tables A.3 and A.4 give the learner and scenario parameters.

The non-stationary mechanisms are hazards converted per frame from per-second rates. Path churn takes a live path down at 0.08/s and revives a dead one at 0.30/s, with at least three of six paths live at all times. Regime shifts resample a path’s capacity multiplier uniformly in $[0.55, 1.25]$ at 0.20/s. Congestion bursts drop capacity to 45% for 0.5 s at 0.10/s. The correlated-failure group $\{5, 6\}$ drops to 50% for 1.2 s at 0.05/s.

Intent conditioning. The conditioned agents of §5.3.5 append a five-component intent vector to the observation: the delay, jitter, and loss weights $(b, c, d/2)$ of Eq. (5.1), the frame deadline, and the latency normalizer, the last two scaled

Table A.3: Soft actor-critic hyperparameters shared by the application and transport agents (§5.2.3). Source: `rl-mpquic/configs/dynamic.yaml`.

Parameter	Value
Hidden layers / width	2 / 256
Learning rate (Adam)	3×10^{-4}
Discount γ	0.99
Target update	soft (Polyak), $\tau = 0.005$
Batch size	256
Replay capacity	2×10^5
Random warm-up / first update	1000 / 1000 environment steps
Temperature α	auto-tuned, target entropy $-d_{\text{act}}$
Critic LayerNorm (DroQ)	enabled
Updates per environment step	2
Prioritized replay	enabled
Episodes $\times$ length	100×30 s (900 frames)
Wall-clock per run	4–4.5 h (16-core CPU, no GPU)

Table A.4: The dynamic six-path scenario (§5.3.1): per-path bottleneck rate, propagation delay, and mean cross-traffic as a fraction of that rate. No path can carry full-quality video alone. Paths 5 and 6 form a shared-bottleneck group that degrades together. Source: `rl-mpquic/configs/dynamic.yaml`.

Path	Rate	Delay	Character
1	3.0 Mbit/s	10 ms	clean, low RTT (cross 0.40)
2	2.5 Mbit/s	15 ms	clean (cross 0.45)
3	2.5 Mbit/s	40 ms	latency trap (cross 0.30)
4	2.0 Mbit/s	20 ms	congested (cross 0.55)
5	3.0 Mbit/s	25 ms	shared bottleneck A (cross 0.50)
6	2.5 Mbit/s	30 ms	shared bottleneck B (cross 0.50)

by 800 ms. The existing observation features are left untouched and the normalizer is disclosed as an input rather than fixed across intents, which is what keeps the per-intent specialists of Table 5.4 a valid baseline against the conditioned policy — and is also the third of the three premise violations reported in §5.1.6, since that normalizer divides a per-path feature. Two conditioning modes are compared: `global`, which supplies the vector to the global context slot alone, and `perpath`, which additionally broadcasts it to every per-path feature row, adding $5 \times 6 = 30$ input dimensions. Training randomizes the intent per episode, stratified over the three anchors of Table 5.1 plus a continuously sampled interior; the randomization is training-only and evaluation never re-

samples. Both arms are trained for six seeds at the budget of Table A.3. Source: `rl-mpquic/configs/dynamic_intent{,_glob}.yaml`.

[TODO: state the static-scenario topology alongside these once §5.3.3 settles which static configuration the chapter reports. Two exist in the implementation and they are not interchangeable: an asymmetric three-path topology (8/4/2 Mbit/s), on which an even split overdrives the weakest path and app-only collapses, and a near-equal four-path topology, on which app-only comes within 2% of the full system. The chapter’s argument in §5.3.3 turns on the difference, so both belong in this table with their headroom spread stated explicitly.]

A.4 Supplementary Results

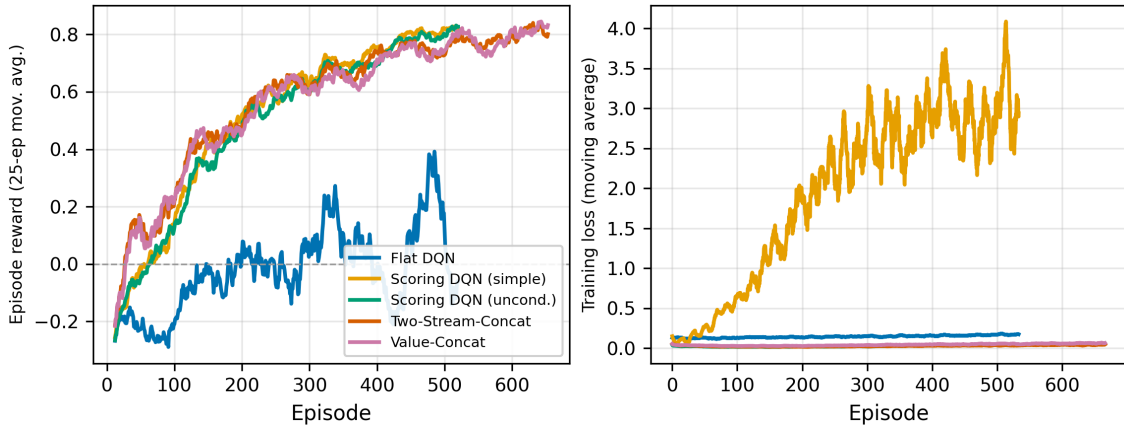


Figure A.1: Training curves for the single-path agents of §4.3.1: episode reward (25-episode moving average, left) and training loss (right) over the 533 or 666 episodes each agent was trained for. The four path-scoring agents converge to within 0.01 of one another at roughly +0.81, while the flat DQN plateaus near +0.05 with a *rising* loss despite the identical budget, data, and optimizer – the evidence that its deficit is architectural rather than a matter of training time. The simple scoring agent’s loss also diverges, which is why the enhanced variant is used throughout.

Figure A.1 substantiates the convergence discussion of §4.3.1. The per-episode statistics also carry the stratified schedule’s profile label for each episode, giving roughly 111 episodes per intent for the conditional agents; these per-intent curves are the only place the two probe-frugality profiles of Table A.1 appear, since §4.4.1 excludes them from the conditioning studies.

[TODO: two further sets of supplementary plots. (i) Per-intent path-choice distributions for the single-path selector. (ii) Multipath training curves — per-episode QoE, loss, and VMAF for the learned pair under each of the three intents — which are the counterpart of Fig. A.1 and are already recorded in the `history` array of every

r1-mpquic/runs//stats.json. The attention-pool seed collapse discussed in §5.3.7 is visible in those curves and is worth showing, since a training curve is the most direct evidence that a training-seed spread is a collapse rather than noise.]*

A.5 Code and Artifact Availability

[TODO: write this section, which the thesis currently lacks entirely. Chapter 5’s tables are scrupulous about citing the artifact behind every number — `r1-mpquic/runs/ab-prof2-ev*-tr*/ab_summary.json` and its siblings appear in six captions — and there is nowhere in the document that says where `r1-mpquic` is. State the repository and the commit hash that produced the reported results, the same for the single-path codebase, and the software versions that matter (Python, PyTorch, NS-3 and `ns3-ai`). Then give the mapping from each table and figure to the run directory backing it; the material already exists in `r1-mpquic/docs/RESULTS.md`, which is organized exactly this way, so this is a transcription rather than an authoring task. Note also which results cannot be reproduced from the artifacts alone — the NS-3 arms need a built simulator, and the checkpoints are not in version control — so the claim made here is accurate. Both examiners and anyone building on this will look for this section first; its absence is the kind of thing that reads as carelessness in a document that is otherwise unusually careful about provenance.]

